System Design Interview - Consolidated Notes

> Consolidated from: https://github.com/presmihaylov/booknotes/tree/master/system-design/system-design-interview

---

System Design Interview - An Insider's Guide (vol 1 & 2)

These notes are based on the System Design Interview books - vol 1 and vol 2.

Instead of getting the physical books, I've bought the online course, so there might be some mismatch with the physical book's chapter indices. In addition to that, there could be some content updates for the online course, but not the physical books.

Note: These notes are a work in progress. I'll remove this remark once I go through the whole book.

- Chapter 2 - Scale From Zero To Millions Of Users
- Chapter 3 - Back-of-the-envelope Estimation
- Chapter 4 - A Framework For System Design Interviews
- Chapter 5 - Design A Rate Limiter
- Chapter 6 - Design Consistent Hashing
- Chapter 7 - Design A Key-Value Store
- Chapter 8 - Design A Unique ID Generator In Distributed Systems
- Chapter 9 - Design A URL Shortener
- Chapter 10 - Design A Web Crawler
- Chapter 11 - Design A Notification System
- Chapter 12 - Design A News Feed System
- Chapter 13 - Design A Chat System
- Chapter 14 - Design A Search Autocomplete System
- Chapter 15 - Design YouTube
- Chapter 16 - Design Google Drive
- Chapter 17 - Proximity Service
- Chapter 18 - Nearby Friends
- Chapter 19 - Google Maps
- Chapter 20 - Distributed Message Queue
- Chapter 21 - Metrics Monitoring And Alerting System
- Chapter 22 - Ad Click Event Aggregation
- Chapter 23 - Hotel Reservation System

- Chapter 24 - Distributed Email Service
- Chapter 25 - S3-like Object Storage
- Chapter 26 - Real-time Gaming Leaderboard
- Chapter 27 - Payment System
- Chapter 28 - Digital Wallet
- Chapter 29 - Stock Exchange

Chapter 2

Scale From Zero to Millions of Users

Here, we're building a system that supports a few users & gradually scale it to support millions.

Single server setup

To start off, we're going to put everything on a single server - web app, database, cache, etc.

Show Image

What's the request flow in there?

- User asks DNS server for the IP of my site (ie `api.mysite.com -> 15.125.23.214`). Usually, DNS is provided by third-parties instead of hosting it yourself.
- HTTP requests are sent directly to server (via its IP) from your device
- Server returns HTML pages or JSON payloads, used for rendering.

Traffic to web server comes from either a web application or a mobile application:

- Web applications use a combo of server-side languages (ie Java, Python) to handle business logic & storage. Client-side languages (ie HTML, JS) are used for presentation.
- Mobile apps use the HTTP protocol for communication between mobile & the web server. JSON is used for formatting transmitted data. Example payload:

```
{
  "id":12,
  "firstName":"John",
  "lastName":"Smith",
  "address":{
     "streetAddress":"21 2nd Street",
     "city":"New York",
     "state":"NY",
     "postalCode":10021
  },
  "phoneNumbers":[
     "212 555-1234",
     "646 555-4567"
  ]
}
```

Database

As the user base grows, storing everything on a single server is insufficient. We can separate our database on another server so that it can be scaled independently from the web tier:

Show Image

Which databases to use?

You can choose either a traditional relational database or a non-relational (NoSQL) one.

- Most popular relational DBs - MySQL, Oracle, PostgreSQL.
- Most popular NoSQL DBs - CouchDB, Neo4J, Cassandra, HBase, DynamoDB

Relational databases represent & store data in tables & rows. You can join different tables to represent aggregate objects.
NoSQL databases are grouped into four categories - key-value stores, graph stores, column stores & document stores. Join operations are generally not supported.

For most use-cases, relational databases are the best option as they've been around the most & have worked quite well historically.

If not suitable though, it might be worth exploring NoSQL databases. They might be a better option if:

- Application requires super-low latency.
- Data is unstructured or you don't need any relational data.

- You only need to serialize/deserialize data (JSON, XML, YAML, etc).
- You need to store a massive amount of data.

Vertical scaling vs. horizontal scaling

Vertical scaling == scale up. This means adding more power to your servers - CPU, RAM, etc.

Horizontal scaling == scale out. Add more servers to your pool of resources.

Vertical scaling is great when traffic is low. Simplicity is its main advantage, but it has limitations:

- It has a hard limit. Impossible to add unlimited CPU/RAM to a single server.
- Lack of fail over and redundancy. If server goes down, whole app/website goes down with it.

Horizontal scaling is more appropriate for larger applications due to vertical scaling's limitations. Its main disadvantage is that it's harder to get right.

In design so far, the server going down (ie due to failure or overload) means the whole application goes down with it.
A good solution for this problem is to use a load balancer.

Load balancer

A load balancer evenly distributes incoming traffic among web servers in a load-balanced set:

Show Image

Clients connect to the public IP of the load balancer. Web servers are unreachable by clients directly.
Instead, they have private IPs, which the load balancer has access to.

By adding a load balancer, we successfully made our web tier more available and we also added possibility for fail over.

How it works?

- If server 1 goes down, all traffic will be routed to server 2. This prevents website from going offline. We'll also add a fresh new server to balance the load.
- If website traffic spikes and two servers are not sufficient to handle traffic, load balancer can handle this gracefully by adding more servers to the pool.

Web tier looks lit now. But what about the data tier?

Database replication

Database replication can usually be achieved via master/slave replication (side note - nowadays, it's usually referred to as primary/secondary replication).

A master database generally only supports writes. Slave databases store copies of the data from the master & only support read operations. This setup works well for most applications as there's usually a higher read to write ratio. Reads can easily be scaled by adding more slave instances.

Show Image

Advantages:

- Better performance - enables more read queries to be processed in parallel.
- Reliability - If one database gets destroyed, data is still preserved.
- High availability - Data is accessible as long as one instance is not offline.

So what if one database goes offline?

- If slave database goes offline, read operations are routed to the master/other slaves temporarily.
- If master goes down, a slave instance will be promoted to the new master. A new slave instance will replace the old master.

  Show Image

Here's the refined request lifecycle:

- user gets IP address of load balancer from DNS
- user connects to load balancer via IP
- HTTP request is routed to server 1 or server 2
- web server reads user data from a slave database instance or routes data modifications to the master instance.

Sweet, let's now improve the load/response time by adding a cache & shifting static content to a CDN.

Cache

Cache is a temporary storage which stores frequently accessed data or results of expensive computations.

In our web application, every time a web page is loaded, expensive queries are sent to the database.
We can mitigate this using a cache.

Cache tier

The cache tier is a temporary storage layer, from which results are fetched much more rapidly than from within a database. It can also be scaled independently from the database.

Show Image

The example above is a read-through cache - server checks if data is available in the cache. If not, data is fetched from the database.

Considerations for using cache

- When to use it - usually useful when data is read frequently but modified infrequently. Caches usually don't preserve data upon restart so it's not a good persistence layer.
- Expiration policy - controls whether (and when) cached data expires and is removed from it. Make it too short - DB will be queried frequently. Make it too long - data will become stale.
- Consistency - How in sync should the data store & cache be? Inconsistency happens if data is changed in DB, but cache is not updated.
- Mitigating failures - A single cache server could be a single point of failure (SPOF). Consider over-provisioning it with a lot of memory and/or provisioning servers in multiple locations.
- Eviction policy - What happens when you want to add items to a cache, but it's full? Cache eviction policy controls that. Common policies - LRU, LFU, FIFO.

Content Delivery Network (CDN)

CDN == network of geographically dispersed servers, used for delivering static content - eg images, HTML, CSS, JS files.

Whenever a user requests some static content, the CDN server closest to the user serves it:

Show Image

Here's the request flow:

Show Image

- User tries fetching an image via URL. URLs are provided by the CDN, eg `https://mysite.cloudfront.net/logo.jpg`
- If the image is not in the cache, the CDN requests the file from the origin - eg web server, S3 bucket, etc.
- Origin returns the image to the CDN with an optional TTL (time to live) parameter, which controls how long that static resource is to be cached.
- Subsequent users fetch the image from the CDN without any requests reaching the origin as long as it's within the TTL.

Considerations of using CDN

- Cost - CDNs are managed by third-parties for which you pay a fee. Be careful not to store infrequently accessed data in there.
- Cache expiry - consider appropriate cache expiry. Too short - frequent requests to origin. Too long - data becomes stale.
- CDN fallback - clients should be able to workaround the CDN provider if there is a temporary outage on their end.
- Invalidation - can be done via an API call or by passing object versions.

Refined design of our web application:

Show Image

Stateless web tier

In order to scale our web tier, we need to make it stateless.

In order to do that, we can store user session data in persistent data storage such as our relational database or a NoSQL database.

Stateful architecture

Stateful servers remember client data across different requests. Stateless servers don't.

Show Image

In the above case, users are coupled to the server which stores their session data. If they make a request to another server, it won't have access to the user's session.

This can be solved via sticky sessions, which most load balancers support, but it adds overhead. Adding/removing servers is much more challenging, which limits our options in case of server failures.

Stateless architecture

Show Image

In this scenario, servers don't store any user data themselves.
Instead, they store it in a shared data store, which all servers have access to.

This way, HTTP requests from users can be served by any web server.

Updated web application architecture:

Show Image

The user session data store could either be a relational database or a NoSQL data store, which is easier to scale for this kind of data.
The next step in the app's evolution is supporting multiple data centers.

Data centers

Show Image

In the above example, clients are geo-routed to the nearest data center based on the IP address.

In the event of an outage, we route all traffic to the healthy data center:

Show Image

To achieve this multi-datacenter setup, there are several issues we need to address:

- traffic redirection - tooling for correctly directing traffic to the right data center. GeoDNS can be used in this case.
- data synchronization - in case of failover, users from DC1 go to DC2. A challenge is whether their user data is there.
- test and deployment - automated deployment & testing is crucial to keep deployments consistent across DCs.

To further scale the system, we need to decouple different system components so they can scale independently.

Message queues

Message queues are durable components, which enable asynchronous communication.

Show Image

Basic architecture:

- Producers create messages.
- Consumers/Subscribers subscribe to new messages and consume them.

Message queues enable producers to be decoupled from consumers.
If a consumer is down, a producer can still publish a message and the consumer will receive it at a later point.

Example use-case in our application - photo processing:

- Web servers publish "photo processing tasks" to a message queue
- A variable number of workers (can be scaled up or down) subscribe to the queue and process those tasks.

Show Image

Logging, metrics, automation

Once your web application grows beyond a given point, investing in monitoring tooling is critical.

- Logging - error logs can be emitted to a data store, which can later be read by service operators.
- Metrics - collecting various types of metrics helps us collect business insight & monitor the health of the system.
- Automation - investing in continuous integration such as automated build, test, deployment can detect various problems early and also increases developer productivity.

Updated system design:

Show Image

Database scaling

There are two approaches to database scaling - vertical and horizontal.

Vertical scaling

Also known as scaling up, it means adding more physical resources to your database nodes - CPU, RAM, HDD, etc.
In Amazon RDS, for example, you can get a database node with 24 TB of RAM.

This kind of database can handle lots of data - eg stackoverflow in 2013 had 10mil monthly unique visitors \w a single database node.

Vertical scaling has some drawbacks, though:

- There are hardware limits to the amount of resources you can add to a node.

- You still have a single point of failure.
- Overall cost is high - the price of powerful servers is high.

Horizontal scaling

Instead of adding bigger servers, you can add more of them:

Show Image

Sharding is a type of database horizontal scaling which separates large data sets into smaller ones.
Each shard shares the same schema, but the actual data is different.

One way to shard the database is based on some key, which is equally distributed on all shards using the modulo operator:

Show Image

Here's how the user data looks like in this example:

Show Image

The sharding key (aka partition key) is the most important factor to consider when using sharding.
In particular, the key should be chosen in a way that distributes the data as evenly as possible.

Although a useful technique, it introduces a lot of complexities in the system:

- Resharding data - you need to do it if a single shard grows too big. This can happen rather quickly if data is distributed unevenly. Consistent hashing helps to avoid moving too much data around.
- Celebrity problem (aka hotspot) - one shard could be accessed much more frequently than others and can lead to server overload. We may have to resort to using separate shards for certain celebrities.
- Join and de-normalization - It is hard to perform join operations across shards. A common workaround is to de-normalize your tables to avoid making joins.

Here's how our application architecture looks like after introducing sharding and a NoSQL database for some of the non-relational data:

Show Image

Millions of users and beyond

Scaling a system is iterative.

What we've learned so far can get us far, but we might need to apply even more sophisticated techniques to scale the application beyond millions of users.

The techniques we saw so far can offer a good foundation to start from.

Here's a summary:

- Keep web tier stateless
- Build redundancy at every layer
- Cache frequently accessed data
- Support multiple data centers
- Host static assets in CDNs
- Scale your data tier via sharding
- Split your big application into multiple services
- Monitor your system & use automation

Chapter 3

Back-of-the-envelope Estimation

You are sometimes asked in a system design interview to estimate performance requirements or system capacity.

These are usually done with thought experiments and common performance numbers, according to Jeff Dean (Google Senior Fellow).

To do this estimation effectively, there are several mechanisms one should be aware of.

Power of two

Data volumes can become enormous, but calculation boils down to basics.

For precise calculations, you need to be aware of the power of two, which corresponds to given data units:

- 2^10 == ~1000 == 1kb
- 2^20 == ~1mil == 1mb
- 2^30 == ~1bil == 1gb
- 2^40 == ~1tril == 1tb
- 2^50 == ~1quad == 1pb

Latency numbers every programmer should know

There's a well-known table of the duration of typical computer operations, created by Jeff Dean.

These might be a bit outdated due to hardware improvements, but they still give a good relative measure among the operations:

- L1 cache reference == 0.5ns
- Branch mispredict == 5ns
- L2 cache reference == 7ns
- Mutex lock/unlock == 100ns
- Main memory reference == 100ns
- Compress 1kb == 10,000ns == 10us
- Send 2kb over 1gbps network == 20,000ns == 20us
- Read 1mb sequentially from memory == 250us
- Round trip within same DC == 500us
- Disk seek == 10ms
- Read 1mb sequentially from network == 10ms
- Read 1mb sequentially from disk == 30ms
- Send packet CA->Netherlands->CA == 150ms

A good visualization of the above:

Show Image

Some conclusions from the above numbers:

- Memory is fast, disk is slow
- Avoid disk seeks if possible

- Compression is usually fast
- Compress data before sending over the network if possible
- Data centers round trips are expensive

Availability numbers

High availability == ability of a system to be continuously operational. In other words, minimizing downtime.

Typically, services aim for availability in the range of 99% to 100%.

An SLA is a formal agreement between a service provider and a customer.
This formally defines the level of uptime your service needs to support.

Cloud providers typically set their uptime at 99.9% or more. Eg AWS EC2 has an SLA of 99.99%

Here's a summary of the allowed downtime based on different SLAs:

Show Image

Example - estimate Twitter QPS and storage requirements

Assumptions:

- 300mil MAU
- 50% of users use twitter daily
- Users post 2 tweets per day on average
- 10% of tweets contain media
- Data is stored for 5y

Estimations:

- Write RPS estimation == 150mil * 2 / 24h / 60m / 60s = 3400-3600 tweets per second, peak=7000 TPS
- Media storage per day == 300mil * 10% == 30mil media storage per day
    - If we assume 1mb per media -> 30mil * 1mb = 30tb per day
    - in 5y -> 30tb * 365 * 5 == 55pb
- tweet storage estimation:
    - 1 tweet = 64byte id + 140 bytes text + 1000 bytes metadata
    - 3500 * 60 * 60 * 24 = 302mb per day
    - In 5y -> 302 * 365 * 5 == 551gb in 5y

Tips

Back-of-the-envelope Estimations are about the process, not the results. Interviewers might test your problem-solving skills.

Some tips to take into consideration:

- Rounding and approximation - don't try to calculate 99987/9.1, round it to 100000/10 instead, which is easier to calculate.
- Write down your assumptions before going forward with estimations
- Label your units explicitly. Write 5mb instead of 5.
- Commonly asked estimations to make - QPS (queries per second), peak QPS, storage, cache, number of servers.

Chapter 4

A Framework for System Design Interviews

System design interviews are intimidating.

How could you possible design a popular system in an hour when it has taken other people decades?

- It is about the problem solving aspect, not the final solution you came up with.
- Goal is to demonstrate your design skills, defend your choices, respond to feedback constructively.

What's the goal of a system design interview?

- It is much more than evaluating a person's technical design skills.
- It is also about evaluating collaboration skills, working under pressure, resolving ambiguity, asking good questions.
- Catching red flags - over-engineering, narrow mindedness, stubborness, etc.

A 4-step process for effective system design interview

Although every interview is different, there are some steps to cover in any interview.

Step 1 - understand the problem and establish design scope

Don't rush to give answers before understanding the problem well enough first.
In fact, this might be a red flag - answering without a thorough understanding of requirements.

Don't jump to solutions. Ask questions to clarify requirements and assumptions.

We have a tendency as engineers to jump ahead and solve a hard problem. But that often leads to designing the wrong system.

When you get your answers (or you're asked to make assumptions yourself), write them down on the whiteboard to not forget.

What kind of questions to ask? - to understand the exact requirements. Some examples:

- What specific features do we need to design?
- How many users does the product have?
- How fast is the company anticipated to scale up? - in 3, 6, 12 months?
- What is the company's tech stack? What existing services can you leverage to simplify the design?

For example, say you have to design a news feed. Here's an example conversation:

- Candidate: Is it mobile, web, both?
- Interviewer: both.
- C: What's the most important features?
- I: Ability to make posts & see friends' news feed.
- C: How is the feed sorted? Just chronologically or based on eg some weight to posts from close friends.
- I: To keep things simple, assume posts are sorted chronologically.
- C: Max friends on a user?
- I: 5000
- C: What's the traffic volume?
- I: 10mil daily active users (DAU)
- C: Is there any media in the feed? - images, video?
- I: It can contain media files, including video & images.

Step 2 - Propose high-level design and get buy-in

The goal of this step is to develop a high-level design, while collaborating with the interviewer.

- Come up with a blueprint, ask for feedback. Many good interviewers involve the interviewer.
- Draw boxes on the whiteboard with key components - clients, APIs, web servers, data stores, cache, cdn, message queue, etc.
- Do back-of-the-envelope calculations to evaluate if the blueprint fits the scale constraints. Communicate with interviewer if this type of estimation is required beforehand.

You could go through some concrete use-cases, which can help you refine the design. It might help you uncover edge cases.

Should we include API schema and database design? Depends on the problem. For larger scale systems, this might be too low level.
Communicate with the interviewer to figure this out.

Example - designing a news feed.

High-level features:

- feed publishing - user creates a post and that post is written in cache/database, after which it gets populated in other news feeds.
- news feed building - news feed is built by aggregating friends' posts in chronological order.

Example diagram - feed publishing:

Show Image

Example diagram - news feed building:

Show Image

Step 3 - Design deep dive

Objectives you should have achieved so far:

- Agreed on overall goals & features
- Sketched out a high-level blueprint of the design
- Obtained feedback about the high-level design
- Received initial ideas on areas you need to focus on in the deep dive

At this stage, you should work with interviewer to prioritize which components you should focus on.

Example things you might have to focus on:

- High-level design.
- System performance characteristics.
- In most cases, dig into the details of some system component.

What details could you dig into? Some examples:

- For URL shortener - the hash function which converts long URLs into small ones
- For Chat system - reducing latency and supporting online/offline status

Time management is essential - don't get carried away with details which don't show off your skills.
For example, don't get carried away about how facebook's EdgeRank algorithm works as it doesn't demonstrate your design skills.

Example design deep dive for feed publishing:

Show Image

Example design deep dive for news feed building:

Show Image

Step 4 - Wrap up

At this stage, the interviewer might ask you some follow-up questions or give you the freedom to discuss anything you want.

A few directions to follow:

- Identify system bottlenecks and discuss improvements. No design is perfect, there are always things to improve.
- Give a recap of your design.
- Failure modes - server failure, network loss, etc.
- Operational issues - monitoring, alerting, rolling out the system.
- What needs to change to support the next scale curve? Eg 1mil -> 10mil users
- Propose other refinements.

Do's:

- Ask for clarification, don't assume your assumption is correct.
- Understand problem requirements.

- There is no right or perfect answer. A solution for a small company is different from one for a larger one.
- Let interviewer know what you're thinking.
- Suggest multiple approaches.
- Agree on blueprint and then design the most critical components first.
- Share ideas with interviewer. They should work with you.

Dont's:

- Come unprepared for typical interview questions.
- Jump into a solution without understanding the requirements first.
- Don't go into too much details on a single component at first. Design at a high-level initially.
- Feel free to ask for hints if you get stuck.
- Communicate, don't think silently.

Time allocation on each step

45 minutes are not enough to cover any design into sufficient detail.

Here's a rough guide on how much time you should spend on each step:

- Understand problem & establish design scope - 3-10m
- Propose high-level design & get buy-in - 10-15m
- Design deep dive - 10-25m
- Wrap-up - 3-5m

Chapter 5

Design a Rate Limiter

The rate limiter's purpose in a distributed system is to control the rate of traffic sent from clients to a given server.

It controls the maximum number of requests allowed in a given time period.
If the number of requests exceeds the threshold, the extra requests are dropped by the rate limiter.

Examples:

- User can write no more than 2 posts per second.
- You can create 10 accounts max per day from the same IP.
- You can claim rewards max 10 times per week.

Almost all APIs have some sort of rate limiting - eg Twitter allows 300 tweets per 3h max.

What are the benefits of using a rate limiter?

- Prevents DoS attacks.
- Reduces cost - fewer servers are allocated to lower priority APIs. Also, you might have a downstream dependency which charges you on a per-call basis, eg making a payment, retrieving health records, etc.
- Prevents servers from getting overloaded.

Step 1 - Understand the problem and establish design scope

There are multiple techniques which you can use to implement a rate limiter, each with its pros and cons.

Example Candidate-Interviewer conversation:

- C: What kind of rate limiter are we designing? Client-side or server-side?
- I: Server-side
- C: Does the rate limiter throttle API requests based on IP, user ID or anything else?
- I: The system should be flexible enough to support different throttling rules.
- C: What's the scale of the system? Startup or big company?
- I: It should handle a large number of requests.
- C: Will the system work in a distributed environment?
- I: Yes.
- C: Should it be a separate service or a library?
- I: Up to you.
- C: Do we need to inform throttled users?
- I: Yes.

Summary of requirements:

- Accurately limit excess requests
- Low latency & as little memory as possible
- Distributed rate limiting.
- Exception handling
- High fault tolerance - if cache server goes down, rate limiter should continue functioning.

Step 2 - Propose high-level design and get buy-in

We'll stick with a simple client-server model for simplicity.

Where to put the rate limiter?

It can be implemented either client-side, server-side or as a middleware.

Client-side - Unreliable, because client requests can easily be forged by malicious actors. We also might not have control over client implementation.

Server-side:

Show Image

As a middleware between client and server:

Show Image

How it works, assuming 2 requests per second are allowed:

Show Image

In cloud microservices, rate limiting is usually implemented in the API Gateway.
This service supports rate limiting, ssl termination, authentication, IP whitelisting, serving static content, etc.

So where should the rate limiter be implemented? On the server-side or in the API gateway?

It depends on several things:

- Current tech stack - if you're implementing it server-side, then your language should be sufficient enough to support it.
- If implemented on the server-side, you have control over the rate limiting algorithm.
- If you already have an API gateway, you might as well add the rate limiter in there.
- Building your own rate limiter takes time. If you don't have sufficient resources, consider using an off-the-shelf third-party solution instead.

Algorithms for rate limiting

There are multiple algorithms for rate limiting, each with its pros and cons.

Some of the popular algorithms - token bucket, leaking bucket, fixed window counter, sliding window log, sliding window counter.

Token bucket algorithm

Simple, well understood and commonly used by popular companies. Amazon and Stripe use it for throttling their APIs.

Show Image

It works as follows:

- There's a container with predefined capacity
- Tokens are periodically put in the bucket
- Once full, no more tokens are added
- Each request consumes a single token
- If no tokens left, request is dropped

Show Image

There are two parameters for this algorithm:

- Bucket size - maximum number of tokens allowed in the bucket
- Refill rate - number of tokens put into the bucket every second

How many buckets do we need? - depends on the requirements:

- We might need different buckets per API endpoints if we need to support 3 tweets per second, 5 posts per second, etc.
- Different buckets per IP if we want to make IP-based throttling.
- A single global bucket if we want to globally setup 10k requests per second max.

Pros:

- Easy to implement
- Memory efficient

- Throttling gets activated in the event of sustained high traffic only. If bucket size is large, this algorithm supports short bursts in traffic as long as they're not prolonged.

Cons:

- Parameters might be challenging to tune properly

Leaking bucket algorithm

Similar to token bucket algorithm, but requests are processed at a fixed rate.

How it works:

- When request arrives, system checks if queue is full. If not, request is added to the queue, otherwise, it is dropped.
- Requests are pulled from the queue and processed at regular intervals.

Show Image

Parameters:

- Bucket size - aka the queue size. It specifies how many requests will be held to be processed at fixed intervals.
- Outflow rate - how many requests to be processed at fixed intervals.

Shopify uses leaking bucket for rate-limiting.

Pros:

- Memory efficient
- Requests processed at fixed interval. Useful for use-cases where a stable outflow rate is required.

Cons:

- A burst of traffic fills up the queue with old requests. Recent requests will be rate limited.
- Parameters might not be easy to tune.

Fixed window counter algorithm

How it works:

- Time is divided in fix windows with a counter for each one
- Each request increments the counter

- Once the counter reaches the threshold, subsequent requests in that window are dropped

Show Image

One major problem with this approach is that a burst of traffic in the edges can allow more requests than allowed to pass through:

Show Image

Pros:

- Memory efficient
- Easy to understand
- Resetting available quota at the end of a unit of time fits certain use cases

Cons:

- Spike in traffic could cause more requests than allowed to go through a given time window

Sliding window log algorithm

To resolve the previous algorithm's issue, we could use a sliding time window instead of a fixed one.

How it works:

- Algorithm keeps track of request timestamps. Timestamp data is usually kept in a cache, such as Redis sorted set.
- When a request comes in, remove outdated timestamps.
- Add timestamp of the new request in the log.
- If the log size is same or lower than threshold, request is allowed, otherwise, it is rejected.

Note that the 3rd request in this example is rejected, but timestamp is still recorded in the log:

Show Image

Pros:

- Rate limiting accuracy is very high

Cons:

- Memory footprint is very high

Sliding window counter algorithm

A hybrid approach which combines the fixed window + sliding window log algorithms.

Show Image

How it works:

- Maintain a counter for each time window. Increment for given time window on each request.
- Derive sliding window counter = `prev_window * prev_window_overlap + curr_window * curr_window_overlap` (see screenshot above)
- If counter exceeds threshold, request is rejected, otherwise it is accepted.

Pros:

- Smooths out spikes in traffic as rate is based on average rate of previous window
- Memory efficient

Cons:

- Not very accurate rate limiting, as it's based on overlaps. But experiments show that only ~0.003% of requests are inaccurately accepted.

High-level architecture

We'll use an in-memory cache as it's more efficient than a database for storing the rate limiting buckets - eg Redis.

Show Image

How it works:

- Client sends request to rate limiting middleware
- Rate limiter fetches counter from corresponding bucket & checks if request is to be let through
- If request is let through, it reaches the API servers

Step 3 - Design deep dive

What wasn't answered in the high-level design:

- How are rate limiting rules created?
- How to handle rate-limited requests?

Let's check those topics out, along with some other topics.

Rate limiting rules

Example rate limiting rules, used by Lyft for 5 marketing messages per day:

Show Image

Another example \w max login attempts in a minute:

Show Image

Rules like these are generally written in config files and saved on disk.

Exceeding the rate limit

When a request is rate limited, a 429 (too many requests) error code is returned.

In some cases, the rate-limited requests can be enqueued for future processing.

We could also include some additional HTTP headers to provide additional metadata info to clients:

```
X-Ratelimit-Remaining: The remaining number of allowed requests within the window.
X-Ratelimit-Limit: It indicates how many calls the client can make per time
window.
X-Ratelimit-Retry-After: The number of seconds to wait until you can make a
request again without being throttled.
```

Detailed design

Show Image

- Rules are stored on disk, workers populate them periodically in an in-memory cache.
- Rate limiting middleware intercepts client requests.
- Middleware loads the rules from the cache. It also fetches counters from the redis cache.
- If request is allowed, it proceeds to API servers. If not, a 429 HTTP status code is returned. Then, request is either dropped or enqueued.

Rate limiter in a distributed environment

How will we scale the rate limited beyond a single server?

There are several challenges to consider:

- Race condition
- Synchronization

In case of race conditions, the counter might not be updated correctly when mutated by multiple instances:

Show Image

Locks are a typical way to solve this issue, but they are costly.
Alternatively, one could use Lua scripts or Redis sorted sets, which solve the race conditions.

If we maintain user information within the application memory, the rate limiter is stateful and we'll need to use sticky sessions to make sure requests from the same user is handled by the same rate limiter instance.

Show Image

To solve this issue, we can use a centralized data store (eg Redis) so that the rate limiter instances are stateless.

Show Image

Performance optimization

There are two things we can do as a performance optimization for our rate limiters:

- Multi-data center setup - so that users interact with instances geographically close to them.
- Use eventual consistency as a synchronization model to avoid excessive locking.

Monitoring

After the rate limiter is deployed, we'd want to monitor if it's effective.

To do so, we need to track:

- If the rate limiting algorithm is effective
- If the rate limiting rules are effective

If too many requests are dropped, we might have to tune some of the rules or the algorithm parameters.

Step 4 - Wrap up

We discussed a bunch of rate-limiting algorithms:

- Token bucket - good for supporting traffic bursts.
- Leaking bucket - good for ensuring consistent inbound request flow to downstream services
- Fixed window - good for specific use-cases where you want time divided in explicit windows
- Sliding window log - good when you want high rate-limiting accuracy at the expense of memory footprint.

- Sliding window counter - good when you don't want 100% accuracy with a very low memory footprint.

Additional talking points if time permits:

- Hard vs. soft rate limiting
  - Hard - requests cannot exceed the specified threshold
  - Soft - requests can exceed threshold for some limited time
- Rate limiting at different layers - L7 (application) vs L3 (network)
- Client-side measures to avoid being rate limited:
  - Client-side cache to avoid excessive calls
  - Understand limit and avoid sending too many requests in a small time frame
  - Gracefully handle exceptions due to being rate limited
  - Add sufficient back-off and retry logic

Chapter 6

Design Consistent Hashing

For horizontal scaling, it is important to distribute requests across servers efficiently.

Consistent hashing is a common approach to achieve this.

The rehashing problem

One way to determine which server a request gets routed to is by applying a simple hash+module formula:

```
serverIndex = hash(key) % N, where N is the number of servers
```

This makes it so requests are distributed uniformly across all servers.
However, whenever new servers are added or removed, the result of the above equation is very different, meaning that a lot of requests will get rerouted across servers.

This causes a lot of cache misses as clients will be connected to new instances which will have to fetch the user data from cache all over again.

Consistent hashing

Consistent hashing is a technique which allows only a K/N servers to be remapped whenever N changes, where K is the number of keys.

For example, K=100, N=10 -> 10 re-mappings, compared to close to 100 in the normal scenario.

Hash space and hash ring

A hash ring is a visualization of the possible key space of a given hash algorithm, which is combined into a ring-like structure:

Show Image

Hash servers

Using the same hash function for the requests, we map the servers based on server IP or name onto the hash ring:

Show Image

Hash keys

The hashes of the requests also get resolved somewhere along the hash ring. Notice that we're not using the modulo operator in this case:

Show Image

Server lookup

Now, to determine which server is going to serve each request, we go clockwise from the request's hash until we reach the first server hash:

Show Image

Add a server

Via this approach, adding a new server causes only one of the requests to get remapped to a new server:

Show Image

Remove a server

Likewise, removing a server causes only a single request to get remapped:

Show Image

Two issues in the basic approach

The first problem with this approach is that hash partitions can be uneven across servers:

Show Image

The second problem derives from the first - it is possible that requests are unevenly distributed across servers:

Show Image

Virtual nodes

To solve this issue, we can map a servers on the hash ring multiple times, creating virtual nodes and assigning multiple partitions to the same server:

Show Image

Now, a request is mapped to the closest virtual node on the hash ring:

Show Image

The more virtual nodes we have, the more evenly distributed the requests will be.

An experiment showed that between 100-200 virtual nodes leads to a standard deviation between 5-10%.

Wrap up

Benefits of consistent hashing:

- Very low number of keys are distributed in a re-balancing event
- Easy to scale horizontally as data is uniformly distributed
- Hotspot issue is mitigated by uniformly distributing data, related to eg a celebrity, which is often accessed

Examples of real-world applications of consistent hashing:

- Amazon's DynamoDB partitioning component
- Data partitioning in Cassandra
- Discord chat application
- Akamai CDN
- Maglev network load balancer

Chapter 7

Design a Key-Value Store

Key-value stores are a type of non-relational databases.

- Each unique identifier is stored as a key with a value associated to it.
- Keys must be unique and can be plain text or hashes.
- Performance-wise, short keys work better.

Example keys:

- plain-text - "last_logged_in_at"
- hashed key - `253DDEC4`

We're now about to design a key-value store which supports:

- `put(key, value)` - insert `value` associated to `key`
- `get(key)` - get `value` associated to `key`

Understand the problem and establish design scope

There's always a trade-off to be made between read/write and memory usage.
Another trade-off is between consistency and availability.

Here are the characteristics we're striving to achieve:

- Key-value pair size is small - 10kb
- We need to be able to store a lot of data.
- High availability - system responds quickly even during failures.
- High scalability - system can be scaled to support large data sets.
- Automatic scaling - addition/deletion of servers should happen automatically based on traffic.
- Tunable consistency.
- Low latency.

Single server key-value store

Single server key-value stores are easy to develop.

We can just maintain an in-memory hash map which stores the key-value pairs.

Memory however, can be a bottleneck, as we can't fit everything in-memory. Here are our options to scale:

- Data compression
- Store only frequently used data in-memory. The rest store on disk.

Even with these optimizations, a single server can quickly reach its capacity.

Distributed key-value store

A distributed key-value store consists of a distributed hash table, which distributes keys across many nodes.

When developing a distributed data store, we need to factor in the CAP theorem

CAP Theorem

This theorem states that a data store can't provide more than two of the following guarantees - consistency, availability, partition tolerance;

- Consistency - all clients see the same data at the same time, no matter which node they're connected to.
- Availability - all clients get a response, regardless of which node they connect to.
- Partition tolerance - A network partition means that not all nodes within the cluster can communicate. Partition tolerance means that the system is operational even in such circumstances.

  Show Image

A distributed system which supports consistency and availability cannot exist in the real world as network failures are inevitable.

Example distributed data store in an ideal situation:

Show Image

In the real world, a network partition can occur which hinders communication with eg node 3:

Show Image

If we favor consistency over availability, all write operations need to be blocked when the above scenario occurs.

If we favor availability on the other hand, the system continues accepting reads and writes, risking some clients receiving stale data.
When node 3 is back online, it will be re-synced with the latest data.

What you choose is something you need to clarify with the interviewer. There are different trade-offs with each option.

System components

This section goes through the key components needed to build a distributed key-value store.

Data partition

For a large enough data set, it is infeasible to maintain it on a single server.

Hence, we can split the data into smaller partitions and distribute them across multiple nodes.

The challenge then, is to distribute data evenly and minimize data movement when the cluster is resized.

Both these problems can be addressed using consistent hashing (discussed in previous chapter):

- Servers are put on a hash ring
- Keys are hashed and put on the closest server in clockwise direction

Show Image

This has the following advantages:

- Automatic scaling - servers can be added/removed at will with minimal impact on key location
- Heterogeneity - servers with higher capacity can be allocated with more virtual nodes

Data replication

To achieve high availability & reliability, data needs to be replicated on multiple nodes.

We can achieve that by allocating a key to multiple nodes on the hash ring:

Show Image

One caveat to keep in mind that your key might get allocated to virtual nodes mapped to the same physical node.
To avoid this, we can only choose unique physical nodes when replicating data.

An additional reliability measure is to replicate data across multiple data centers as nodes in the same data centers can fail at the same time.

Consistency

Since data is replicated, it must be synchronized.

Quorum consensus can guarantee consistency for both reads and writes:

- N - number of replicas
- W - write quorum. Write operations must be acknowledged by W nodes.

- R - read quorum. Read operations must be acknowledged by R nodes.

  Show Image

The configuration of W and R is a trade-off between latency and consistency.

- W = 1, R = 1 -> low latency, eventual consistency
- W + R > N -> strong consistency, high latency

Other configurations:

- R = 1, W = N -> strong consistency, fast reads, slow writes
- R = N, W = 1 -> strong consistency, fast writes, slow reads

Consistency models

There are multiple flavors of consistency we can tune our key-value store for:

- Strong consistency - read operations return a value corresponding to most up-to-date data. Clients never see stale data.
- Weak consistency - read operations might not see the most up-to-date data.
- Eventual consistency - read operations might not see most up-to-date data, but with time, all keys will converge to the latest state.

Strong consistency usually means a node not accepting reads/writes until all nodes have acknowledged a write.
This is not ideal for highly available systems as it can block new operations.

Eventual consistency (supported by Cassandra and DynamoDB) is preferable for our key-value store.
This allows concurrent writes to enter the system and clients need to reconcile the data mismatches.

Inconsistency resolution: versioning

Replication provides high availability, but it leads to data inconsistencies across replicas.

Example inconsistency:

Show Image

This kind of inconsistency can be resolved using a versioning system using a vector clock. A vector clock is a [server, version] pair, associated with a data item. Each time a data item is changed in a server, it's associated vector clock changes to [server_id, curr_version+1].

Example inconsistency resolution:

Show Image

- Client writes D1, handled by Sx, which writes version [Sx, 1]
- Another client reads D1, updates it and Sx increments version to [Sx, 2]
- Client writes D3 based on D2 in Sy -> D3([Sx, 2][Sy, 1]).
- Simultaneously, another one writes D4 in Sz -> D4([Sx, 2][Sz, 1])
- A client reads D3 and D4 and detects a conflict. It makes a resolution and adds the updated version in Sx -> D5([Sx, 3][Sy, 1][Sz, 1])

A conflict is detected by checking whether a version is an ancestor of another one. That can be done by verifying that all version stamps are less than or equal to the other one.

- [s0, 1][s1, 1] is an ancestor of [s0, 1][s1, 2] -> no conflict.
- [s0, 1] is not an ancestor of [s1, 1] -> conflict needs to be resolved.

This conflict resolution technique comes with trade-offs:

- Client complexity is increased as clients need to resolve conflicts.
- Vector clocks can grow rapidly, increasing the memory footprint of each key-value pair.

Handling failures

At a large enough scale, failures are inevitable. It is important to determine your error detection & resolution strategies.

Failure detection

In a distributed system, it is insufficient to conclude that a server is down just because you can't reach it. You need at least another source of information.

One approach to do that is to use all-to-all multi-casting. This, however, is inefficient when there are many servers in the system.

Show Image

A better solution is to use a decentralized failure detection mechanism, such as a gossip protocol:

- Each node maintains a node membership list with member IDs and heartbeat counters.
- Each node periodically increments its heartbeat counter
- Each node periodically sends heartbeats to a random set of other nodes, which propagate it onwards.
- Once a node receives a heartbeat, its membership list is updated.
- If a heartbeat is not received after a given threshold, the member is marked offline.

Show Image

In the above scenario, s0 detects that s2 is down as no heartbeat is received for a long time.
It propagates that information to other nodes which also verify that the heartbeat hasn't been updated.
Hence, s2 is marked offline.

Handling temporary failures

A nice little trick to improve availability in the event of failures is hinted handoff.

What it means is that if a server if temporarily offline, you can promote another healthy service in its place to process data temporarily.
After the server is back online, the data & control is handed back to it.

Handling permanent failures

Hinted handoff is used when the failure is intermittent.

If a replica is permanently unavailable, we implement an anti-entropy protocol to keep the replicas in-sync.

This is achieved by leveraging merkle trees in order to reduce the amount of data transmitted and compared to a minimum.

The merkle tree works by building a tree of hashes where leaf nodes are buckets of key-value pairs.

If any of the buckets across two replicas is different, then the merkle tree's hashes will be different all the way to the root:

Show Image

Using merkle trees, two replicas will compare only as much data as is different between them, instead of comparing the entire data set.

Handling data center outage

A data center outage could happen due to a natural disaster or serious hardware failure.

To ensure resiliency, make sure your data is replicated across multiple data centers.

System architecture diagram

Show Image

Main features:

- Clients communicate with the key-value store through a simple API
- A coordinator is a proxy between the clients and the key-value store
- Nodes are distributed on the ring using consistent hashing
- System is decentralized, hence adding and removing nodes is supported and can be automated
- Data is replicated at multiple nodes
- There is no single point of failure

Some of the tasks each node is responsible for:

Show Image

Write path

Show Image

- Write requests are persisted in a commit log
- Data is saved in the memory cache
- When memory cache is full or reaches a given threshold, data is flushed to an SSTable on disk

SSTable == Sorted String Table. Holds a sorted list of key-value pairs.

Read path

Read path when data is in memory:

Show Image

Read path when data is not in memory:

Show Image

- If data is in memory, fetch it from there. Otherwise, find it in the SSTable.
- A bloom filter is used for efficient lookup in the SSTable.
- The SSTables returns the resulting data, which is returned to the client

Summary

We covered a lot of concepts and techniques, here's a summary:

Goal/Problems	Technique
Ability to store big data	Use consistent hashing to spread load across servers
High availability reads	Data replication Multi-datacenter setup
Highly available writes	Versioning and conflict resolution with vector clocks
Dataset partition	Consistent Hashing
Incremental scalability	Consistent Hashing
Heterogeneity	Consistent Hashing
Tunable consistency	Quorum consensus
Handling temporary failures	Sloppy quorum and hinted handoff
Handling permanent failures	Merkle tree
Handling data center outage	Cross-datacenter replication

Chapter 8

Design a Unique ID Generator in Distributed Systems

We need to design a unique ID generator, compatible with distributed systems.

A primary key with auto_increment won't work here, because generating IDs across multiple database servers has high latency.

Step 1 - Understand the problem and establish design scope

- C: What characteristics should the unique IDs have?
- I: They should be unique and sortable.
- C: For each record, does the ID increment by 1?
- I: IDs increment by time, but not necessarily by 1.
- C: Do IDs contain only numerical values?
- I: Yes
- C: What is the ID length requirement?
- I: 64 bits
- C: What's the system scale?
- I: We should be able to generate 10,000 IDs per second

Step 2 - Propose high-level design and get buy-in

Here's the options we'll consider:

- Multi-master replication
- Universally-unique IDs (UUIDs)
- Ticket server
- Twitter snowflake approach

Multi-master replication

Show Image

This uses the database's auto_increment feature, but instead of increasing by 1, we increase by K where K = number of servers.

This solves the scalability issues as id generation is confined within a single server, but it introduces other challenges:

- Hard to scale \w multiple data centers
- IDs do not go up in time across servers
- Adding/removing servers breaks this mechanism

UUID

A UUID is a 128-byte unique ID.

The probability of UUID collision across the whole world is very little.

Example UUID - `09c93e62-50b4-468d-bf8a-c07e1040bfb2`.

Pros:

- UUIDs can be generated independently across servers without any synchronization or coordination.
- Easy to scale.

Cons:

- IDs are 128 bytes, which doesn't fit our requirement
- IDs do not increase with time
- IDs can be non-numeric

Ticket server

A ticket server is a centralized server for generating unique primary keys across multiple services:

Show Image

Pros:

- Numeric IDs
- Easy to implement & works for small & medium applications

Cons:

- Single point of failure.
- Additional latency due to network call.

Twitter snowflake approach

Twitter's snowflake meets our design requirements because it is sortable by time, 64-bits and can be generated independently in each server.

Show Image

Breakdown of the different sections:

- Sign bit - always 0. Reserved for future use.
- Timestamp - 41 bits. Milliseconds since epoch (or since custom epoch). Allows 69 years max.
- Datacenter ID - 5 bits, which enables 32 data centers max.
- Machine ID - 5 bits, which enables 32 machines per data center.
- Sequence number - For every generated ID, the sequence number is incremented. Reset to 0 on every millisecond.

Step 3 - Design deep dive

We'll use twitter's snowflake algorithm as it fits our needs best.

Datacenter ID and machine ID are chosen at startup time. The rest is determined at runtime.

Step 4 - wrap up

We explored multiple ways to generate unique IDs and settled on snowflake eventually as it serves our purpose best.

Additional talking points:

- Clock synchronization - network time protocol can be used to resolve clock inconsistencies across different machines/CPU cores.
- Section length tuning - we could sacrifice some sequence number bits for more timestamp bits in case of low concurrency and long-term applications.
- High availability - ID generators are a critical component and must be highly available.

Chapter 9

Design a URL Shortener

We're tackling a classical system design problem - designing a URL shortening service like tinyurl.

Step 1 - Understand the problem and establish design scope

- C: Can you give an example of how a URL shortening service works?
- I: Given URL `https://www.systeminterview.com/q=chatsystem&c=loggedin&v=v3&l=long` and alias `https://tinyurl.com/y7keocwj`. You open the alias and get to the original URL.
- C: What is the traffic volume?
- I: 100 million URLs are generated per day.
- C: How long is the shortened URL?
- I: As short as possible
- C: What characters are allowed?
- I: numbers and letters
- C: Can shortened URLs be updated or deleted?
- I: For simplicity, let's assume they can't.

Other functional requirements - high availability, scalability, fault tolerance.

Back of the envelope calculation

- 100 mil URLs per day -> ~1200 URLs per second.

- Assuming read-to-write ratio of 10:1 -> 12000 reads per second.
- Assuming URL shortener will run for 10 years, we need to support 365bil records.
- Average URL length is 100 characters
- Storage requirements for 10y - 36.5 TB

Step 2 - Propose high-level design and get buy-in

API Endpoints

We'll make a REST API.

A URL shortening service needs two endpoints:

- `POST api/v1/data/shorten` - accepts long url and returns a short one.
- `GET api/v1/shortURL` - return long URL for HTTP redirection.

URL Redirecting

How it works:

Show Image

What's the difference between 301 and 302 statuses?

- 301 (Permanently moved) - indicates that the URL permanently points to the new URL. This instructs the browser to bypass the tinyurl service on subsequent calls.
- 302 (Temporarily moved) - indicates that the URL is temporarily moved to the new URL. Browser will not bypass the tinyurl service on future calls.

Choose 301 if you want to avoid extra server load. Choose 302 if tracking analytics is important.

Easiest way to implement the URL redirection is to store the `<shortURL, longURL>` pair in an in-memory hash-table.

URL Shortening

To support the URL shortening, we need to find a suitable hash function.

It needs to support hashing long URL to shortURL and mapping them back.

Details are discussed in the detailed design.

Step 3 - Design deep dive

We'll explore the data model, hash function, URL shortening and redirection.

Data model

In the simplified version, we're storing the URLs in a hash table. That is problematic as we'll run out of memory and also, in-memory doesn't persist across server reboot.

That's why we can use a simple relational table instead:

Show Image

Hash function

The hash value consists of characters `[0-9a-zA-Z]`, which gives a max of 62 characters.

To figure out the smallest hash value we can use, we need to calculate n in `62^n >= 365bil` -> this results in `n=7`, which can support ~3.5 trillion URLs.

For the hash function itself, we can either use `base62 conversion` or `hash + collision detection`.

In the latter case, we can use something like MD-5 or SHA256, but only taking the first 7 characters. To resolve collisions, we can reiterate \w an some padding to input string until there is no collision:

Show Image

The problem with this method is that we have to query the database to detect collision. Bloom filters could help in this case.

Alternatively, we can use base62 conversion, which can convert an arbitrary ID into a string consisting of the 62 characters we need to support.

Comparison between the two approaches:

Hash + collision resolution	Base 62 conversion
Fixed short URL length.	Short URL length is not fixed. It goes up with the ID.
Does not need a unique ID generator.	This option depends on a unique ID generator.
Collision is possible and needs to be resolved.	Collision is not possible because ID is unique.
It’s not possible to figure out the next available short URL because it doesn’t depend on ID.	It is easy to figure out what is the next available short URL if ID increments by 1 for a new entry. This can be a security concern.

URL shortening deep dive

To keep our service simple, we'll use base62 encoding for the URL shortening.

Here's the whole workflow:

Show Image

To ensure our ID generator works in a distributed environment, we can use Twitter's snowflake algorithm.

URL redirection deep dive

We've introduced a cache as there are more reads than writes, in order to improve read performance:

Show Image

- User clicks short URL
- Load balancer forwards the request to one of the service instances
- If shortURL is in cache, return the longURL directly
- Otherwise, fetch the longURL from the database and store in cache. If not found, then the short URL doesn't exist

Step 4 - Wrap up

We discussed:

- API design
- data model
- hash function
- URL shortening
- URL redirecting

Additional talking points:

- Rate limiter - We can introduce a rate limiter to protect us against malicious actors, trying to make too many URL shortening requests.
- Web server scaling - We can easily scale the web tier by introducing more service instances as it's stateless.
- Database scaling - Replication and sharding are a common approach to scale the data layer.
- Analytics - Integrating analytics tracking in our URL shortener service can reap some business insights for clients such as "how many users clicked the link".
- Availability, consistency, reliability - At the core of every distributed systems. We'd leverage concepts already discussed in Chapter 02.

Chapter 10

Design a Web Crawler

We'll focus next on designing a web crawler - a classical system design problem.

Web crawlers (aka robots) are used to discover new or updated content on the web, such as articles, videos, PDFs, etc.

Show Image

Use-cases:

- Search engine indexing - for creating a local index of a search engine, eg Google's Googlebot.
- Web archiving - collect data from the web and preserve it for future uses.

- Web mining - it can also be used for data mining. Eg finding important insights such as shareholder meetings for trading firms.
- Web monitoring - monitor the internet for copyright infringements or eg company internal information leaks.

The complexities of building a web crawler depend on our target scale. It can be very simple (eg a student project) or a multi-year project, maintained by a dedicated team.

Step 1 - Understand the problem and establish design scope

How it works at a high-level:

- Given a set of URLs, download all the pages these URLs point to
- Extract URLs from the web pages
- Add the new URLs to the list of URLs to be traversed

A real web crawler is much more complicated, but this is what it does in a nutshell.

You'll need to clarify what kind of features your interviewer would like you to support exactly:

- C: What's the main purpose of the web crawler? Search engine indexing, data mining, something else?
- I: Search engine indexing
- C: How many web pages does it collect per month
- I: 1 billion
- C: What content types are included? HTML, PDF, images?
- I: HTML only
- C: Should we consider newly added/edited content?
- I: Yes
- C: Do we need to persist the crawled web pages?
- I: Yes, for 5 years
- C: What do we do with pages with duplicate content
- I: Ignore them

This is an example conversation. It is important to go through this even if the project is simple. Your assumptions and the ones of your interviewer could differ.

Other characteristics of a good web crawler:

- Scalable - it should be extremely efficient
- Robust - handle edge-cases such as bad HTML, infinite loops, server crashes, etc
- Polite - not make too many requests to a server within a short time interval

- Extensibility - it should be easy to add support for new types of content, eg images in the future

Back of the envelope estimation

Given 1 billion pages per month -> ~400 pages per second
Peak QPS = 800 pages per second

Given average web page size is 500kb -> 500 TB per month -> 30 PB for 5y.

Step 2 - Propose high-level design and get buy-in

Show Image

What's going on in there?

- Seed URLs - These URLs are the starting point for crawlers. It's important to pick the seed URLs well in order to traverse the web appropriately.
- URL Frontier - This component stores the URLs to be downloaded in a FIFO queue.
- HTML Downloader - component downloads HTML pages from URLs in the frontier.
- DNS Resolver - Resolve the IP for a given URL's domain.
- Content Parser - validate that the web page is ok. You don't want to store & process malformed web pages.
- Content Seen? - component eliminates pages which have already been processed. This compares the content, not the URLs. Efficient way to do it is by comparing web page hashes.
- Content Storage - Storage system for HTML documents. Most content is stored on disk \w most popular ones in memory for fast retrieval.
- URL Extractor - component which extracts links from an HTML document.
- URL Filter - exclude URLs which are invalid, unsupported content types, blacklisted, etc.
- URL Seen? - component which keeps track of visited URLs to avoid traversing them again. Bloom filters are an efficient way to implement this component.
- URL Storage - Store already visited URLs.

Those are all the component, but what about the workflow?

Show Image

1. Add Seed URLs to URL Frontier
2. HTML Downloader fetches a list of URLs from frontier
3. Match URLs to IP Addresses via the DNS resolver
4. Parse HTML pages and discard if malformed
5. Once validated, content is passed to "Content Seen?"
6. Check if HTML page is already in storage. If yes - discard. If no - process.
7. Extract links from HTML page
8. Pass extracted links to URL Filter
9. Pass filtered links to "URL Seen?" component
10. If URL is in storage - discard. Otherwise - process.
11. If URL is not processed before, it is added to URL Frontier

Step 3 - Design Deep Dive

Let's now explore some of the most important mechanisms in the web crawler:

- DFS vs. BFS
- URL frontier
- HTML Downloader
- Robustness
- Extensibility
- Detect and avoid problematic content

DFS vs. BFS

The web is a directed graph, where the links in a web page are the edges to other pages (the nodes).

Two common approaches for traversing this data structure are DFS and BFS.
DFS is usually not a good choice as the traversal depth can get very big

BFS is typically preferable. It uses a FIFO queue which traverses URLs in order of encountering them.

There are two problems with traditional BFS though:

- Most links in a page are backlinks to the same domain, eg wikipedia.com/page -> wikipedia.com. When a crawler attempts to visit those links, the server is flooded with requests which is "impolite".
- Standard BFS doesn't take URL priority into account

URL Frontier

The URL Frontier helps address these problems. It prioritizes URLs and ensures politeness.

Politeness

A web crawler should avoid sending too many requests to the same host in a short time frame as it can cause excessive traffic to traversed website.

Politeness is implemented by maintaining a download queue per hostname \w a delay between element processing:

Show Image

- The queue router ensures that each queue contains URLs from the same host.
- Mapping table - maps each host to a queue.

  Show Image

- FIFO queues maintain URLs belonging to the same host.
- Queue selector - Each worker thread is mapped to a FIFO queue and only downloads URLs from that queue. Queue selector chooses which worker processes which queue.
- Worker thread 1 to N - Worker threads download web pages one by one from the same host. Delay can be added between two download tasks.

Priority

We prioritize URLs by usefulness, which can be determined based on PageRank web traffic, update frequency, etc.

Prioritizer manages the priority for each URL:

Show Image

- Takes URLs as input and calculates priority
- Queues have different priorities. URLs are put into respective queue based on its priority.
- Queue selector - randomly choose queue to select from with bias towards high-priority ones.

Freshness

Web pages are constantly updated. We need to periodically recrawl updated content.

We can choose to recrawl based on the web page's update history. We can also prioritize recrawling important pages which are updated first.

Storage for URL Frontier

In the real world, the URLs in the frontier can be millions. Putting everything in-memory is infeasible.
But putting it on disk is also slow and can cause a bottleneck for our crawling logic.

We've adopted a hybrid approach where most URLs are on disk, but we maintain a buffer in-memory with URLs which are currently processed.
We periodically flush that to disk.

HTML Downloader

This component downloads HTML pages from the web using the HTTP protocol.

One protocol we need to also bear in mind is the Robots Exclusion Protocol.

It is a `robots.txt` file, available on websites, which website owners use to communicate with web crawlers. It is used to communicate which web pages are ok to be traversed and which ones should be skipped.

It looks like this:

```
User-agent: Googlebot
Disallow: /creatorhub/\*
Disallow: /rss/people/\*/reviews
Disallow: /gp/pdp/rss/\*/reviews
Disallow: /gp/cdp/member-reviews/
Disallow: /gp/aw/cr/
```

We need to respect that file and avoid crawling the pages specified in there. We can cache it to avoid downloading it all the time.

Performance optimization

Some performance optimizations we can consider for the HTML downloader.

- Distributed crawl - We can parallelize crawl jobs to multiple machines which run multiple threads to crawl more efficiently.

  Show Image

- Cache DNS Resolver - We can maintain our own DNS cache to avoid making requests to the DNS resolver all the time, which can be costly. It's updated periodically by cron jobs.
- Locality - We can distribute crawl jobs based on geography. When crawlers are physically closer to website servers, latency is lower.
- Short timeout - We need to add a timeout in case servers are unresponsive beyond a given threshold. Otherwise, our crawlers can spend a lot of time waiting for pages which will never come.

Robustness

Some approaches to achieve robustness:

- Consistent hashing - To enable easy rescaling of our workers/crawlers/etc, we can use consistent hashing when load balancing jobs among them.
- Save crawl state and data - In the event of server crashes, it would be useful to store intermediary results on disk so that other workers can pick up from where the last one left.
- Exception handling - We need to handle exceptions gracefully without crashing the servers as these are inevitable in a large enough system.
- Data validation - important safety measure to prevent system errors.

Extensibility

We need to ensure the crawler is extendable if we want to support new content types in the future:

Show Image

Example extensions:

- PNG Downloader is added in order to crawl PNG images.
- Web monitor is added to monitor for copyright infringements.

Detect and avoid problematic content

Some common types of problematic content to be aware of:

- Redundant content - ~30% of web pages on the internet are duplicates. We need to avoid processing them more than once using hashes/checksums.
- Spider traps - A web page which leads to an infinite loop on the crawler, eg an extremely deep directory structure. This can be avoided by specifying a max length for URLs. But there are other sorts of spider traps as well. We can introduce the ability to manually intervene and blacklist spider trap websites.
- Data noise - Some content has no value for our target use-case. Eg advertisements, code snippets, spam, etc. We need to filter those.

Step 4 - Wrap up

Characteristics of a good crawler - scalable, polite, extensible, robust.

Other relevant talking points:

- Server-side rendering - Numerous sites dynamically generate HTML. If we parse the HTML without generating it first, we'll miss the information on the site. To solve this, we do server-side rendering first before parsing a page.
- Filter out unwanted pages - Anti-spam component is beneficial for filtering low quality pages.
- Database replication and sharding - useful techniques to improve the data layer's availability, scalability, reliability.
- Horizontal scaling - key is to keep servers stateless to enable horizontally scaling every type of server/worker/crawler/etc.
- Availability, consistency, reliability - concepts at the core of any large system's success.
- Analytics - We might also have to collect and analyze data in order to fine tune our system further.

Chapter 11

Design a Notification System

Notification systems are a popular feature in many applications - it alerts a user for important news, product updates, events, etc.

There are multiple flavors of a notification:

- Mobile push notification
- SMS
- Email

Step 1 - Understand the problem and establish design scope

- C: What types of notifications does the system support?
- I: Push notifications, SMS, Email
- C: Is it a real-time system?
- I: Soft real-time. We want user to receive notification as soon as possible, but delays are okay if system is under high load.
- C: What are the supported devices?
- I: iOS devices, android devices, laptop/desktop.
- C: What triggers notifications?
- I: Notifications can be triggered by client applications or on the server-side.
- C: Will users be able to opt-out?
- I: Yes
- C: How many notifications per day?
- I: 10mil mobile push, 1mil SMS, 5mil email

Step 2 - Propose high-level design and get buy-in

This section explores the high-level design of the notification system.

Different types of notifications

How do the different notification types work at a high level?

iOS push notification

Show Image

- Provider - builds and sends notification requests to Apple Push Notification Service (APNS). To do that, it needs some inputs:
  - Device token - unique identifier used for sending push notifications
  - Payload - JSON payload for the notification, eg:

```
{
   "aps":{
      "alert":{
         "title":"Game Request",
         "body":"Bob wants to play chess",
         "action-loc-key":"PLAY"
      },
      "badge":5
   }
}
```

- APNS - service, provided by Apple for sending mobile push notifications
- iOS Device - end client, which receives the push Notifications

Android Push Notification

Android adopts a similar approach. A common alternative to APNS is Firebase Cloud Messaging:

Show Image

SMS Message

For SMS, third-party providers like Twilio are available:

Show Image

Email

Although clients can setup their own mail servers, most clients opt-in to use third-party services, like Mailchimp:

Show Image

Here's final design after including all notification providers:

Show Image

Contact info gathering form

In order to send notifications, we need to gather some inputs from the user first. That is done at user signup:

Show Image

Example database tables for storing contact info:

Show Image

Notification sending/receiving flow

Here's the high-level design of our notification system:

Show Image

- Service 1 to N - other services in the system or cron jobs which trigger notification sending events.

- Notification system - accepts notification sending messages and propagates to the correct provider.
- Third-party services - responsible for delivering the messages to the correct users via the appropriate medium. This part should be build \w extensibility in case we change third-party service providers in the future.
- iOS, Android, SMS, Email - Users receive notifications on their devices.

Some problems in this design:

- Single point of failure - only a single notification service
- Hard to scale - since notification system handles everything, it is hard to independently scale eg the cache/database/service layer/etc.
- Performance bottleneck - handling everything in one system can be a bottleneck especially for resource-intensive tasks such as building HTML pages.

High-level design (improved)

Some changes from the original naive design:

- Move database & cache out of the notification service
- Add more notification servers & setup autoscaling & load balancing
- Introduce message queues to decouple system components

Show Image

- Service 1 to N - services which send notifications within our system
- Notification servers - provide APIs for sending notifications. Visible to internal services or verified clients. Do basic validation. Fetch notification templates from database. Put notification data in message queues for parallel processing.
- Cache - user info, device info, notification templates
- DB - stores data about users, notifications, settings, etc.
- Message queues - Remove dependencies across components. They serve as buffers for notifications to be sent out. Each notification provider has a different message queue assigned to avoid outages in one third-party provider to affect the rest.
- Workers - pull notification events from message queues and send them to corresponding third-party services.
- Third-party services - already covered in initial design.

- iOS, Android, SMS, Email - already covered in initial design.

Example API call to send an email:

```
{
   "to":[
      {
         "user_id":123456
      }
   ],
   "from":{
      "email":"from_address@example.com"
   },
   "subject":"Hello World!",
   "content":[
      {
         "type":"text/plain",
         "value":"Hello, World!"
      }
   ]
}
```

Example lifecycle of a notification:

- Service makes a call to make a notification
- Notification service fetch metadata (user info, settings, etc) from database/cache
- Notification event is sent to corresponding queue for processing for each third-party provider.
- Workers pull notifications from the message queues and send them to third-party services.
- Third-party services deliver nofications to end users.

Step 3 - Design deep dive

In this section, we discuss some additional considerations for our improved design.

Reliability

Some questions to consider in terms of making the system reliable:

- What happens in the event of data loss?
- Will recipients receive notifications exactly once?

To avoid data loss, we can persist notifications in a notification log database on the workers, which retry them in case a notification doesn't go through:

Show Image

What about duplicate notifications?

It will occasionally happen as we can't guarantee exactly-once delivery (unless the third-party API provides idempotency keys).
If they don't we can still try to reduce probability of this happening by having a dedup mechanism on our end, which discards an event id if it is already seen.

Additional components and considerations

Notification templates

To avoid building every notification from scratch on the client side, we'll introduce notification templates as many notifications can reuse them:

```
BODY:
You dreamed of it. We dared it. [ITEM NAME] is back — only until [DATE].

CTA:
Order Now. Or, Save My [ITEM NAME]
```

Notification setting

Before sending any notification, we first check if user has opted in for the given communication channel via this database table:

```
user_id bigInt
channel varchar # push notification, email or SMS
opt_in boolean # opt-in to receive notification
```

Rate limiting

To avoid overwhelming users with too many notifications, we can introduce some client-side rate limiting (on our end) so that they don't opt out of notifications immediately once they get bombarded.

Retry mechanism

If a third-party provider fails to send a notification, it will be put into a retry queue. If problem persists, developers are notified.

Security in push notifications

Only verified and authenticated clients are allowed to send push notifications through our APIs. We do this by requiring an appKey and appSecret, inspired by Android/Apple notification servers.

Monitor queued notifications

A critical metric to keep track of is number of queued notifications. If it gets too big, we might have to add more workers:

Show Image

Events tracking

We might have to track certain events related to a notification, eg open rate/click rate/etc.

Usually, this is done by integrating with an Analytics service, so we'll need to integrate our notification system with one.

Show Image

Updated design

Putting everything together, here's our final design:

Show Image

Other features we've added:

- Notification servers are equipped with authentication and rate limiting.
- Added a retry mechanism to handle notification failures.
- Notification templates are added to provide a coherent notification experience.
- Monitoring and tracking systems are added to keep track of system health for future improvements.

Step 4 - Wrap up

We introduced a robust notification system which supports push notifications, sms and email. We introduced message queues to decouple system components.

We also dug deeper into some components and optimizations:

- Reliability - added robust retry mechanism in case of failures
- Security - Appkey/appSecret is used to ensure only verified clients can make notifications.
- Tracking and monitoring - implemented to monitor important stats.
- Respect user settings - Users can opt-out of receiving notifications. Service checks the user settings first, before sending notifications.
- Rate limiting - Users would appreciate if we don't bombard them with a dozen of notifications all of a sudden.

Chapter 12

Design a News Feed System

News feed == constantly updating list of stories on your home page.

It includes status updates, photos, videos, links, etc.

Similar interview questions - design facebook news feed, twitter timeline, instagram feed, etc.

Step 1 - Understand the problem and establish design scope

First step is to clarify what the interviewer has in mind exactly:

- C: Mobile, web app?
- I: Both
- C: What are the important features?
- I: User can publish posts and see friends' posts on news feed.
- C: Is news feed sorted in reverse chronological order or based on rank, eg best friends' posts first.
- I: To keep it simple, let's assume reverse chrono order
- C: Max number of friends?
- I: 5000
- C: Traffic volume?
- I: 10mil DAU
- C: Can the feed contain media?
- I: It can contain images and video

Step 2 - Propose high-level design and get buy-in

There are two parts to the design:

- Feed publishing - when user publishes a post, corresponding data is written to cache and DB. Post is populated to friends' news feed.
- Newsfeed building - built by aggregating friends' posts in news feed.

Newsfeed API

The Newsfeed API is the primary gateway for users to the news feed services.

Here's some of the main endpoints.

- `POST /v1/me/feed` - publish a post. Payload includes `content` + `auth_token`.
- `GET /v1/me/feed` - retrieve news feed. Payload includes `auth_token`.

Feed publishing

Show Image

- User makes a new post via API.
- Load balancer - distributes traffic to web servers.
- Web servers - redirect traffic to internal services.
- Post service - persist post in database and cache.
- Fanout service - push posts to friends' news feeds.
- Notification service - inform new friends that content is available.

Newsfeed building

Show Image

- User sends request to retrieve news feed.
- Load balancer redirects traffic to web servers.
- Web servers - route requests to newsfeed service.

- Newsfeed service - fetch news feed from cache.
- Newsfeed cache - store pre-computed news feeds for fast retrieval.

Step 3 - Design deep dive

Let's discuss the two flows we covered in more depth.

Feed publishing deep dive

Show Image

Web servers

besides a gateway to the internal services, these do authentication and apply rate limits, in order to prevent spam.

Fanout service

This is the process of delivering posts to friends. There are two types of fanouts - fanout on write (push model) and fanout on read (pull model).

Fanout on write (push model) - posts are pre-computed during post publishing.

Pros:

- news feed is generated in real-time and can be delivered instantly to friends' news feed.
- fetching the news feed is fast as it's precomputed

Cons:

- if a friend has many friends, generating the news feed takes a lot of time, which slows down post publishing speed. This is the hotkey problem.
- for inactive users, pre-computing the news feed is a waste.

Fanout on read (pull model) - news feed is generated during read time.

Pros:

- Works better for inactive users, as news feeds are not generated for them.
- Data is not pushed to friends, hence, no hotkey problem.

Cons:

- Fetching the news feed is slow as it's not pre-computed.

We'll adopt a hybrid approach - we'll pre-compute the news feed for people without many friends and use the pull model for celebrities and users with many friends/followers.

System diagram of fanout service:

Show Image

- Fetch friend IDs from graph database. They're suited for managing friend relationships and recommendations.
- Get friends info from user cache. Filtering is applied here for eg muted/blocked friends.
- Send friends list and post ID to the message queue.
- Fanout workers fetch the messages and store the news feed data in a cache. They store a `<post_id, user_id>` mappings** in it which can later be retrieved.

** I think there is some kind of error in this part of the book. It doesn't make sense to store a `<post_id, user_id>` mapping in the cache. Instead, it should be a `<user_id, post_id>` mapping as that allows one to quickly fetch all posts for a given user, which are part of their news feed. In addition to that, the example in the book shows that you can store multiple user_ids or post_ids as keys in the cache, which is typically not supported in eg a hashmap, but it is actually supported when you use the `Redis Sets` feature, but that is not explicitly mentioned in the chapter.

News feed retrieval deep dive

Show Image

- user sends request to retrieve news feed.
- Load balancer distributes request to a set of web servers.
- Web servers call news feed service.
- News feed service gets a list of `post_id` from the news feed cache.
- Then, the posts in the news feed are hydrated with usernames, content, media files, etc.
- Fully hydrated news feed is returned as a JSON to the user.
- Media files are also stored in CDN and fetched from there for better user experience.

Cache architecture

Cache is very important for a news feed service. We divided it into 5 layers:

Show Image

- news feed - stores ids of news feeds
- content - stores every post data. Popular content is stored in hot cache.
- social graph - store user relationship data.
- action - store info about whether a user liked, replied or took actions on a post.
- counters - counters for replies, likes, followers, following, etc.

Step 4 - wrap up

In this chapter, we designed a news feed system and we covered two main use-cases - feed publishing and feed retrieval.

Talking points, related to scalability:

- vertical vs. horizontal database scaling
- SQL vs. NoSQL
- Master-slave replication
- Read replicas
- Consistency models
- Database sharding

Other talking points:

- keep web tier stateless
- cache data as much as possible
- multiple data center setup
- Loose coupling components via message queues
- Monitoring key metrics - QPS and latency.

Chapter 13

Design a Chat System

We'll be designing a chat system similar to Messenger, WhatsApp, etc.

In this case, it is very important to nail down the exact requirements because chat systems can differ a lot - eg ones focused on group chats vs. one-on-one conversations.

Step 1 - Understand the problem and establish design scope

- C: What kind of chat app should we design? One-on-one convos or group chat?
- I: It should support both cases.
- C: Mobile app, web app, both?
- I: Both
- C: What's the app scale? Startup or massive application?
- I: It should support 50mil DAU
- C: For group chat, what is the member limit?
- I: 100 people
- C: What features are important? Eg attachments?
- I: 1-on-1 and group chats. Online indicator. Text messages only.
- C: Is there message size limit?
- I: Text length is less than 100,000 chars long.
- C: End-to-end encryption required?
- I: Not required, but will discuss if time permits.
- C: How long should chat history be stored?
- I: Forever

Summary of features we'll focus on:

- One-on-one chat with low delivery latency
- Small group chats (100 ppl)
- Online presence
- Same account can be logged in via multiple devices.
- Push notifications
- Scale of 50mil DAU

Step 2 - Propose high-level design and get buy-in

Let's understand how clients and servers communicate first.

- In this system, clients can be mobile devices or web browsers.
- They don't connect to each other directly. They are connected to a server.

Main functions the chat service should support:

- Receive messages from clients
- Find the right recipients for a message and relay it
- If recipient is not online, hold messages for them until they get back online.

Show Image

When clients connect to the server, they can do it via one or more network protocols.
One option is HTTP. That is okay for the sender-side, but not okay for receiver-side.

There are multiple options to handle a server-initiated message for the client - polling, long-polling, web sockets.

Polling

Polling requires the client to periodically ask the server for status updates:

Show Image

This is easy to implement but it can be costly as there are many requests, which often yield no results

Long polling

Show Image

With long polling, clients hold the connection open while waiting for an event to occur on the server-side.
This still has some wasted requests if users don't chat much, but it is more efficient than polling.

Other caveats:

- Server has no good way to determine if client is disconnected.
- senders and receivers might be connected to different servers.

WebSocket

Most common approach when bidirectional communication is needed:

Show Image

The connection is initiated by the client and starts as HTTP, but can be upgraded after handshake.
In this setup, both clients and servers can initiate messages.

One caveat with web sockets is that this is a persistent protocol, making the servers stateful. Efficient connection management is necessary when using it.

High-level design

Although we mentioned how web sockets can be useful for exchanging messages, most other standard features of our chat can use the normal request/response protocol over HTTP.

Given this remark, our service can be broken down into three parts - stateless API, stateful websocket API and third-party integration for notifications:

Show Image

Stateless Services

Traditional public-facing request/response services are used to manage login, signup, user profile, etc.

These services sit behind a load-balancer, which distributes requests across a set of service replicas.

The service discovery service, in particular, is interesting and will be discussed more in-depth in the deep dive.

Stateful Service

The only stateful service is our chat service. It is stateful as it maintain a persistent connection with clients which connect to it.

In this case, a client doesn't switch to other chat services as long as the existing one stays alive.

Service discovery coordinates closely with the chat services to avoid overload.

Third-party Integration

It is important for a chat application to support push notifications in order to get notified when someone sends you a message.

This component won't be discussed extensively as it's already covered in the Design a notification system chapter.

Scalability

On a small scale, we can fit everything in a single server.

With 1mil concurrent users, assuming each connection takes up 10k memory, a single server will need to use 10GB of memory to service them all.

Despite this, we shouldn't propose a single-server setup as it raises a red flag in the interviewer. One big drawback of a single server design is the single point of failure.

It is fine, however, to start from a single-server design and extend it later as long as you explicitly state that during the interview.

Here's our refined high-level design:

Show Image

- clients maintain a persistent web socket connection with a chat server for real-time messaging
- The chat servers facilitate message sending/receiving
- Presense servers manage online/offline status
- API servers handle traditional request/response-based responsibilities - login, sign up, change profile, etc.
- Notification servers manage push notifications
- Key-value store is used for storing chat history. When offline user goes online, they will see their chat history and missed messages.

Storage

One important decision for the storage/data layer is whether we should go with a SQL or NoSQL database.

To make the decision, we need to examine the read/write access patterns.

Traditional data such as user profile, settings, user friends list can be stored in a traditional relational database.
Replication and sharding are common techniques to meet scalability needs for relational databases.

Chat history data, on the other hand, is very specific kind of data of chat systems due to its read/write pattern:

- Amount of data is enormous, a study revealed that Facebook and WhatsApp process 60bil messages per day.
- Only recent chats are accessed frequently. Users typically don't go too far back in chat history.
- Although chat history is accessed infrequently, we should still be able to search within it as users can use a search bar for random access.
- Read to write ratio is 1:1 on chat apps.

Selecting the correct storage system for this kind of data is crucial. Author recommends using a key-value store:

- they allow easy horizontal scaling
- they provide low latency access to data
- Relational databases don't handle long-tail (less-frequently accessed but large part of a distribution) of data well. When indexes grow large, random access is expensive.
- Key-value stores are widely adopted for chat systems. Facebook and Discord both use key-value stores. Facebook uses HBase, Discord uses Cassandra.

Data models

Let's take a look at the data model for our messages.

Message table for one-on-one chat:

Show Image

One caveat is that we'll use the primary key (message_id) instead of created_at to determine message sequence as messages can be sent at the same time.

Message table for a group chat:

Show Image

In the above table, `(channel_id, message_id)` is the primary key, while `channel_id` is also the sharding key.

One interesting discussion is how should the `message_id` be generated, as it is used for message ordering. It should have two important attributes:

- IDs must be unique
- IDs must be sortable by time

One option is to use the `auto_increment` feature of relational databases. But that's not supported in key-value stores. An alternative is to use Snowflake - Twitter's algorithm for generating 64-byte IDs which are globally unique and sortable by time.

Finally, we could also use a local sequence number generator, which is unique only within a group.
We can afford this because we only need to guarantee message sequence within a chat, but not between different chats.

Step 3 - Design deep-dive

In a system design interview, typically you are asked to go deeper into some of the components.

In this case, we'll go deeper into the service discovery component, messaging flows and online/offline indicator.

Service discovery

The primary goal of service discovery is to choose the best server based on some criteria - eg geographic location, server capacity, etc.

Apache Zookeeper is a popular open-source solution for service discovery. It registers all available chat servers and picks the best one based on a predefined criteria.

Show Image

- User A tries to login to the app
- Load balancer sends request to API servers.

- After authentication, service discovery chooses the best chat server for user A. In this case, chat server 2 is chosen.
- User A connects to chat server 2 via web sockets protocol.

Message flows

The message flows are an interesting topic to deep dive into. We'll explore one on one chats, message synchronization and group chat.

1 on 1 chat flow

Show Image

- User A sends a message to chat server 1
- Chat server 1 obtains a message_id from Id generator
- Chat server 1 sends the message to the "message sync" queue.
- Message is stored in a key-value store.
- If User B is online, message is forwarded to chat server 2, where User B is connected.
- If offline, push notification is sent via the push notification servers.
- Chat server 2 forwards the message to user B.

Message synchronization across devices

Show Image

- When user A logs in via phone, a web socket is established for that device with chat server 1.
- Each device maintains a variable called `cur_max_message_id`, keeping track of latest message received on given device.
- Messages whose recipient ID is currently logged in (via any device) and whose message_id is greater than `cur_max_message_id` are considered new

Small group chat flow

Group chats are a bit more complicated:

Show Image

Whenever User A sends a message, the message is copied across each message queue of participants in the group (User B and C).

Using one inbox per user is a good choice for small group chats as:

- it simplifies message sync since each user only need to consult their own queue.
- storing a message copy in each participant's inbox is feasible for small group chats.

This is not acceptable though, for larger group chats.

As for the recipient, in their queue, they can receive messages from different group chats:

Show Image

Online presence

Presence servers manage the online/offline indication in chat applications.

Whenever the user logs in, their status is changed to "online":

Show Image

Once the user send a logout message to the presence servers (and subsequently disconnects), their status is changed to "offline":

Show Image

One caveat is handling user disconnection. A naive approach to handle that is to mark a user as "offline" when they disconnect from the presence server.
This makes for a poor user experience as a user could frequently disconnect and reconnect to presence servers due to poor internet.

To mitigate this, we'll introduce a heartbeat mechanism - clients periodically send a heartbeat to the presence servers to indicate online status. If a heartbeat is not received within a given time frame, user is marked offline:

Show Image

How does a user's friend find out about a user's presence status though?

We'll use a fanout mechanism, where each friend pair have a queue assigned and status changes are sent to the respective queues:

Show Image

This is effective for small group chats. WeChat uses a similar approach and its user group is capped to 500 users.

If we need to support larger groups, a possible mitigation is to fetch presence status only when a user enters a group or refreshes the members list.

Step 4 - Wrap up

We managed to build a chat system which supports both one-on-one and group chats.

- We used web sockets for real-time communication between clients and servers.

System components:

- chat servers (real-time messages)
- presence servers (online/offline status)
- push notification servers
- key-value stores for chat history
- API servers for everything else

Additional talking points:

- Extend chat app to support media - video, images, voice. Compression, cloud storage and thumbnails can be discussed.
- End-to-end encryption - only sender and receiver can read messages.
- Caching messages on client-side is effective to reduce server-client data transfer.

- Improve load time - Slack built a geographically distributed network to cache user data, channels, etc for better load time.
- Error handling
- Chat server error - what happens if a chat server goes down. Zookeeper can facilitate a hand off to another chat server.
- Message resend mechanism - retrying and queueing are common approaches for re-sending messages.

Chapter 14

Design A Search Autocomplete System

Search autocomplete is the feature provided by many platforms such as Amazon, Google and others when you put your cursor in your search bar and start typing something you're looking for:

Show Image

Step 1 - Understand the problem and establish design scope

- C: Is the matching only supported at the beginning of a search term or eg at the middle?
- I: Only at the beginning
- C: How many autocompletion suggestions should the system return?
- I: 5
- C: Which suggestions should the system choose?
- I: Determined by popularity based on historical query frequency
- C: Does system support spell check?
- I: Spell check or auto-correct is not supported.
- C: Are search queries in English?
- I: Yes, if time allows, we can discuss multi-language support
- C: Is capitalization and special characters supported?
- I: We assume all queries use lowercase characters
- C: How many users use the product?
- I: 10mil DAU

Summary:

- Fast response time. An article about facebook autocomplete reviews that suggestions should be returned with 100ms delay at most to avoid stuttering
- Relevant - autocomplete suggestions should be relevant to search term
- Sorted - suggestions should be sorted by popularity
- Scalable - system can handle high traffic volumes
- Highly available - system should be up even if parts of the system are unresponsive

Back of the envelope estimation

- Assume we have 10mil DAU
- On average, person performs 10 searches per day
- 10mil * 10 = 100mil searches per day = 100 000 000 / 86400 = 1200 searches.
- given 4 works of 5 chars search on average -> 1200 * 20 = 24000 QPS. Peak QPS = 48000 QPS.
- 20% of daily queries are new -> 100mil * 0.2 = 20mil new searches * 20 bytes = 400mb new data per day.

Step 2 - Propose high-level design and get buy-in

At a high-level, the system has two components:

- Data gathering service - gathers user input queries and aggregates them in real-time.
- Query service - given search query, return topmost 5 suggestions.

Data gathering service

This service is responsible for maintaining a frequency table:

Show Image

Query service

Given a frequency table like the one above, this service is responsible for returning the top 5 suggestions based on the frequency column:

Show Image

Querying the data set is a matter of running the following SQL query:

Show Image

This is acceptable for small data sets but becomes impractical for large ones.

Step 3 - Design deep dive

In this section, we'll deep dive into several components which will improve the initial high-level design.

Trie data structure

We use relational databases in the high-level design, but to achieve a more optimal solution, we'll need to leverage a suitable data structure.

We can use tries for fast string prefix retrieval.

- It is a tree-like data structure
- The root represents the empty string
- Each node has 26 children, representing each of the next possible characters. To save space, we don't store empty links.
- Each node represents a single word or prefix
- For this problem, apart from storing the strings, we'll need to store the frequency against each leaf

  Show Image

To implement the algorithm, we need to:

- first find the node representing the prefix (time complexity O(p), where p = length of prefix)
- traverse subtree to find all leafs (time complexity O(c), where c = total children)
- sort retrieved children by their frequencies (time complexity O(clogc), where c = total children)

Show Image

This algorithm works, but there are ways to optimize it as we'll have to traverse the entire trie in the worst-case scenario.

Limit the max length of prefix

We can leverage the fact that users rarely use a very long search term to limit max prefix to 50 chars.

This reduces the time complexity from `O(p) + O(c) + O(clogc)` -> `O(1) + O(c) + O(clogc)`.

Cache top search queries at each node

To avoid traversing the whole trie, we can cache the top k most frequently accessed works in each node:

Show Image

This reduces the time complexity to `O(1)` as top K search terms are already cached. The trade-off is that it takes much more space than a traditional trie.

Data gathering service

In previous design, when user types in search term, data is updated in real-time. This is not practical on a bigger scale due to:

- billions of queries per day
- Top suggestions may not change much once trie is built

Hence, we'll instead update the trie asynchronously based on analytics data:

Show Image

The analytics logs contain raw rows of data related to search terms \w timestamps:

Show Image

The aggregators' responsibility is to map the analytics data into a suitable format and also aggregate it to lesser records.

The cadence at which we aggregate depends on the use-case for our auto-complete functionality. If we need the data to be relatively fresh & updated real-time (eg twitter search), we can aggregate once every eg 30m.
If, on the other hand, we don't need the data to be updated real-time (eg google search), we can aggregate once per week.

Example weekly aggregated data:

Show Image

The workers are responsible for building the trie data structure, based on aggregated data, and storing it in DB.

The trie cache keeps the trie loaded in-memory for fast read. It takes a weekly snapshot of the DB.

The trie DB is the persistent storage. There are two options for this problem:

- Document store (eg MongoDB) - we can periodically build the trie, serialize it and store it in the DB.
- Key-value store (eg DynamoDB) - we can also store the trie in hashmap format.

  Show Image

Query service

The query service fetches top suggestions from Trie Cache or fallbacks to Trie DB on cache miss:

Show Image

Some additional optimizations for the Query service:

- Using AJAX requests on client-side - these prevent the browser from refreshing the page.
- Data sampling - instead of logging all requests, we can log a sample of them to avoid too many logs.
- Browser caching - since auto-complete suggestions don't change often, we can leverage the browser cache to avoid extra calls to backend.

Example with Google search caching search results on the browser for 1h:

Show Image

Trie operations

Let's briefly describe common trie operations.

Create

The trie is created by workers using aggregated data, collected via analytics logs.

Update

There are two options to handling updates:

- Not updating the trie, but reconstructing it instead. This is acceptable if we don't need real-time suggestions.
- Updating individual nodes directly - we prefer to avoid it as it's slow. Updating a single node required updating all parent nodes as well due to the cached suggestions:

  Show Image

Delete

To avoid showing suggestions including hateful content or any other content we don't want to

show, we can add a filter between the trie cache and the API servers:

Show Image

The database is asynchronously updated to remove hateful content.

Scale the storage

At some point, our trie won't be able to fit on a single server. We need to devise a sharding mechanism.

One option to achieve this is to shard based on the letters of the alphabet - eg `a-m` goes on one shard, `n-z` on the other.

This doesn't work well as data is unevenly distributed due to eg the letter `a` being much more frequent than `x`.

To mitigate this, we can have a dedicated shard mapper, which is responsible for devising a smart sharding algorithm, which factors in the uneven distribution of search terms:

Show Image

Step 4 - Wrap up

Other talking points:

- How to support multi-language - we store unicode characters in trie nodes, instead of ASCII.
- What if top search queries differ across countries - we can build different tries per country and leverage CDNs to improve response time.
- How can we support trending (real-time) search queries? - current design doesn't support this and improving it to support it is beyond the scope of the book. Some options:
  - Reduce working data set via sharding
  - Change ranking model to assign more weight to recent search queries
  - Data may come as streams which you filter upon and use map-reduce technologies to process it - Hadoop, Apache Spark, Apache Storm, Apache Kafka, etc.

Chapter 15

Design YouTube

This chapter is about designing a video sharing platform such as youtube. Its solution can be applied to also eg designing Netflix, Hulu.

Step 1 - Understand the problem and establish design scope

- C: What features are important?
- I: Upload video + watch video
- C: What clients do we need to support?
- I: Mobile apps, web apps, smart TV
- C: How many DAUs do we have?
- I: 5mil
- C: Average time per day spend on YouTube?
- I: 30m
- C: Do we need to support international users?
- I: Yes
- C: What video resolutions do we need to support?
- I: Most of them
- C: Is encryption required?
- I: Yes
- C: File size requirement for videos?
- I: Max file size is 1GB
- C: Can we leverage existing cloud infra from Google, Amazon, Microsoft?
- I: Yes, building everything from scratch is not a good idea.

Features, we'll focus on:

- Upload videos fast
- Smooth video streaming
- Ability to change video quality
- Low infrastructure cost
- High availability, scalability, reliability
- Supported clients - web, mobile, smart TV

Back of the envelope estimation

- Assume product has 5mil DAU
- Users watch 5 videos per day
- 10% of users upload 1 video per day
- Average video size is 300mb
- Daily storage cost needed - 5mil * 10% * 300mb = 150TB
- CDN Cost, assuming 0.02$ per GB - 5mil * 5 videos * 0.3GB * 0.02$ = USD 150k per day

Step 2 - Propose high-level design and get buy-in

As previously discussed, we won't be building everything from scratch.

Why?

- In a system design interview, choosing the right technology is more important than explaining how the technology works.
- Building scalable blob storage over CDN is complex and costly. Even big tech don't build everything from scratch. Netflix uses AWS and Facebook uses Akamai's CDN.

Here's our system design at a high-level:

Show Image

- Client - you can watch youtube on web, mobile and TV.
- CDN - videos are stored in CDN.
- API Servers - Everything else, except video streaming goes through the API servers. Feed recommendation, generating video URL, updating metadata db and cache, user signup.

Let's explore high-level design of video streaming and uploading.

Video uploading flow

Show Image

- Users watch videos on a supported client

- Load balancer evenly distributes requests across API servers
- All user requests go through API servers, except video streaming
- Metadata DB - sharded and replicated to meet performance and availability requirements
- Metadata cache - for better performance, video metadata and user objects are cached
- A blob storage system is used to store the actual videos
- Transcoding/encoding servers - transform videos to various formats (eg MPEG, HLS, etc) which are suitable for different devices and bandwidth
- Transcoded storage stores result files from transcoding
- Videos are cached in CDN - clicking play streams the video from CDN
- Completion queue - stores events about video transcoding results
- Completion handler - a set of workers which pull event data from completion queue and update metadata cache and database

Let's now explore the flow of uploading videos and video metadata. Metadata includes info about video URL, size, resolution, format, etc.

Here's how the video uploading flow works:

Show Image

- Videos are uploaded to original storage
- Transcoding servers fetch videos from storage and start transcoding
- Once transcoding is complete, two steps are executed in parallel:
  - Transcoded videos are sent to transcoded storage and distributed to CDN
  - Transcoding completion events are queued in completion queue, workers pick up the events and update metadata database & cache
- API servers inform user that uploading is complete

Here's how the metadata update flow works:

Show Image

- While file is being uploaded, user sends a request to update the video metadata - file name, size, format, etc.
- API servers update metadata database & cache

Video streaming flow

Show Image

Whenever users watch videos on YouTube, they don't download the whole video at once. Instead, they download a little and start watching it while downloading the rest.
This is referred to as streaming. Stream is served from closest CDN server for lowest latency.

Some popular streaming protocols:

- MPEG-DASH - "Moving Picture Experts Group"-"Dynamic Adaptive Streaming over HTTP"
- Apple HLS - "HTTP Live Streaming"
- Microsoft Smooth Streaming
- Adobe HTTP Dynamic Streaming (HDS)

You don't need to understand those protocols in detail. It is important to understand, though, that different streaming protocols support different video encodings and playback players.

We need to choose the right streaming protocol to support our use-case.

Step 3 - Design deep dive

In this part, we'll deep dive into the video uploading and video streaming flows.

Video transcoding

Video transcoding is important for a few reasons:

- Raw video consumes a lot of storage space.
- Many browsers have constraints on the type of videos they can support. It is important to encode a video for compatibility reasons.
- To ensure good UX, you ought to serve HD videos to users with good network connection and lower-quality formats for the ones with slower connection.
- Network conditions can change, especially on mobile. It is important to be able to automatically switch video formats at runtime for smooth UX.

Most transcoding formats consist of two parts:

- Container - the basket which contains the video file. Recognized by the file extension, eg .avi, .mov, .mp4

- Codecs - Compression and decompression algorithms, which reduce video size while preserving quality. Most popular ones - H.264, VP9, HEVC.

Directed Acyclic Graph (DAG) model

Transcoding video is computationally expensive and time-consuming.
In addition to that, different creators have different inputs - some provide thumbnails, others do not, some upload HD, others don't.

In order to support video processing pipelines, dev customisations, high parallelism, we adopt a DAG model:

Show Image

Some of the tasks applied on a video file:

- Ensure video has good quality and is not malformed
- Video is encoded to support different resolutions, codecs, bitrates, etc.
- Thumbnail is automatically added if a user doesn't specify it.
- Watermark - image overlay on video if specified by creator

  Show Image

Video transcoding architecture

Show Image

Preprocessor

Show Image

The preprocessor's responsibilities:

- Video splitting - video is split in group of pictures (GOP) alignment, ie arranged groups of chunks which can be played independently
- Cache - intermediary steps are stored in persistent storage in order to retry on failure.
- DAG generation - DAG is generated based on config files specified by programmers.

Example DAG configuration with two steps:

Show Image

DAG Scheduler

Show Image

DAG scheduler splits a DAG into stages of tasks and puts them in a task queue, managed by a resource manager:

Show Image

In this example, a video is split into video, audio and metadata stages which are processed in parallel.

Resource manager

Show Image

Resource manager is responsible for optimizing resource allocation.

Show Image

- Task queue is a priority queue of tasks to be executed
- Worker queue is a queue of available workers and worker utilization info
- Running queue contains info about currently running tasks and which workers they're assigned to

How it works:

- task scheduler gets highest-priority task from queue
- task scheduler gets optimal task worker to run the task
- task scheduler instructs worker to start working on the task
- task scheduler binds worker to task & puts task/worker info in running queue
- task scheduler removes the job from the running queue once the job is done

Task workers

Show Image

The workers execute the tasks in the DAG. Different workers are responsible for different tasks and can be scaled independently.

Show Image

Temporary storage

Show Image

Multiple storage systems are used for different types of data. Eg temporary images/video/audio is put in blob storage. Metadata is put in an in-memory cache as data size is small.

Data is freed up once processing is complete.

Encoded video

Show Image

Final output of the DAG. Example output - `funny_720p.mp4`.

System Optimizations

Now it's time to introduce some optimizations for speed, safety, cost-saving.

Speed optimization - parallelize video uploading

We can split video uploading into separate units via GOP alignment:

Show Image

This enables fast resumable uploads if something goes wrong. Splitting the video file is done by the client.

Speed optimization - place upload centers close to users

Show Image

This can be achieved by leveraging CDNs.

Speed optimization - parallelism everywhere

We can build a loosely coupled system and enable high parallelism.

Currently, components rely on inputs from previous components in order to produce outputs:

Show Image

We can introduce message queues so that components can start doing their task independently of previous one once events are available:

Show Image

Safety optimization - pre-signed upload URL

To avoid unauthorized users from uploading videos, we introduce pre-signed upload URLs:

Show Image

How it works:

- client makes request to API server to fetch upload URL
- API servers generate the URL and return it to the client
- Client uploads the video using the URL

Safety optimization - protect your videos

To protect creators from having their original content stolen, we can introduce some safety options:

- Digital right management (DRM) systems - Apple FairPlay, Google Widevine, Microsoft PlayReady
- AES encryption - you can encrypt a video and configure an authorization policy. It is decrypted on playback.
- Visual watermarking - image overlay on top of video which contains your identifying information, eg company name.

Cost-saving optimization

CDN is expensive, as we've seen in our back of the envelope estimation.

We can piggyback on the fact that video streams follow a long-tail distribution - ie a few popular videos are accessed frequently, but everything else is not.

Hence, we can store popular videos in CDN and serve everything else from high capacity storage servers:

Show Image

Other cost-saving optimizations:

- We might not need to store many encoded versions for less popular videos. Short videos can be encoded on-demand.
- Some videos are only popular in certain regions. We can avoid distributing them in all regions.
- Build your own CDN. Can make sense for large streaming companies like Netflix.

Error Handling

For a large-scale system, errors are unavoidable. To make a fault-tolerant system, we need to handle errors gracefully and recover from them.

There are two types of errors:

- Recoverable error - can be mitigated by retrying a few times. If retrying fails, a proper error code is returned to the client.
- Non-recoverable error - system stops running related tasks and returns proper error code to the client.

Other typical errors and their resolution:

- Upload error - retry a few times
- Split video error - entire video is passed to server if older clients don't support GOP alignment.
- Transcoding error - retry
- Preprocessor error - regenerate DAG
- DAG scheduler error - retry scheduling
- Resource manager queue down - use a replica
- Task worker down - retry task on different worker
- API server down - they're stateless so requests can be redirected to other servers
- Metadata db/cache server down - replicate data across multiple nodes

- Master is down - Promote one of the slaves to become master
- Slave is down - If slave goes down, you can use another slave for reads and bring up another slave instance

Step 4 - Wrap up

Additional talking points:

- Scaling the API layer - easy to scale horizontally as API layer is stateless
- Scale the database - replication and sharding
- Live streaming - our system is not designed for live streams, but it shares some similarities, eg uploading, encoding, streaming. Notable differences:
    - Live streaming has higher latency requirements so it might demand a different streaming protocol
    - Lower requirement for parallelism as small chunks of data are already processed in real time
    - different error handling, as there is a timeout after which we need to stop retrying
    - Video takedowns - videos that violate copyrights, pornography, any other illegal acts need to be removed either during upload flow or based on user flagging.

Chapter 16

Design Google Drive

Google Drive is a cloud file storage product, which helps you store documents, videos, etc from the cloud.

You can access them from any device and share them with friends and family.

Step 1 - Understand the problem and establish design scope

- C: What are most important features?
- I: Upload/download files, file sync and notifications
- C: Mobile or web?
- I: Both
- C: What are the supported file formats?
- I: Any file type
- C: Do files need to be encrypted?
- I: Yes, files in storage need to be encrypted
- C: Is the a file size limit?

- I: Yes, files need to be 10 gb or smaller
- C: How many users does the app have?
- I: 10mil DAU

Features we'll focus on:

- Adding files
- Downloading files
- Sync files across devices
- See file revisions
- Share file with friends
- Send a notification when file is edited/deleted/shared

Features not discussed:

- Collaborative editing

Non-functional requirements:

- Reliability - data loss is unacceptable
- Fast sync speed
- Bandwidth usage - users will get unhappy if app consumes too much network traffic or battery
- Scalability - we need to handle a lot of traffic
- High availability - users should be able to use the system even when some services are down

Back of the envelope estimation

- Assume 50mil sign ups and 10mil DAU
- Users get 10 gb free space
- Users upload 2 files per day, average size is 500kb
- 1:1 read-write ratio
- Total space allocated - 50mil * 10gb = 500pb
- QPS for upload API - 10mil * 2 uploads / 24h / 3600s = ~240
- Peak QPS = 480

Step 2 - propose high-level design and get buy-in

In this chapter, we'll use a different approach than other ones - we'll start building the design from a single server and scale out from there.

We'll start from:

- A web server to upload and download files
- A database to keep track of metadata - user data, login info, files info, etc
- Storage system to store the files

Example storage we could use:

Show Image

APIs

Upload file:

```
https://api.example.com/files/upload?uploadType=resumable
```

This endpoint is used for uploading files with support for simple upload and resumable upload, which is used for large files.
Resumable upload is achieved by retrieving an upload URL and uploading the file while monitoring upload state. If disturbed, resume the upload.

Download file:

```
https://api.example.com/files/download
```

The payload specifies which file to download:

```
{
    "path": "/recipes/soup/best_soup.txt"
}
```

Get file revisions:

```
https://api.example.com/files/list_revisions
```

params:

- path to file for which revision history is retrieved
- maximum number of revisions to return

All the APIs require authentication and use HTTPS.

Move away from single server

As more files are uploaded, at some point, you reach your storage's capacity.

One option to scale your storage server is by implementing sharing - each user's data is stored on separate servers:

Show Image

This solves your issue but you're still worried about potential data loss.

A good option to address that is to use an off-the-shelf solution like Amazon S3 which offers replication (same-region/cross-region) out of the box:

Show Image

Other areas you could improve:

- Load balancing - this ensures evenly distributed network traffic to your web server replicas.
- More web servers - with the advent of a load balancer, you can easily scale your web server layer by adding more servers.
- Metadata database - move the database away from the server to avoid single points of failure. You can also setup replication and sharding to meet scalability requirements.
- File storage - Amazon S3 for storage. To ensure availability and durability, files are replicated in two separate geographical regions.

Here's the updated design:

Show Image

Sync conflicts

Once the user base grows sufficiently, sync conflicts are unavoidable.

To address this, we can apply a strategy where the first who manages to modify a file first wins:

Show Image

What happens once you get a conflict? We generate a second version of the file which represents the alternative file version and it's up to the user to merge it:

Show Image

High-level design

Show Image

- User uses the application through a browser or a mobile app
- Block servers upload files to cloud storage. Block storage is a technology which allows you to split a big file in blocks and store the blocks in a backing storage. Dropbox, for example, stores blocks of size 4mb.
- Cloud storage - a file split into multiple blocks is stored in cloud storage
- Cold storage - used for storing inactive files, infrequently accessed.
- Load balancer - evenly distributes requests among API servers.
- API servers - responsible for anything other than uploading files. Authentication, user profile management, updating file metadata, etc.
- Metadata database - stores metadata about files uploaded to cloud storage.
- Metadata cache - some of the metadata is cached for fast retrieval.
- Notification service - Publisher/subscriber system which notifies users when a file is updated/edited/removed so that they can pull the latest changes.
- Offline backup queue - used to queue file changes for users who are offline so that they can pull them once they come back online.

Step 3 - Design deep dive

Let's explore:

- block servers
- metadata database
- upload/download flow
- notification service
- saving storage space
- failure handling

Block servers

For large files, it's infeasible to send the whole file on each update as it consumes a lot of bandwidth.

Two optimizations we're going to explore:

- Delta sync - once a file is modified, only modified blocks are sent to the block servers instead of the whole file.
- Compression - applying compression on blocks can significantly reduce data size. Different algorithms are suitable for different file types, eg for text files, we'll use gzip/bzip2.

Apart from splitting files in blocks, the block servers also apply encryption prior to storing files in file storage:

Show Image

Example delta sync:

Show Image

High consistency requirement

Our system requires strong consistency as it's unacceptable to show different versions of a file to different people.

This is mainly problematic when we use caches, in particular the metadata cache in our example. To sustain strong consistency, we need to:

- keep cache master and replicas consistent
- invalidate caches on database write

For the database, strong consistency is guaranteed as long as we use a relational database, which supports ACID (all typically do).

Metadata database

Here's a simplified table schema for the metadata db (only interesting fields are shown):

Show Image

- User table contains basic information about the user such as username, email, profile photo, etc.
- Device table stores device info. Push_id is used for sending push notifications. Users can have multiple devices.
- Namespace - root directory of a user
- File table stores everything related to a file
- File_version stores the version history of a file. Existing fields are read-only to sustain file integrity.
- Block - stores everything related to a file block. A file version can be reconstructed by joining all blocks in the correct version.

Upload flow

Show Image

In the above flow, two requests are sent in parallel - updating file metadata and uploading the file to cloud storage.

Add file metadata:

- Client 1 sends request to update file metadata
- New file metadata is stored and upload status is set to "pending"
- Notify the notification service that a new file is being added.
- Notification service notifies relevant clients about the file upload.

Upload files to cloud storage:

- Client 1 uploads file contents to block servers

- Block servers chunk the file in blocks, compresses, encrypts them and uploads to cloud storage
- Once file is uploaded, upload completion callback is triggered. Request is sent to API servers.
- File status is changed to "uploaded" in Metadata DB.
- Notification service is notified of file uploaded event and client 2 is notified about the new file.

Download flow

Download flow is triggered when file is added or edited elsewhere. Client is notified via:

- Notification if online
- New changes are cached until user comes online if offline at the moment

Once a client is notified of the changes, it requests the file metadata and then downloads the blocks to reconstruct the file:

Show Image

- Notification service informs client 2 of file changes
- Client 2 fetches metadata from API servers
- API servers fetch metadata from metadata DB
- Client 2 gets the metadata
- Once client receives the metadata, it sends requests to block servers to download blocks
- Block servers download blocks from cloud storage and forwards them to the client

Notification service

The notification service enables file changes to be communicated to clients as they happen.

Clients can communicate with the notification service via:

- long polling (eg Dropbox uses this approach)
- Web sockets - communication is persistent and bidirectional

Both options work well but we opt for long polling because:

- Communication for notification service is not bi-directional. Server sends information to clients, not vice versa.

- WebSocket is meant for real-time bidirectional communication. For google drive, notifications are sent infrequently.

With long polling, the client sends a request to the server which stays open until a change is received or timeout is reached.
After that, a subsequent request is sent for next couple of changes.

Save storage space

To support file version history and ensure reliability, multiple versions of a file are stored across multiple data centers.

Storage space can be filled up quickly. Three techniques can be applied to save storage space:

- De-duplicate data blocks - if two blocks have the same hash, we can only store them once.
- Adopt an intelligent backup strategy - set a limit on max version history and aggregate frequent edits into a single version.
- Move infrequently accessed data to cold storage - eg Amazon S3 glacier is a good option for this, which is much cheaper than Amazon S3.

Failure handling

Some typical failures and how you could resolve them:

- Load balancer failure - If a load balancer fails, a secondary becomes active and picks up the traffic.
- Block server failure - If a block server fails, other replicas pick up the traffic and finish the job.
- Cloud storage failure - S3 buckets are replicated across regions. If one region fails, traffic is redirected to the other one.
- API server failure - Traffic is redirected to other service instances by the load balancer.
- Metadata cache failure - Metadata cache servers are replicated multiple times. If one goes down, other nodes are still available.
- Metadata DB failure - if master is down, promote one of the slaves to be master. If slave is down, use another one for read operations.
- Notification service failure - If long polling connections are lost, clients reconnect to a different service replica, but reconnection of millions of clients will take some time.
- Offline backup queue failure - Queues are replicated multiple times. If one queue fails, consumers need to resubscribe to the backup queue.

Step 4 - Wrap up

Properties of our Google Drive system design in a nutshell:

- Strongly consistent
- Low network bandwidth
- Fast sync

Our design contains two flows - file upload and file sync.

If time permits, you could discuss alternative design approaches as there is no perfect design. For example, we can upload blocks directly to cloud storage instead of going through block servers.

This is faster than our approach but has drawbacks:

- Chunking, compression, encryption need to be implemented on different platforms (Android, iOS, Web).
- Client can be hacked so implementing encryption client-side is not ideal.

Another interesting discussion is moving online/offline logic to separate service so that other services can reuse it to implement interesting functionality.

Chapter 17

Proximity Service

A proximity service enables you to discover nearby places such as restaurants, hotels, theatres, etc.

Step 1 - Understand the problem and establish design scope

Sample questions to understand the problem better:

- C: Can a user specify a search radius? What if there are not enough businesses within the search area?
- I: We only care about businesses within a certain area. If time permits, we can discuss enhancing the functionality.
- C: What's the max radius allowed? Can I assume it's 20km?
- I: Yes, that is a reasonable assumption
- C: Can a user change the search radius via the UI?
- I: Yes, let's say we have the options - 0.5km, 1km, 2km, 5km, 20km
- C: How is business information modified? Do we need to reflect changes in real-time?
- I: Business owners can add/delete/update a business. Assume changes are going to be propagated on the next day.
- C: How do we handle search results while the user is moving?

- I: Let's assume we don't need to constantly update the page since users are moving slowly.

Functional requirements

- Return all businesses based on user's location
- Business owners can add/delete/update a business. Information is not reflected in real-time.
- Customers can view detailed information about a business

Non-functional requirements

- Low latency - users should be able to see nearby businesses quickly
- Data privacy - Location info is sensitive data and we should take this into consideration in order to comply with regulations
- High availability and scalability requirements - We should ensure system can handle spike in traffic during peak hours in densely populated areas

Back-of-the-envelope calculation

- Assuming 100mil daily active users and 200mil businesses
- Search QPS == 100mil * 5 (average searches per day) / 10^5 (seconds in day) == 5000

Step 2 - Propose High-Level Design and get Buy-In

API Design

We'll use a RESTful API convention to design a simplified version of the APIs.

```
GET /v1/search/nearby
```

This endpoint returns businesses based on search criteria, paginated.

Request parameters - latitude, longitude, radius

Example response:

```
{
  "total": 10,
  "businesses":[{business object}]
}
```

The endpoint returns everything required to render a search results page, but a user might require additional details about a particular business, fetched via other endpoints.

Here's some other business APIs we'll need:

- `GET /v1/businesses/{:id}` - return business detailed info
- `POST /v1/businesses` - create a new business
- `PUT /v1/businesses/{:id}` - update business details
- `DELETE /v1/businesses/{:id}` - delete a business

Data model

In this problem, the read volume is high because these features are commonly used:

- Search for nearby businesses
- View the detailed information of a business

On the other hand, write volume is low because we rarely change business information. Hence for a read-heavy workflow, a relational database such as MySQL is ideal.

In terms of schema, we'll need one main `business` table which holds information about a business:

Show Image

We'll also need a geo-index table so that we efficiently process spatial operations. This table will be discussed later when we introduce the concept of geohashes.

High-level design

Here's a high-level overview of the system:

Show Image

- The load balancer automatically distributes incoming traffic across multiple services. A company typically provides a single DNS entry point and internally routes API calls to appropriate services based on URL paths.
- Location-based service (LBS) - read-heavy, stateless service, responsible for serving read requests for nearby businesses
- Business service - supports CRUD operations on businesses.
- Database cluster - stores business information and replicates it in order to scale reads. This leads to some inconsistency for LBS to read business information, which is not an issue for

our use-case

- Scalability of business service and LBS - since both services are stateless, we can easily scale them horizontally

Algorithms to fetch nearby businesses

In real life, one might use a geospatial database, such as Geohash in Redis or Postgres with PostGIS extension.

Let's explore how these databases work and what other alternative algorithms there are for this type of problem.

Two-dimensional search

The most intuitive and naive approach to solving this problem is to draw a circle around the person and fetch all businesses within the circle's radius:

Show Image

This can easily be translated to a SQL query:

```
SELECT business_id, latitude, longitude,
FROM business
WHERE (latitude BETWEEN {:my_lat} - radius AND {:my_lat} + radius) AND
      (longitude BETWEEN {:my_long} - radius AND {:my_long} + radius)
```

This query is not efficient because we need to query the whole table. An alternative is to build an index on the longitude and latitude columns but that won't improve performance by much.

This is because we still need to subsequently filter a lot of data regardless of whether we index by long or lat:

Show Image

We can, however, build 2D indexes and there are different approaches to that:

Show Image

We'll discuss the ones highlighted in purple - geohash, quadtree and google S2 are the most popular approaches.

Evenly divided grid

Another option is to divide the world in small grids:

Show Image

The major flaw with this approach is that business distribution is uneven as there are a lot of businesses concentrated in new york and close to zero in the sahara desert.

Geohash

Geohash works similarly to the previous approach, but it recursively divides the world into smaller and smaller grids, where each two bits correspond to a single quadrant:

Show Image

Geohashes are typically represented in base32. Here's the example geohash of google headquarters:

```
1001 10110 01001 10000 11011 11010 (base32 in binary) → 9q9hvu (base32)
```

It supports 12 levels of precision, but we only need up to 6 levels for our use-case:

Show Image

Geohashes enable us to quickly locate neighboring regions based on a substring of the geohash:

Show Image

However, one issue \w geohashes is that there can be places which are very close to each other which don't share any prefix, because they're on different sides of the equator or meridian:

Show Image

Another issue is that two businesses can be very close but not share a common prefix because they're in different quadrants:

Show Image

This can be mitigated by fetching neighboring geohashes, not just the geohash of the user.

A benefit of using geohashes is that we can use them to easily implement the bonus problem of increasing search radius in case insufficient businesses are fetched via query:

Show Image

This can be done by removing the last letter of the target geohash to increase radius.

Quadtree

A quadtree is a data structure, which recursively subdivides quadrants as deep as it needs to, based on business needs:

Show Image

This is an in-memory solution which can't easily be implemented in a database.

Here's how it might look conceptually:

Show Image

Example pseudocode to build a quadtree:

```
public void buildQuadtree(TreeNode node) {
    if (countNumberOfBusinessesInCurrentGrid(node) > 100) {
        node.subdivide();
        for (TreeNode child : node.getChildren()) {
            buildQuadtree(child);
        }
    }
}
```

In a leaf node, we store:

- Top-left, bottom-right coordinates to identify the quadrant dimensions
- List of business IDs in the grid

In an internalt node we store:

- Top-left, bottom-right coordinates of quadrant dimensions
- 4 pointers to children

The total memory to represent the quadtree is calculated as ~1.7GB in the book if we assume that we operate with 200mil businesses.

Hence, a quadtree can be stored in a single server, in-memory, although we can of course replicate it for redundancy and load balancing purposes.

One consideration to take into consideration if this approach is adopted - startup time of server can be a couple of minutes while the quadtree is being built.

Hence, this should be taken into account during the deployment process. Eg a healthcheck endpoint can be exposed and queried to signal when the quadtree build is finished.

Another consideration is how to update the quadtree. Given our requirements, a good option would be to update it every night using a nightly job due to our commitment of reflecting changes at start of next day.

It is nevertheless possible to update the quadtree on the fly, but that would complicate the implementation significantly.

Example quadtree of Denver:

Show Image

Google S2

Google S2 is a geometry library, which supports mapping 2D points on a 1D plane using hilbert

curves. Objects close to each other on the 2D plane are close on the hilbert curve as well:

Show Image

This library is great for geofencing, which supports covering arbitrary areas vs. confining yourself to specific quadrants.

Show Image

This functionality can be used to support more advanced use-cases than nearby businesses.

Another benefit of Google S2 is its region cover algorithm, which enables us to define more granular precision levels, than those provided by geohashes.

Recommendation

There is no perfect solution, different companies adopt different solutions:

Show Image

Author suggest choosing geohashes or quadtree in an interview as those are easier to explain than Google S2.

Here's a quick summary of geohashes:

- Easy to use and implement, no need to build a tree
- supports returning businesses within a specified radius
- Geohash precision is fixed. More complex logic is required if a more granular precision is needed
- Updating the index is easy

Here's a quick summary of quadtrees:

- Slightly harder to implement as it requires us to build a tree
- Supports fetching k-nearest neighbors instead of businesses within radius, which can be a good use-case for certain features
- Grid size can be dynamically adjusted based on population density

- Updating the index is more complicated than updating the geohash variant. All problems with updating and balancing trees are present when working with quad trees.

Step 3 - Design Deep Dive

Let's dive deeper into some areas of the design.

Scale the database

The business table can be scaled by sharding it in case it doesn't fit in a single server instance.

The geohash table can be represented by two columns:

Show Image

We don't need to shard the geohash table as we don't have that much data. We calculated that it takes ~1.7gb to build a quad tree and geohash space usage is similar.

We can, however, replicate the table to scale the read load.

Caching

Before using caching, we should ask ourselves if it is really necessary. In our case, the workflow is read-heavy and data can fit into a single server, so this kind of data is ripe for caching.

We should be careful when choosing the cache key. Location coordinates are not a good cache key as they often change and can be inaccurate.

Using the geohash is a more suitable key candidate.

Here's how we could query all businesses in a geohash:

```
SELECT business_id FROM geohash_index WHERE geohash LIKE `{:geohash}%`
```

Here's example code to cache the data in redis:

```
public List<String> getNearbyBusinessIds(String geohash) {
    String cacheKey = hash(geohash);
    List<string> listOfBusinessIds = Redis.get(cacheKey);
    if (listOfBusinessIDs  == null) {
        listOfBusinessIds = Run the select SQL query above;
        Cache.set(cacheKey, listOfBusinessIds, "1d");
    }
    return listOfBusinessIds;
}
```

We can cache the data on all precisions we support, which are not a lot, ie `geohash_4, geohash_5, geohash_6`.

As we already discussed, the storage requirements are not high and can fit into a single redis server, but we could replicate it for redundancy purposes as well as to scale reads.

We can even deploy multiple redis replicas across different data centers.

We could also cache `business_id -> business_data` as users could often query the details of the same popular restaurant.

Region and availability zones

We can deploy multiple LBS service instances across the globe so that users query the instance closest to them. This leads to reduced latency;

Show Image

It also enables us to spread traffic evenly across the globe. This could also be required in order to comply with certain data privacy laws.

Follow-up question - filter businesses by type or time

Once businesses are filtered, the result set is going to be small, hence, it is acceptable to filter the data in-memory.

Final design diagram

Show Image

- Client tries to locate restaurants within 500meters of their location
- Load balancer forwards the request to the LBS
- LBS maps the radius to geohash with length 6
- LBS calculates neighboring geohashes and adds them to the list
- For each geohash, LBS calls the redis server to fetch corresponding business IDs. This can be done in parallel.
- Finally, LBS hydrates the business ids, filters the result and returns it to the user

- Business-related APIs are separated from the LBS into the business service, which checks the cache first for any read requests before consulting the database
- Business updates are handled via a nightly job, which updates the geohash store

Step 4 - Wrap Up

Summary of some of the more interesting topics we covered:

- Discussed several indexing options - 2d search, evenly divided grid, geohash, quadtree, google S2
- Discussed caching, replication, sharding, cross-DC deployments in the deep dive section

---

Chapter 18

Nearby Friends

This chapter focuses on designing a scalable backend for an application which enables user to share their location and discover friends who are nearby.

The major difference with the proximity chapter is that in this problem, locations constantly change, whereas in that one, business addresses more or less stay the same.

Step 1 - Understand the Problem and Establish Design Scope

Some questions to drive the interview:

- C: How geographically close is considered to be "nearby"?
- I: 5 miles, this number should be configurable
- C: Is distance calculated as straight-line distance vs. taking into consideration eg a river in-between friends
- I: Yes, that is a reasonable assumption
- C: How many users does the app have?
- I: 1bil users and 10% of them use the nearby friends feature
- C: Do we need to store location history?
- I: Yes, it can be valuable for eg machine learning
- C: Can we assume inactive friends will disappear from the feature in 10min
- I: Yes
- C: Do we need to worry about GDPR, etc?
- I: No, for simlicity's sake

Functional requirements

- Users should be able to see nearby friends on their mobile app. Each friend has a distance and timestamp, indicating when the location was updated
- Nearby friends list should be updated every few seconds

Non-functional requirements

- Low latency - it's important to receive location updates without too much delay
- Reliability - Occassional data point loss is acceptable, but system should be generally available
- Eventual consistency - Location data store doesn't need strong consistency. Few seconds delay in receiving location data in different replicas is acceptable

Back-of-the-envelope

Some estimations to determine potential scale:

- Nearby friends are friends within 5mile radius
- Location refresh interval is 30s. Human walking speed is slow, hence, no need to update location too frequently.
- On average, 100mil users use the feature every day \w 10% concurrent users, ie 10mil
- On average, a user has 400 friends, all of them use the nearby friends feature
- App displays 20 nearby friends per page
- Location Update QPS = 10mil / 30 == ~334k updates per second

Step 2 - Propose High-Level Design and Get Buy-In

Before exploring API and data model design, we'll study the communication protocol we'll use as it's less ubiquitous than traditional request-response communication model.

High-level design

At a high-level we'd want to establish effective message passing between peers. This can be done via a peer-to-peer protocol, but that's not practical for a mobile app with flaky connection and tight power consumption constraints.

A more practical approach is to use a shared backend as a fan-out mechanism towards friends you want to reach:

Show Image

What does the backend do?

- Receives location updates from all active users
- For each location update, find all active users which should receive it and forward it to them
- Do not forward location data if distance between friends is beyond the configured threshold

This sounds simple but the challenge is to design the system for the scale we're operating with.

We'll start with a simpler design at first and discuss a more advanced approach in the deep dive:

Show Image

- The load balancer spreads traffic across rest API servers as well as bidirectional web socket servers
- The rest API servers handles auxiliary tasks such as managing friends, updating profiles, etc
- The websocket servers are stateful servers, which forward location update requests to respective clients. It also manages seeding the mobile client with nearby friends locations at initialization (discussed in detail later).
- Redis location cache is used to store most recent location data for each active user. There is a TTL set on each entry in the cache. When the TTL expires, user is no longer active and their data is removed from the cache.
- User database stores user and friendship data. Either a relational or NoSQL database can be used for this purpose.
- Location history database stores a history of user location data, not necessarily used directly within nearby friends feature, but instead used to track historical data for analytical purposes
- Redis pubsub is used as a lightweight message bus which enables different topics for each user channel for location updates.

  Show Image

In the above example, websocket servers subscribe to channels for the users which are connected to them & forward location updates whenever they receive them to appropriate users.

Periodic location update

Here's how the periodic location update flow works:

Show Image

- Mobile client sends a location update to the load balancer
- Load balancer forwards location update to the websocket server's persistent connection for that client
- Websocket server saves location data to location history database
- Location data is updated in location cache. Websocket server also saves location data in-memory for subsequent distance calculations for that user
- Websocket server publishes location data in user's channel via redis pub sub
- Redis pubsub broadcasts location update to all subscribers for that user channel, ie servers responsible for the friends of that user
- Subscribed web socket servers receive location update, calculate which users the update should be sent to and sends it

Here's a more detailed version of the same flow:

Show Image

On average, there's going to be 40 location updates to forward as a user has 400 friends on average and 10% of them are online at a time.

API Design

Websocket Routines we'll need to support:

- periodic location update - user sends location data to websocket server
- client receives location update - server sends friend location data and timestamp
- websocket client initialization - client sends user location, server sends back nearby friends location data
- Subscribe to a new friend - websocket server sends a friend ID mobile client is supposed to track eg when friend appears online for the first time

- Unsubscribe a friend - websocket server sends a friend ID, mobile client is supposed to unsubscribe from due to eg friend going offline

HTTP API - traditional request/response payloads for auxiliary responsibilities.

Data model

- The location cache will store a mapping between `user_id` and `lat,long,timestamp`. Redis is a great choice for this cache as we only care about current location and it supports TTL eviction which we need for our use-case.
- Location history table stores the same data but in a relational table \w the four columns stated above. Cassandra can be used for this data as it is optimized for write-heavy loads.

Step 3 - Design Deep Dive

Let's discuss how we scale the high-level design so that it works at the scale we're targetting.

How well does each component scale?

- API servers - can be easily scaled via autoscaling groups and replicating server instances
- Websocket servers - we can easily scale out the ws servers, but we need to ensure we gracefully shutdown existing connections when tearing down a server. Eg we can mark a server as "draining" in the load balancer and stop sending connections to it, prior to being finally removed from the server pool
- Client initialization - when a client first connects to a server, it fetches the user's friends, subscribes to their channels on redis pubsub, fetches their location from cache and finally forwards to client
- User database - We can shard the database based on user_id. It might also make sense to expose user/friends data via a dedicated service and API, managed by a dedicated team
- Location cache - We can shard the cache easily by spinning up several redis nodes. Also, the TTL puts a limit on the max memory we could have taken up at a time. But we still want to handle the large write load
- Redis pub/sub server - we leverage the fact that no memory is consumed if there are channels initialized but are not in use. Hence, we can pre-allocate channels for all users who use the nearby friends feature to avoid having to deal with eg bringing up a new channel when a user comes online and notifying active websocket servers

Scaling deep-dive on redis pub/sub component

We will need around 200gb of memory to maintain all pub/sub channels. This can be achieved by using 2 redis servers with 100gb each.

Given that we need to push ~14mil location updates per second, we will however need at least 140 redis servers to handle that amount of load, assuming that a single server can handle ~100k pushes per second.

Hence, we'll need a distributed redis server cluster to handle the intense CPU load.

In order to support a distributed redis cluster, we'll need to utilize a service discovery component, such as zookeeper or etcd, to keep track of which servers are alive.

What we need to encode in the service discovery component is this data:

Show Image

Web socket servers use that encoded data, fetched from zookeeper to determine where a particular channel lives. For efficiency, the hash ring data can be cached in-memory on each websocket server.

In terms of scaling the server cluster up or down, we can setup a daily job to scale the cluster as needed based on historical traffic data. We can also overprovision the cluster to handle spikes in loads.

The redis cluster can be treated as a stateful storage server as there is some state maintained for the channels and there is a need for coordination with subscribers so that they hand-off to newly provisioned nodes in the cluster.

We have to be mindful of some potential issues during scaling operations:

- There will be a lot of resubscription requests from the web socket servers due to channels being moved around
- Some location updates might be missed from clients during the operation, which is acceptable for this problem, but we should still minimize it from happening. Consider doing such operation when traffic is at lowest point of the day.
- We can leverage consistent hashing to minimize amount of channels moved in the event of adding/removing servers

  Show Image

Adding/removing friends

Whenever a friend is added/removed, websocket server responsible for affected user needs to subscribe/unsubscribe from the friend's channel.

Since the "nearby friends" feature is part of a larger app, we can assume that a callback on the mobile client side can be registered whenever any of the events occur and the client will send a message to the websocket server to do the appropriate action.

Users with many friends

We can put a cap on the total number of friends one can have, eg facebook has a cap of 5000 max friends.

The websocket server handling the "whale" user might have a higher load on its end, but as long as we have enough web socket servers, we should be okay.

Nearby random person

What if the interviewer wants to update the design to include a feature where we can occasionally see a random person pop up on our nearby friends map?

One way to handle this is to define a pool of pubsub channels, based on geohash:

Show Image

Anyone within the geohash subscribes to the appropriate channel to receive location updates for random users:

Show Image

We could also subscribe to several geohashes to handle cases where someone is close but in a bordering geohash:

Show Image

Alternative to Redis pub/sub

An alternative to using Redis for pub/sub is to leverage Erlang - a general programming language, optimized for distributed computing applications.

With it, we can spawn millions of small, erland processes which communicate with each other. We can handle both websocket connections and pub/sub channels within the distributed erlang

application.

A challenge with using Erlang, though, is that it's a niche programming language and it could be hard to source strong erlang developers.

Step 4 - Wrap Up

We successfully designed a system, supporting the nearby friends features.

Core components:

- Web socket servers - real-time comms between client and server
- Redis - fast read and write of location data + pub/sub channels

We also explored how to scale restful api servers, websocket servers, data layer, redis pub/sub servers and we also explored an alternative to using Redis Pub/Sub. We also explored a "random nearby person" feature.

Chapter 19

Google Maps

We'll design a simple version of Google Maps.

Some facts about google maps:

- Started in 2005
- Provides various services - satellite imagery, street maps, real-time traffic conditions, route planning
- By 2021, had 1bil daily active users, 99% coverage of the world, 25mil updates daily of real-time location info

Step 1 - Understand the Problem and Establish Design Scope

Sample Q&A between candidate and interviewer:

- C: How many daily active users are we dealing with?
- I: 1bil DAU
- C: What features should we focus on?
- I: Location update, navigation, ETA, map rendering
- C: How large is road data? Do we have access to it?
- I: We obtained road data from various sources, it's TBs of raw data
- C: Should we take traffic conditions into consideration?
- I: Yes, we should for accurate time estimations

- C: How about different travel modes - by foot, biking, driving?
- I: We should support those
- C: How about multi-stop directions?
- I: Let's not focus on that for scope of interview
- C: Business places and photos?
- I: Good question, but no need to consider those

We'll focus on three key features - user location update, navigation service including ETA, map rendering.

Non-functional requirements

- Accuracy - user shouldn't get wrong directions
- Smooth navigation - Users should experience smooth map rendering
- Data and battery usage - Client should use as little data and battery as possible. Important for mobile devices.
- General availability and scalability requirements

Map 101

Before jumping into the design, there are some map-related concepts we should understand.

Positioning system

World is a sphere, rotating on its axis. Positiions are defined by latitude (how far north/south you are) and longitude (how far east/west you are):

Show Image

Going from 3D to 2D

The process of translating points from 3D to 2D plane is called "map projection".

There are different ways to do it and each comes with its pros and cons. Almost all distort the actual geometry.

Show Image

Google maps selected a modified version of Mercator projection called "Web Mercator".

Geocoding

Geocoding is the process of converting addresses to geographic coordinates.

The reverse process is called "reverse geocoding".

One way to achieve this is to use interpolation - leveraging data from different sources (eg GIS-es) where street network is mapped to geo coordinate space.

Geohashing

Geohashing is an encoding system which encodes a geographic area into a string of letters and digits.

It depicts the world as a flattened surface and recursively sub-divides it into four quadrants:

Show Image

Map rendering

Map rendering happens via tiling. Instead of rendering entire map as one big custom image, world is broken up into smaller tiles.

Client only downloads relevant tiles and renders them like stitching together a mosaic.

There are different tiles for different zoom levels. Client chooses appropriate tiles based on the client's zoom level.

Eg, zooming out the entire world would download only a single 256x256 tile, representing the whole world.

Road data processing for navigation algorithms

In most routing algorithms, intersections are represented as nodes and roads are represented as edges:

Show Image

Most navigation algorithms use a modified version of Djikstra or A* algorithms.

Pathfinding performance is sensitive to the size of the graph. To work at scale, we can't represent the whole world as a graph and run the algorithm on it.

Instead, we use a technique similar to tiling - we subdivide the world into smaller and smaller graphs.

Routing tiles hold references to neighboring tiles and algorithms can stitch together a bigger road graph as it traverses interconnected tiles:

Show Image

This technique enables us to significantly reduce memory bandwidth and only load the tiles we need for the given source/destination pair.

However, for larger routes, stitching together small, detailed routing tiles would still be time/memory consuming. Instead, there are routing tiles with different level of detail and the algorithm uses the appropriately-detailed tiles, based on the destination we're headed for:

Show Image

Back-of-the-envelope estimation

For storage, we need to store:

- map of the world - estimated as ~70pb based on all the tiles we need to store, but factoring in compression of very similar tiles (eg vast desert)
- metadata - negligible in size, so we can skip it from calculation
- Road info - stored as routing tiles

Estimated QPS for navigation requests - 1bil DAU at 35min of usage per week -> 5bil minutes per day.
Assuming gps update requests are batched, we arrive at 200k QPS and 1mil QPS at peak load

Step 2 - Propose High-Level Design and Get Buy-In

Show Image

Location service

Show Image

It is responsible for recording a user's location updates:

- location updates are sent every `t` seconds
- location data streams can be used to improve the service over time, eg provide more accurate ETAs, monitor traffic data, detect closed roads, analyze user behavior, etc

Instead of sending location updates to the server all the time, we can batch the updates on the client-side and send batches instead:

Show Image

Despite this optimization, for a system of Google Maps scale, load will still be significant. Therefore, we can leverage a database, optimized for heavy writes such as Cassandra.

We can also leverage Kafka for efficient stream processing of location updates, meant for further analysis.

Example location update request payload:

```
POST /v1/locations
Parameters
  locs: JSON encoded array of (latitude, longitude, timestamp) tuples.
```

Navigation service

This component is responsible for finding fast routes between A and B in a reasonable time (a little bit of latency is okay). Route need not be the fastest, but accuracy is important.

Example request payload:

```
GET /v1/nav?origin=1355+market+street,SF&destination=Disneyland
```

Example response:

```
json
```

```
{
  "distance": {"text":"0.2 mi", "value": 259},
  "duration": {"text": "1 min", "value": 83},
  "end_location": {"lat": 37.4038943, "lng": -121.9410454},
  "html_instructions": "Head <b>northeast</b> on <b>Brandon St</b> toward <b>Lumin W
  "polyline": {"points": "_fhcFjbhgVuAwDsCal"},
  "start_location": {"lat": 37.4027165, "lng": -121.9435809},
  "geocoded_waypoints": [
    {
       "geocoder_status" : "OK",
       "partial_match" : true,
       "place_id" : "ChIJwZNMti1fawwRO2aVVVX2yKg",
       "types" : [ "locality", "political" ]
    },
    {
       "geocoder_status" : "OK",
       "partial_match" : true,
       "place_id" : "ChIJ3aPgQGtXawwRLYeiBMUi7bM",
       "types" : [ "locality", "political" ]
    }
  ],
  "travel_mode": "DRIVING"
}
```


Traffic changes and reroutes are not taken into consideration yet, those will be tackled in the deep dive section.

Map rendering

Holding the entire data set of mapping tiles on the client-side is not feasible as it's petabytes in size.

They need to be fetched on-demand from the server, based on the client's location and zoom level.

When should new tiles be fetched - while user is zooming in/out and during navigation, while they're going towards a new tile.

How should the map tiles be served to the client?

- They can be built dynamically, but that puts a huge load on the server and also makes caching hard
- Map tiles are served statically, based on their geohash, which a client can calculate. They can be statically stored & served from a CDN

Show Image

CDNs enable users to fetch map tiles from point-of-presence servers (POP) which are closest to users in order to minimize latency:

Show Image

Options to consider for determining map tiles:

- geohash for map tile can be calculated on the client-side. If that's the case, we should be careful that we commit to this type of map tile calculation for the long-term as forcing clients to update is hard
- alternatively, we can have simple API which calculates the map tile URLs on behalf of the clients at the cost of additional API call

Show Image

Step 3 - Design Deep Dive

Data model

Let's discuss how we store the different types of data we're dealing with.

Routing tiles

Initial road data set is obtained from different sources. It is improved over time based on location updates data.

The road data is unstructured. We have a periodic offline processing pipeline, which transforms this raw data into the graph-based routing tiles our app needs.

Instead of storing these tiles in a database as we don't need any database features. We can store them in S3 object storage, while caching them agressively.

We can also leverage libraries to compress adjacency lists into binary files efficiently.

User location data

User location data is very useful for updaring traffic conditions and doing all sorts of other analysis.

We can use Cassandra for storing this kind of data as its nature is to be write-heavy.

Example row:

Show Image

Geocoding database

This database stores a key-value pair of lat/long pairs and places.

We can use Redis for its fast read access speed, as we have frequent read and infrequent writes.

Precomputed images of the world map

As we discussed, we will precompute map tiling images and store them in CDN.

Show Image

Services

Location service

Let's focus on the database design and how user location is stored in detail for this service.

Show Image

We can use a NoSQL database to facilitate the heavy write load we have on location updates. We prioritize availability over consistency as user location data often changes and becomes stale as new updates arrive.

We'll choose Cassandra as our database choice as it nicely fits all our requirements.

Example row we're going to store:

Show Image

- `user_id` is the partition key in order to quickly access all location updates for a particular user
- `timestamp` is the clustering key in order to store the data sorted by the time a location update is received

We also leverage Kafka to stream location updates to various other service which need the location updates for various purposes:

Show Image

Rendering map

Map tiles are stored at various zoom levels. At the lowest zoom level, the entire world is represented by a single 256x256 tile.

As zoom levels increase, the number of map tiles quadruples:

Show Image

One optimization we can use is to not send the entire image information over the network, but instead represent tiles as vectors (paths & polygons) and let the client render the tiles dynamically.

This will have substantial bandwidth savings.

Navigation service

This service is responsible for finding the fastest routes:

Show Image

Let's go through each component in this sub-system.

First, we have the geocoding service which resolves an address to a location of lat/long pair.

Example request:

```
https://maps.googleapis.com/maps/api/geocode/json?
address=1600+Amphitheatre+Parkway,+Mountain+View,+CA
```

Example response:

```json
{
   "results" : [
      {
         "formatted_address" : "1600 Amphitheatre Parkway, Mountain View, CA 94043,
         "geometry" : {
            "location" : {
               "lat" : 37.4224764,
               "lng" : -122.0842499
            },
            "location_type" : "ROOFTOP",
            "viewport" : {
               "northeast" : {
                  "lat" : 37.4238253802915,
                  "lng" : -122.0829009197085
               },
               "southwest" : {
                  "lat" : 37.4211274197085,
                  "lng" : -122.0855988802915
               }
            }
         },
         "place_id" : "ChIJ2eUgeAK6j4ARbn5u_wAGqWA",
         "plus_code": {
            "compound_code": "CWC8+W5 Mountain View, California, United States",
            "global_code": "849VCWC8+W5"
         },
         "types" : [ "street_address" ]
      }
   ],
   "status" : "OK"
}
```

The route planner service computes a suggested route, optimized for travel time according to current traffic conditions.

The shortest-path service runs a variation of the A* algorithm against the routing tiles in object storage to compute an optimal path:

- It receives the source/destination pairs, converts them to lat/long pairs and derives the geohashes from those pairs to derive the routing tiles
- The algorithm starts from the initial routing tile and starts traversing it until a good enough path is found to the destination tile

Show Image

The ETA service is called by the route planner to get estimated time based on machine learning algorithms, predicting ETA based on traffic data.

The ranker service is responsible to rank different possible paths based on filters, passed by the user, ie flags to avoid toll roads or freeways.

The updater service asynchronously update some of the important databases to keep them up-to-date.

Improvement - adaptive ETA and rerouting

One improvement we can do is to adaptively update in-flight routes based on newly available traffic data.

One way to implement this is to store users who are currently navigating through a route in the database by storing all the tiles they're supposed to go through.

Data might look like this:

```
user_1: r_1, r_2, r_3, …, r_k
user_2: r_4, r_6, r_9, …, r_n
user_3: r_2, r_8, r_9, …, r_m
...
user_n: r_2, r_10, r21, ..., r_l
```

If a traffic accident happens on some tile, we can identify all users whose path goes through that tile and re-route them.

To reduce the amount of tiles we store in the database, we can instead store the origin routing tile and several routing tiles in different resolution levels until the destination tile is also included:

```
user_1, r_1, super(r_1), super(super(r_1)), ...
```

Show Image

Using this, we only need to check if the final tile of a user includes the traffic accident tile to see if user is impacted.

We can also keep track of all possible routes for a navigating user and notify them if a faster re-route is available.

Delivery protocols

We have several options, which enable us to proactively push data to clients from the server:

- Mobile push notifications don't work because payload is limited and it's not available for web apps
- WebSocket is generally a better option than long-polling as it has less compute footprint on servers
- We can also use server-sent events (SSE) but lean towards web sockets as they support bi-directional communication which can come in handy for eg a last-mile delivery feature

Step 4 - Wrap Up

This is our final design:

Show Image

One additional feature we could provide is multi-stop navigation which can be sold to enterprise customers such as Uber or Lyft in order to determine optimal path for visiting a set of locations.

Chapter 20

Distributed Message Queue

We'll be designing a distributed message queue in this chapter.

Benefits of message queues:

- Decoupling - Eliminates tight coupling between components. Let them update separately.

- Improved scalability - Producers and consumers can be scaled independently based on traffic.
- Increased availability - If one part of the system goes down, other parts continue interacting with the queue.
- Better performance - Producers can produce messages without waiting for consumer confirmation.

Some popular message queue implementations - Kafka, RabbitMQ, RocketMQ, Apache Pulsar, ActiveMQ, ZeroMQ.

Strictly speaking, Kafka and Pulsar are not message queues. They are event streaming platforms. There is however a convergence of features which blurs the distinction between message queues and event streaming platforms.

In this chapter, we'll be building a message queue with support for more advanced features such as long data retention, repeated message consumption, etc.

Step 1 - Understand the problem and establish design scope

Message queues ought to support few basic features - producers produce messages and consumers consume them.
There are, however, different considerations with regards to performance, message delivery, data retention, etc.

Here's a set of potential questions between Candidate and Interviewer:

- C: What's the format and average message size? Is it text only?
- I: Messages are text-only and usually a few KBs
- C: Can messages be repeatedly consumed?
- I: Yes, messages can be repeatedly consumed by different consumers. This is an added requirement, which traditional message queues don't support.
- C: Are messages consumed in the same order they were produced?
- I: Yes, order guarantee should be preserved. This is an added requirement, traditional message queues don't support this.
- C: What are the data retention requirements?
- I: Messages need to have a retention of two weeks. This is an added requirement.
- C: How many producers and consumers do we want to support?
- I: The more, the better.
- C: What data delivery semantic do we want to support? At-most-once, at-least-once, exactly-once?
- I: We definitely want to support at-least-once. Ideally, we can support all and make them configurable.
- C: What's the target throughput for end-to-end latency?

- I: It should support high throughput for use cases like log aggregation and low throughput for more traditional use cases.

Functional requirements:

- Producers send messages to a message queue
- Consumers consume messages from the queue
- Messages can be consumed once or repeatedly
- Historical data can be truncated
- Message size is in the KB range
- Order of messages needs to be preserved
- Data delivery semantics is configurable - at-most-once/at-least-once/exactly-once.

Non-functional requirements:

- High throughput or low latency. Configurable based on use-case
- Scalable - system should be distributed and support a sudden surge in message volume
- Persistent and durable - data should be persisted on disk and replicated among nodes

Traditional message queues typically don't support data retention and don't provide ordering guarantees. This greatly simplifies the design and we'll discuss it.

Step 2 - Propose high-level design and get buy-in

Key components of a message queue:

Show Image

- Producer sends messages to a queue
- Consumer subscribes to a queue and consumes the subscribed messages
- Message queue is a service in the middle which decouples producers from consumers, letting them scale independently.
- Producer and consumer are both clients, while the message queue is the server.

Messaging models

The first type of messaging model is point-to-point and it's commonly found in traditional message queues:

Show Image

- A message is sent to a queue and it's consumed by exactly one consumer.
- There can be multiple consumers, but a message is consumed only once.
- Once message is acknowledged as consumed, it is removed from the queue.
- There is no data retention in the point-to-point model, but there is such in our design.

On the other hand, the publish-subscribe model is more common for event streaming platforms:

Show Image

- In this model, messages are associated to a topic.
- Consumers are subscribed to a topic and they receive all messages sent to this topic.

Topics, partitions and brokers

What if the data volume for a topic is too large? One way to scale is by splitting a topic into partitions (aka sharding):

Show Image

- Messages sent to a topic are evenly distributed across partitions
- The servers that host partitions are called brokers
- Each topic operates like a queue using FIFO for message processing. Message order is preserved within a partition.
- The position of a message within the partition is called an **offset**.
- Each message produced is sent to a specific partition. A partition key specifies which partition a message should land in.
    - Eg a `user_id` can be used as a partition key to guarantee order of messages for the same user.

- Each consumer subscribes to one or more partitions. When there are multiple consumers for the same messages, they form a consumer group.

Consumer groups

Consumer groups are a set of consumers working together to consume messages from a topic:

Show Image

- Messages are replicated per consumer group (not per consumer).
- Each consumer group maintains its own offset.
- Reading messages in parallel by a consumer group improves throughput but hampers the ordering guarantee.
- This can be mitigated by only allowing one consumer from a group to be subscribed to a partition.
- This means that we can't have more consumers in a group than there are partitions.

High-level architecture

Show Image

- Clients are producer and consumer. Producer pushes messages to a designated topic. Consumer group subscribes to messages from a topic.
- Brokers hold multiple partitions. A partition holds a subset of messages for a topic.
- Data storage stores messages in partitions.
- State storage keeps the consumer states.
- Metadata storage stores configuration and topic properties
- The coordination service is responsible for service discovery (which brokers are alive) and leader election (which broker is leader, responsible for assigning partitions).

Step 3 - Design Deep Dive

In order to achieve high throughput and preserve the high data retention requirement, we made some important design choices:

- We chose an on-disk data structure which takes advantage of the properties of modern HDD and disk caching strategies of modern OS-es.
- The message data structure is immutable to avoid extra copying, which we want to avoid in a high volume/high traffic system.
- We designed our writes around batching as small I/O is an enemy of high throughput.

Data storage

In order to find the best data store for messages, we must examine a message's properties:

- Write-heavy, read-heavy
- No update/delete operations. In traditional message queues, there is a "delete" operation as messages are not retained.
- Predominantly sequential read/write access pattern.

What are our options:

- Database - not ideal as typical databases don't support well both write and read heavy systems.
- Write-ahead log (WAL) - a plain text file which only supports appending to it and is very HDD-friendly.
    - We split partitions into segments to avoid maintaining a very large file.
    - Old segments are read-only. Writes are accepted by latest segment only.

Show Image

WAL files are extremely efficient when used with traditional HDDs.

There is a misconception that HDD acces is slow, but that hugely depends on the access pattern. When the access pattern is sequential (as in our case), HDDs can achieve several MB/s write/read speed which is sufficient for our needs.
We also piggyback on the fact that the OS caches disk data in memory aggressively.

Message data structure

It is important that the message schema is compliant between producer, queue and consumer to avoid extra copying. This allows much more efficient processing.

Example message structure:

Show Image

The key of the message specifies which partition a message belongs to. An example mapping is `hash(key) % numPartitions`. For more flexibility, the producer can override default keys in order to control which partitions messages are distributed to.

The message value is the payload of a message. It can be plaintext or a compressed binary block.

Note: Message keys, unlike traditional KV stores, need not be unique. It is acceptable to have duplicate keys and for it to even be missing.

Other message files:

- Topic - topic the message belongs to
- Partition - The ID of the partition a message belongs to
- Offset - The position of the message in a partition. A message can be located via `topic`, `partition`, `offset`.
- Timestamp - When the message is stored
- Size - the size of this message
- CRC - checksum to ensure message integrity

Additional features such as filtering can be supported by adding additional fields.

Batching

Batching is critical for the performance of our system. We apply it in the producer, consumer and message queue.

It is critical because:

- It allows the operating system to group messages together, amortizing the cost of expensive network round trips
- Messages are written to the WAL in groups sequentially, which leads to a lot of sequential writes and disk caching.

There is a trade-off between latency and throughput:

- High batching leads to high throughput and higher latency.
- Less batching leads to lower throughput and lower latency.

If we need to support lower latency since the system is deployed as a traditional message queue, the system could be tuned to use a smaller batch size.

If tuned for throughput, we might need more partitions per topic to compensate for the slower sequential disk write throughput.

Producer flow

If a producer wants to send a message to a partition, which broker should it connect to?

One option is to introduce a routing layer, which route messages to the correct broker. If replication is enabled, the correct broker is the leader replica:

Show Image

- Routing layer reads the replication plan from the metadata store and caches it locally.
- Producer sends a message to the routing layer.
- Message is forwarded to broker 1 who is the leader of the given partition
- Follower replicas pull the new message from the leader. Once enough confirmations are received, the leader commits the data and responds to the producer.

The reason for having replicas is to enable fault tolerance.

This approach works but has some drawbacks:

- Additional network hops due to the extra component
- The design doesn't enable batching messages

To mitigate these issues, we can embed the routing layer into the producer:

Show Image

- Fewer network hops lead to lower latency
- Producers can control which partition a message is routed to
- The buffer allows us to batch messages in-memory and send out larger batches in a single request, which increases throughput.

The batch size choice is a classical trade-off between throughput and latency.

Show Image

- Larger batch size leads to longer wait time before batch is committed.
- Smaller batch size leads to request being sent sooner and having lower latency but lower throughput.

Consumer flow

The consumer specifies its offset in a partition and receives a chunk of messages, beginning from that offset:

Show Image

One important consideration when designing the consumer is whether to use a push or a pull model:

- Push model leads to lower latency as broker pushes messages to consumer as it receives them.
    - However, if rate of consumption falls behind the rate of production, the consumer can be overwhelmed.
    - It is challenging to deal with consumers with varying processing power as the broker controls the rate of consumption.
- Pull model leads to the consumer controlling the consumption rate.
    - If rate of consumption is slow, consumer will not be overwhelmed and we can scale it to catch up.
    - The pull model is more suitable for batch processing, because with the push model, the broker can't know how many messages a consumer can handle.
    - With the pull model, on the other hand, consumers can aggressively fetch large message batches.
    - The down side is the higher latency and extra network calls when there are no new messages. Latter issue can be mitigated using long polling.

Hence, most message queues (and us) choose the pull model.

Show Image

- A new consumer subscribes to topic A and joins group 1.
- The correct broker node is found by hashing the group name. This way, all consumers in a group connect to the same broker.
- Note that this consumer group coordinator is different from the coordination service (ZooKeeper).
- Coordinator confirms that the consumer has joined the group and assigns partition 2 to that consumer.
- There are different partition assignment strategies - round-robin, range, etc.
- Consumer fetches latest messages from the last offset. The state storage keeps the consumer offsets.
- Consumer processes messages and commits the offset to the broker. The order of those operations affects the message delivery semantics.

Consumer rebalancing

Consumer rebalancing is responsible for deciding which consumers are responsible for which partition.

This process occurs when a consumer joins/leaves or a partition is added/removed.

The broker, acting as a coordinator plays a huge role in orchestrating the rebalancing workflow.

Show Image

- All consumers from the same group are connected to the same coordinator. The coordinator is found by hashing the group name.
- When the consumer list changes, the coordinator chooses a new leader of the group.
- The leader of the group calculates a new partition dispatch plan and reports it back to the coordinator, which broadcasts it to the other consumers.

When the coordinator stops receiving heartbeats from the consumers in a group, a rebalancing is triggered:

Show Image

Let's explore what happens when a consumer joins a group:

Show Image

- Initially, only consumer A is in the group and it consumes all partitions.
- Consumer B sends a request to join the group.
- The coordinator notifies all group members that it's time to rebalance passively - as a response to the heartbeat.
- Once all consumers rejoin the group, the coordinator chooses a leader and notifies the rest about the election result.
- The leader generates the partition dispatch plan and sends it to the coordinator. Others wait for the dispatch plan.
- Consumers start consuming from the newly assigned partitions.

Here's what happens when a consumer leaves the group:

Show Image

- Consumer A and B are in the same group
- Consumer B asks to leave the group
- When coordinator receives A's heartbeat, it informs them that it's time to rebalance.
- The rest of the steps are the same.

The process is similar when a consumer doesn't send a heartbeat for a long time:

Show Image

State storage

The state storage stores mapping between partitions and consumers, as well as the last consumed offsets for a partition.

Show Image

Group 1's offset is at 6, meaning all previous messages are consumed. If a consumer crashes, the new consumer will continue from that message on wards.

Data access patterns for consumer states:

- Frequent read/write operations, but low volume
- Data is updated frequently, but rarely deleted
- Random read/write
- Data consistency is important

Given these requirements, a fast KV storage like Zookeeper is ideal.

Metadata storage

The metadata storage stores configuration and topic properties - partition number, retention period, replica distribution.

Metadata doesn't change often and volume is small, but there is a high consistency requirement. Zookeeper is a good choice for this storage.

ZooKeeper

Zookeeper is essential for building distributed message queues.

It is a hierarchical key-value store, commonly used for a distributed configuration, synchronization service and naming registry (ie service discovery).

Show Image

With this change, the broker only needs to maintain data for the messages. Metadata and state storage is in Zookeeper.

Zookeeper also helps with leader election of the broker replicas.

Replication

In distributed systems, hardware issues are inevitable. We can tackle this via replication to achieve high availability.

Show Image

- Each partition is replicated across multiple brokers, but there is only one leader replica.
- Producers send messages to leader replicas
- Followers pull the replicated messages from the leader
- Once enough replicas are synchronized, the leader returns acknowledgment to the producer
- Distribution of replicas for each partition is called the replica distribution plan.
- The leader for a given partition creates the replica distribution plan and saves it in Zookeeper

In-sync replicas

One problem we need to tackle is keeping messages in-sync between the leader and the followers for a given partition.

In-sync replicas (ISR) are replicas for a partition that stay in-sync with the leader.

The `replica.lag.max.messages` defines how many messages can a replica be lagging behind the leader to be considered in-sync.

Show Image

- Committed offset is 13
- Two new messages are written to the leader, but not committed yet.
- A message is committed once all replicas in the ISR have synchronized that message
- Replica 2 and 3 have fully caught up with leader, hence, they are in ISR
- Replica 4 has lagged behind, hence, is removed from ISR for now

ISR reflects a trade-off between performance and durability.

- In order for producers not to lose messages, all replicas should be in sync before sending an acknowledgment
- But a slow replica will cause the whole partition to become unavailable

Acknowledgment handling is configurable.

`ACK=all` means that all replicas in ISR have to sync a message. Message sending is slow, but message durability is highest.

Show Image

`ACK=1` means that producer receives acknowledgment once leader receives the message. Message sending is fast, but message durability is low.

Show Image

`ACK=0` means that producer sends messages without waiting for any acknowledgment from leader. Message sending is fastest, message durability is lowest.

Show Image

On the consumer side, we can connect all consumers to the leader for a partition and let them read messages from it:

- This makes for the simplest design and easiest operation
- Messages in a partition are sent to only one consumer in a group, which limits the connections to the leader replica
- The number of connections to leader replica is typically not high as long as the topic is not super hot
- We can scale a hot topic by increasing the number of partitions and consumers
- In certain scenarios, it might make sense to let a consumer lead from an ISR, eg if they're located in a separate DC

The ISR list is maintained by the leader who tracks the lag between itself and each replica.

Scalability

Let's evaluate how we can scale different parts of the system.

Producer

The producer is much smaller than the consumer. Its scalability can easily be achieved by adding/removing new producer instances.

Consumer

Consumer groups are isolated from each other. It is easy to add/remove consumer groups at will.

Rebalancing help handle the case when consumers are added/removed from a group gracefully.

Consumer groups are rebalancing help us achieve scalability and fault tolerance.

Broker

How do brokers handle failure?

Show Image

- Once a broker fails, there are still enough replicas to avoid partition data loss
- A new leader is elected and the broker coordinator redistributes partitions which were at the failed broker to existing replicas
- Existing replicas pick up the new partitions and act as followers until they're caught up with the leader and become ISR

Additional considerations to make the broker fault-tolerant:

- The minimum number of ISRs balances latency and safety. You can fine-tune it to meet your needs.
- If all replicas of a partition are in the same node, then it's a waste of resources. Replicas should be across different brokers.
- If all replicas of a partition crash, then the data is lost forever. Spreading replicas across data centers can help, but it adds up a lot of latency. One option is to use data mirroring as a work around.

How do we handle redistribution of replicas when a new broker is added?

Show Image

- We can temporarily allow more replicas than configured, until new broker catches up
- Once it does, we can remove the partition replica which is no longer needed

Partition

Whenever a new partition is added, the producer is notified and consumer rebalancing is triggered.

In terms of data storage, we can only store new messages to the new partition vs. trying to copy all old ones:

Show Image

Decreasing the number of partitions is more involved:

Show Image

- Once a partition is decommissioned, new messages are only received by remaining partitions
- The decommissioned partition isn't removed immediately as messages can still be consumed from it
- Once a pre-configured retention period passes, do we truncate the data and storage space is freed up
- During the transitional period, producers only send messages to active partitions, but consumers read from all
- Once retention period expires, consumers are rebalanced

Data delivery semantics

Let's discuss different delivery semantics.

At-most once

With this guarantee, messages are delivered not more than once and could not be delivered at all.

Show Image

- Producer sends a message asynchronously to a topic. If message delivery fails, there is no retry.
- Consumer fetches message and immediately commits offset. If consumer crashes before processing the message, the message will not be processed.

At-least once

A message can be sent more than once and no message should be left unprocessed.

Show Image

- Producer sends message with `ack=1` or `ack=all`. If there is any issue, it will keep retrying.
- Consumer fetches the message and consumes the offset only after it's done processing it.
- It is possible for a message to be delivered more than once if eg consumer crashes before committing offset but after processing it.
- This is why, this is good for use-cases where data duplication is acceptable or deduplication is possible.

Exactly once

Extremely costly to implement for the system, albeit it's the friendliest guarantee to users:

Show Image

Advanced features

Let's discuss some advanced features, we might discuss in the interview.

Message filtering

Some consumers might want to only consume messages of a certain type within a partition.

This can be achieved by building separate topics for each subset of messages, but this can be costly if systems have too many differing use-cases.

- It is a waste of resources to store the same message on different topics
- Producer is now tightly coupled to consumers as it changes with each new consumer requirement

We can resolve this using message filtering.

- A naive approach would be to do the filtering on the consumer-side, but that introduces unnecessary consumer traffic
- Alternatively, messages can have tags attached to them and consumers can specify which tags they're subscribed to
- Filtering could also be done via the message payloads but that can be challenging and unsafe for encrypted/serialized messages
- For more complex mathematical formulaes, the broker could implement a grammar parser or script executor, but that can be heavyweight for the message queue

Show Image

Delayed messages & scheduled messages

For some use-cases, we might want to delay or schedule message delivery.
For example, we might submit a payment verification check for 30m from now, which triggers the consumer to see if a payment was successful.

This can be achieved by sending messages to temporary storage in the broker and moving the message to the partition at the right time:

Show Image

- The temporary storage can be one or more special message topics
- The timing function can be achieved using dedicated delay queues or a hierarchical time wheel

Step 4 - Wrap up

Additional talking points:

- Protocol of communication. Important considerations - support all use-cases and high data volume, as well as verify message integrity. Popular protocols - AMQP and Kafka protocol.
- Retry consumption - if we can't process a message immediately, we could send it to a dedicated retry topic to be attempted later.
- Historical data archive - old messages can be backed up in high-capacity storages such as HDFS or object storage (eg S3).

Chapter 21

Metrics Monitoring and Alerting System

This chapter focuses on designing a highly scalable metrics monitoring and alerting system, which is critical for ensuring high availability and reliability.

Step 1 - Understand the Problem and Establish Design Scope

A metrics monitoring system can mean a lot of different things - eg you don't want to design a logs aggregation system, when the interviewer is interested in infra metrics only.

Let's try to understand the problem first:

- C: Who are we building the system for? An in-house monitoring system for a big tech company or a SaaS like DataDog?
- I: We are building for internal use only.
- C: Which metrics do we want to collect?
- I: Operational system metrics - CPU load, Memory, Data disk space. But also high-level metrics like requests per second. Business metrics are not in scope.
- C: What is the scale of the infrastructure we're monitoring?
- I: 100mil daily active users, 1000 server pools, 100 machines per pool
- C: How long should we keep the data?
- I: Let's assume 1y retention.
- C: May we reduce metrics data resolution for long-term storage?
- I: Keep newly received metrics for 7 days. Roll them up to 1m resolution for next 30 days. Further roll them up to 1h resolution after 30 days.
- C: What are the supported alert channels?
- I: Email, phone, PagerDuty or webhooks.

- C: Do we need to collect logs such as error or access logs?
- I: No
- C: Do we need to support distributed system tracing?
- I: No

High-level requirements and assumptions

The infrastructure being monitored is large-scale:

- 100mil DAU
- 1000 server pools * 100 machines * ~100 metrics per machine -> ~10mil metrics
- 1-year data retention
- Data retention policy - raw for 7d, 1-minute resolution for 30d, 1h resolution for 1y

A variety of metrics can be monitored:

- CPU load
- Request count
- Memory usage
- Message count in message queues

Non-functional requirements

- Scalability - System should be scalable to accommodate more metrics and alerts
- Low latency - System needs to have low query latency for dashboards and alerts
- Reliability - System should be highly reliable to avoid missing critical alerts
- Flexibility - System should be able to easily integrate new technologies in the future

What requirements are out of scope?

- Log monitoring - the ELK stack is very popular for this use-case
- Distributed system tracing - this refers to collecting data about a request lifecycle as it flows through multiple services within the system

Step 2 - Propose High-Level Design and Get Buy-In

Fundamentals

There are five core components involved in a metrics monitoring and alerting system:

Show Image

- Data collection - collect metrics data from different sources
- Data transmission - transfer data from sources to the metrics monitoring system
- Data storage - organize and store incoming data
- Alerting - Analyze incoming data, detect anomalies and generate alerts
- Visualization - Present data in graphs, charts, etc

Data model

Metrics data is usually recorded as a time-series, which contains a set of values with timestamps. The series can be identified by name and an optional set of tags.

Example 1 - What is the CPU load on production server instance i631 at 20:00?

Show Image

The data can be identified by the following table:

Show Image

The time series is identified by the metric name, labels and a single point in at a specific time.

Example 2 - What is the average CPU load across all web servers in the us-west region for the last 10min?

```
CPU.load host=webserver01,region=us-west 1613707265 50

CPU.load host=webserver01,region=us-west 1613707265 62

CPU.load host=webserver02,region=us-west 1613707265 43

CPU.load host=webserver02,region=us-west 1613707265 53

...

CPU.load host=webserver01,region=us-west 1613707265 76

CPU.load host=webserver01,region=us-west 1613707265 83
```

This is an example data we might pull from storage to answer that question.
The average CPU load can be calculated by averaging the values in the last column of the rows.

The format shown above is called the line protocol and is used by many popular monitoring software in the market - eg Prometheus, OpenTSDB.

What every time series consists of:

Show Image

A good way to visualize how data looks like:

Show Image

- The x axis is the time
- the y axis is the dimension you're querying - eg metric name, tag, etc.

The data access pattern is write-heavy and spiky reads as we collect a lot of metrics, but they are infrequently accessed, although in bursts when eg there are ongoing incidents.

The data storage system is the heart of this design.

- It is not recommended to use a general-purpose database for this problem, although you could achieve good scale \w expert-level tuning.
- Using a NoSQL database can work in theory, but it is hard to devise a scalable schema for effectively storing and querying time-series data.

There are many databases, specifically tailored for storing time-series data. Many of them support custom query interfaces which allow for effective querying of time-series data.

- OpenTSDB is a distributed time-series database, but it is based on Hadoop and HBase. If you don't have that infrastructure provisioned, it would be hard to use this tech.
- Twitter uses MetricsDB, while Amazon offers Timestream.
- The two most popular time-series databases are InfluxDB and Prometheus.
- They are designed to store large volumes of time-series data. Both of them are based on in-memory cache + on-disk storage.

Example scale of InfluxDB - more than 250k writes per second when provisioned with 8 cores and 32gb RAM:

Show Image

It is not expected for you to understand the internals of a metrics database as it is niche knowledge. You might be asked only if you've mentioned it on your resume.

For the purposes of the interview, it is sufficient to understand that metrics are time-series data and to be aware of popular time-series databases, like InfluxDB.

One nice feature of time-series databases is the efficient aggregation and analysis of large amounts of time-series data by labels.
InfluxDB, for example, builds indexes for each label.

It is critical, however, to keep the cardinality of labels low - ie, not using too many unique labels.

High-level Design

Show Image

- Metrics source - can be application servers, SQL databases, message queues, etc.
- Metrics collector - Gathers metrics data and writes to time-series database
- Time-series database - stores metrics as time-series. Provides a custom query interface for analyzing large amounts of metrics.
- Query service - Makes it easy to query and retrieve data from the time-series DB. Could be replaced entirely by the DB's interface if it's sufficiently powerful.

- Alerting system - Sends alert notifications to various alerting destinations.
- Visualization system - Shows metrics in the form of graphs/charts.

Step 3 - Design Deep Dive

Let's deep dive into several of the more interesting parts of the system.

Metrics collection

For metrics collection, occasional data loss is not critical. It's acceptable for clients to fire and forget.

Show Image

There are two ways to implement metrics collection - pull or push.

Here's how the pull model might look like:

Show Image

For this solution, the metrics collector needs to maintain an up-to-date list of services and metrics endpoints.
We can use Zookeeper or etcd for that purpose - service discovery.

Service discovery contains contains configuration rules about when and where to collect metrics from:

Show Image

Here's a detailed explanation of the metrics collection flow:

Show Image

- Metrics collector fetches configuration metadata from service discovery. This includes pulling interval, IP addresses, timeout & retry params.
- Metrics collector pulls metrics data via a pre-defined http endpoint (eg `/metrics`). This is typically done by a client library.
- Alternatively, the metrics collector can register a change event notification with the service discovery to be notified once the service endpoint changes.
- Another option is for the metrics collector to periodically poll for metrics endpoint configuration changes.

At our scale, a single metrics collector is not enough. There must be multiple instances. However, there must also be some kind of synchronization among them so that two collectors don't collect the same metrics twice.

One solution for this is to position collectors and servers on a consistent hash ring and associate a set of servers with a single collector only:

Show Image

With the push model, on the other hand, services push their metrics to the metrics collector proactively:

Show Image

In this approach, typically a collection agent is installed alongside service instances. The agent collects metrics from the server and pushes them to the metrics collector.

Show Image

With this model, we can potentially aggregate metrics before sending them to the collector, which reduces the volume of data processed by the collector.

On the flip side, metrics collector can reject push requests as it can't handle the load.
It is important, hence, to add the collector to an auto-scaling group behind a load balancer.

so which one is better? There are trade-offs between both approaches and different systems use different approaches:

- Prometheus uses a pull architecture
- Amazon Cloud Watch and Graphite use a push architecture

Here are some of the main differences between push and pull:

	Pull	**Push**
Easy debugging	The /metrics endpoint on application servers used for pulling metrics can be used to view metrics at any time. You can even do this on your laptop. Pull wins.	If the metrics collector doesn't receive metrics, the problem might be caused by network issues.
Health check	If an application server doesn't respond to the pull, you can quickly figure out if an application server is down. Pull wins.	If the metrics collector doesn't receive metrics, the problem might be caused by network issues.
Short-lived jobs		Some of the batch jobs might be short-lived and don't last long enough to be pulled. Push wins. This can be fixed by introducing push gateways for the pull model [22].
Firewall or complicated network setups	Having servers pulling metrics requires all metric endpoints to be reachable. This is potentially problematic in multiple data center setups. It might require a more elaborate network infrastructure.	If the metrics collector is set up with a load balancer and an auto-scaling group, it is possible to receive data from anywhere. Push wins.
Performance	Pull methods typically use TCP.	Push methods typically use UDP. This means the push method provides lower-latency transports of metrics. The counterargument here is that the effort of establishing a TCP connection is small compared to sending the metrics payload.
Data authenticity	Application servers to collect metrics from are defined in config files in advance. Metrics gathered from those servers are guaranteed to be authentic.	Any kind of client can push metrics to the metrics collector. This can be fixed by whitelisting servers from which to accept metrics, or by requiring authentication.

There is no clear winner. A large organization probably needs to support both. There might not be a way to install a push agent in the first place.

Scale the metrics transmission pipeline

Show Image

The metrics collector is provisioned in an auto-scaling group, regardless if we use the push or pull model.

There is a chance of data loss if the time-series DB is down, however. To mitigate this, we'll provision a queuing mechanism:

Show Image

- Metrics collectors push metrics data into kafka
- Consumers or stream processing services such as Apache Storm, Flink or Spark process the data and push it to the time-series DB

This approach has several advantages:

- Kafka is used as a highly-reliable and scalable distributed message platform
- It decouples data collection and data processing from one another
- It can prevent data loss by retaining the data in Kafka

Kafka can be configured with one partition per metric name, so that consumers can aggregate data by metric names. To scale this, we can further partition by tags/labels and categorize/prioritize metrics to be collected first.

Show Image

The main downside of using Kafka for this problem is the maintenance/operation overhead. An alternative is to use a large-scale ingestion system like Gorilla. It can be argued that using that would be as scalable as using Kafka for queuing.

Where aggregations can happen

Metrics can be aggregated at several places. There are trade-offs between different choices:

- Collection agent - client-side collection agent only supports simple aggregation logic. Eg collect a counter for 1m and send it to the metrics collector.
- Ingestion pipeline - To aggregate data before writing to the DB, we need a stream processing engine like Flink. This reduces write volume, but we lose data precision as we don't store raw data.
- Query side - We can aggregate data when we run queries via our visualization system. There is no data loss, but queries can be slow due to a lot of data processing.

Query Service

Having a separate query service from the time-series DB decouples the visualization and alerting system from the database, which enables us to decouple the DB from clients and change it at will.

We can add a Cache layer here to reduce the load to the time-series database:

Show Image

We can also avoid adding a query service altogether as most visualization and alerting systems have powerful plugins to integrate with most time-series databases.
With a well-chosen time-series DB, we might not need to introduce our own caching layer as well.

Most time-series DBs don't support SQL simply because it is ineffective for querying time-series data. Here's an example SQL query for computing an exponential moving average:

```
select id,
       temp,
       avg(temp) over (partition by group_nr order by time_read) as rolling_avg
from (
  select id,
         temp,
         time_read,
         interval_group,
         id - row_number() over (partition by interval_group order by time_read)
as group_nr
  from (
    select id,
    time_read,
    "epoch"::timestamp + "900 seconds"::interval * (extract(epoch from
time_read)::int4 / 900) as interval_group,
    temp
    from readings
  ) t1
) t2
order by time_read;
```

Here's the same query in Flux - query language used in InfluxDB:

```
from(db:"telegraf")
  |> range(start:-1h)
  |> filter(fn: (r) => r._measurement == "foo")
  |> exponentialMovingAverage(size:-10s)
```

Storage layer

It is important to choose the time-series database carefully.

According to research published by Facebook, ~85% of queries to the operational store were for data from the past 26h.

If we choose a database, which harnesses this property, it could have significant impact on system performance. InfluxDB is one such option.

Regardless of the database we choose, there are some optimizations we might employ.

Data encoding and compression can significantly reduce the size of data. Those features are usually built into a good time-series database.

Show Image

In the above example, instead of storing full timestamps, we can store timestamp deltas.

Another technique we can employ is down-sampling - converting high-resolution data to low-resolution in order to reduce disk usage.

We can use that for old data and make the rules configurable by data scientists, eg:

- 7d - no down-sampling
- 30d - down-sample to 1min
- 1y - down-sample to 1h

For example, here's a 10-second resolution metrics table:

metric	timestamp	hostname	Metric_value
cpu	2021-10-24T19:00:00Z	host-a	10
cpu	2021-10-24T19:00:10Z	host-a	16
cpu	2021-10-24T19:00:20Z	host-a	20
cpu	2021-10-24T19:00:30Z	host-a	30
cpu	2021-10-24T19:00:40Z	host-a	20
cpu	2021-10-24T19:00:50Z	host-a	30

down-sampled to 30-second resolution:

metric	timestamp	hostname	Metric_value (avg)
cpu	2021-10-24T19:00:00Z	host-a	19
cpu	2021-10-24T19:00:30Z	host-a	25

Finally, we can also use cold storage to use old data, which is no longer used. The financial cost for cold storage is much lower.

Alerting system

Show Image

Configuration is loaded to cache servers. Rules are typically defined in YAML format. Here's an example:

```
- name: instance_down
  rules:

  # Alert for any instance that is unreachable for >5 minutes.
  - alert: instance_down
    expr: up == 0
    for: 5m
    labels:
      severity: page
```

The alert manager fetches alert configurations from cache. Based on configuration rules, it also calls the query service at a predefined interval.
If a rule is met, an alert event is created.

Other responsibilities of the alert manager are:

- Filtering, merging and deduplicating alerts. Eg if an alert of a single instance is triggered multiple times, only one alert event is generated.
- Access control - it is important to restrict alert-management operations to certain individuals only
- Retry - the manager ensures that the alert is propagated at least once.

The alert store is a key-value database, like Cassandra, which keeps the state of all alerts. It ensures a notification is sent at least once.
Once an alert is triggered, it is published to Kafka.

Finally, alert consumers pull alerts data from Kafka and send notifications over to different channels - Email, text message, PagerDuty, webhooks.

In the real-world, there are many off-the-shelf solutions for alerting systems. It is difficult to justify building your own system in-house.

Visualization system

The visualization system shows metrics and alerts over a time period. Here's an dashboard built

with Grafana:

Show Image

A high-quality visualization system is very hard to build. It is hard to justify not using an off-the-shelf solution like Grafana.

Step 4 - Wrap up

Here's our final design:

Show Image

Chapter 22

Ad Click Event Aggregation

Digital advertising is a big industry with the rise of Facebook, YouTube, TikTok, etc.

Hence, tracking ad click events is important. In this chapter, we explore how to design an ad click event aggregation system at Facebook/Google scale.

Digital advertising has a process called real-time bidding (RTB), where digital advertising inventory is bought and sold:

Show Image

Speed of RTB is important as it usually occurs within a second.
Data accuracy is also very important as it impacts how much money advertisers pay.

Based on ad click event aggregations, advertisers can make decisions such as adjust target audience and keywords.

Step 1 - Understand the Problem and Establish Design Scope

- C: What is the format of the input data?

- I: 1bil ad clicks per day and 2mil ads in total. Number of ad-click events grows 30% year-over-year.
- C: What are some of the most important queries our system needs to support?
- I: Top queries to take into consideration:
  - Return number of click events for ad X in last Y minutes
  - Return top 100 most clicked ads in the past 1min. Both parameters should be configurable. Aggregation occurs every minute.
  - Support data filtering by `ip`, `user_id`, `country` for the above queries
- C: Do we need to worry about edge cases? Some of the ones I can think of:
  - There might be events that arrive later than expected
  - There might be duplicate events
  - Different parts of the system might be down, so we need to consider system recovery
- I: That's a good list, take those into consideration
- C: What is the latency requirement?
- I: A few minutes of e2e latency for ad click aggregation. For RTB, it is less than a second. It is ok to have that latency for ad click aggregation as those are usually used for billing and reporting.

Functional requirements

- Aggregate the number of clicks of `ad_id` in the last Y minutes
- Return top 100 most clicked `ad_id` every minute
- Support aggregation filtering by different attributes
- Dataset volume is at Facebook or Google scale

Non-functional requirements

- Correctness of the aggregation result is important as it's used for RTB and ads billing
- Properly handle delayed or duplicate events
- Robustness - system should be resilient to partial failures
- Latency - a few minutes of e2e latency at most

Back-of-the-envelope estimation

- 1bil DAU
- Assuming user clicks 1 ad per day -> 1bil ad clicks per day
- Ad click QPS = 10,000
- Peak QPS is 5 times the number = 50,000

- A single ad click occupies 0.1KB storage. Daily storage requirement is 100gb
- Monthly storage = 3tb

Step 2 - Propose High-Level Design and Get Buy-In

In this section, we discuss query API design, data model and high-level design.

Query API Design

The API is a contract between the client and the server. In our case, the client is the dashboard user - data scientist/analyst, advertiser, etc.

Here's our functional requirements:

- Aggregate the number of clicks of `ad_id` in the last Y minutes
- Return top N most clicked `ad_id` in the last M minutes
- Support aggregation filtering by different attributes

We need two endpoints to achieve those requirements. Filtering can be done via query parameters on one of them.

Aggregate number of clicks of ad_id in the last M minutes:

```
GET /v1/ads/{:ad_id}/aggregated_count
```

Query parameters:

- from - start minute. Default is now - 1 min
- to - end minute. Default is now
- filter - identifier for different filtering strategies. Eg 001 means "non-US clicks".

Response:

- ad_id - ad identifier
- count - aggregated count between start and end minutes

Return top N most clicked ad_ids in the last M minutes

```
GET /v1/ads/popular_ads
```

Query parameters:

- count - top N most clicked ads
- window - aggregation window size in minutes
- filter - identifier for different filtering strategies

Response:

- list of ad_ids

Data model

In our system, we have raw and aggregated data.

Raw data looks like this:

```
[AdClickEvent] ad001, 2021-01-01 00:00:01, user 1, 207.148.22.22, USA
```

Here's an example in a structured format:

ad_id	click_timestamp	user	ip	country
ad001	2021-01-01 00:00:01	user1	207.148.22.22	USA
ad001	2021-01-01 00:00:02	user1	207.148.22.22	USA
ad002	2021-01-01 00:00:02	user2	209.153.56.11	USA

Here's the aggregated version:

ad_id	click_minute	filter_id	count
ad001	202101010000	0012	2
ad001	202101010000	0023	3
ad001	202101010001	0012	1
ad001	202101010001	0023	6

The `filter_id` helps us achieve our filtering requirements.

filter_id	region	IP	user_id
0012	US	*	*
0013	*	123.1.2.3	*

To support quickly returning top N most clicked ads in the last M minutes, we'll also maintain this structure:

most_clicked_ads

window_size	integer	The aggregation window size (M) in minutes
update_time_minute	timestamp	Last updated timestamp (in 1-minute granularity)
most_clicked_ads	array	List of ad IDs in JSON format.

What are some pros and cons between storing raw data and storing aggregated data?

- Raw data enables using the full data set and supports data filtering and recalculation
- On the other hand, aggregated data allows us to have a smaller data set and a faster query
- Raw data means having a larger data store and a slower query
- Aggregated data, however, is derived data, hence there is some data loss.

In our design, we'll use a combination of both approaches:

- It's a good idea to keep the raw data around for debugging. If there is some bug in aggregation, we can discover the bug and backfill.
- Aggregated data should be stored as well for faster query performance.
- Raw data can be stored in cold storage to avoid extra storage costs.

When it comes to the database, there are several factors to take into consideration:

- What does the data look like? Is it relational, document or blob?
- Is the workload read-heavy, write-heavy or both?
- Are transactions needed?
- Do the queries rely on OLAP functions like SUM and COUNT?

For the raw data, we can see that the average QPS is 10k and peak QPS is 50k, so the system is write-heavy.
On the other hand, read traffic is low as raw data is mostly used as backup if anything goes wrong.

Relational databases can do the job, but it can be challenging to scale the writes.
Alternatively, we can use Cassandra or InfluxDB which have better native support for heavy write loads.

Another option is to use Amazon S3 with a columnar data format like ORC, Parquet or AVRO.
Since this setup is unfamiliar, we'll stick to Cassandra.

For aggregated data, the workload is both read and write heavy as aggregated data is constantly queried for dashboards and alerts.

It is also write-heavy as data is aggregated and written every minute by the aggregation service. Hence, we'll use the same data store (Cassandra) here as well.

High-level design

Here's how our system looks like:

Show Image

Data flows as an unbounded data stream on both inputs and outputs.

In order to avoid having a synchronous sink, where a consumer crashing can cause the whole system to stall, we'll leverage asynchronous processing using message queues (Kafka) to decouple consumers and producers.

Show Image

The first message queue stores ad click event data:

ad_id	click_timestamp	user_id	ip	country

The second message queue contains ad click counts, aggregated per-minute:

ad_id	click_minute	count

As well as top N clicked ads aggregated per minute:

update_time_minute	most_clicked_ads

The second message queue is there in order to achieve end to end exactly-once atomic commit semantics:

Show Image

For the aggregation service, using the MapReduce framework is a good option:

Show Image

Show Image

Each node is responsible for one single task and it sends the processing result to the downstream node.

The map node is responsible for reading from the data source, then filtering and transforming the data.

For example, the map node can allocate data across different aggregation nodes based on the `ad_id`:

Show Image

Alternatively, we can distribute ads across Kafka partitions and let the aggregation nodes subscribe directly within a consumer group.
However, the mapping node enables us to sanitize or transform the data before subsequent processing.

Another reason might be that we don't have control over how data is produced, so events related to the same `ad_id` might go on different partitions.

The aggregate node counts ad click events by `ad_id` in-memory every minute.

The reduce node collects aggregated results from aggregate node and produces the final result:

Show Image

This DAG model uses the MapReduce paradigm. It takes big data and leverages parallel distributed computing to turn it into regular-sized data.

In the DAG model, intermediate data is stored in-memory and different nodes communicate with each other using TCP or shared memory.

Let's explore how this model can now help us to achieve our various use-cases.

Use-case 1 - aggregate the number of clicks:

Show Image

- Ads are partitioned using `ad_id % 3`

Use-case 2 - return top N most clicked ads:

Show Image

- In this case, we're aggregating the top 3 ads, but this can be extended to top N ads easily
- Each node maintains a heap data structure for fast retrieval of top N ads

Use-case 3 - data filtering: To support fast data filtering, we can predefine filtering criterias and pre-aggregate based on it:

ad_id	click_minute	country	count
ad001	202101010001	USA	100
ad001	202101010001	GPB	200
ad001	202101010001	others	3000
ad002	202101010001	USA	10
ad002	202101010001	GPB	25
ad002	202101010001	others	12

This technique is called the **star schema** and is widely used in data warehouses. The filtering fields are called **dimensions**.

This approach has the following benefits:

- Simple to undertand and build
- Current aggregation service can be reused to create more dimensions in the star schema.
- Accessing data based on filtering criteria is fast as results are pre-calculated

A limitation of this approach is that it creates many more buckets and records, especially when we have lots of filtering criterias.

Step 3 - Design Deep Dive

Let's dive deeper into some of the more interesting topics.

Streaming vs. Batching

The high-level architecture we proposed is a type of stream processing system.
Here's a comparison between three types of systems:

	Services (Online system)	**Batch system (offline system)**	**Streaming system (near real-time system)**
Responsiveness	Respond to the client quickly	No response to the client needed	No response to the client needed
Input	User requests	Bounded input with finite size. A large amount of data	Input has no boundary (infinite streams)
Output	Responses to clients	Materialized views, aggregated metrics, etc.	Materialized views, aggregated metrics, etc.
Performance measurement	Availability, latency	Throughput	Throughput, latency
Example	Online shopping	MapReduce	Flink [13]

In our design, we used a mixture of batching and streaming.

We used streaming for processing data as it arrives and generates aggregated results in near real-time.
We used batching, on the other hand, for historical data backup.

A system which contains two processing paths - batch and streaming, simultaneously, this architecture is called lambda.
A disadvantage is that you have two processing paths with two different codebases to maintain.

Kappa is an alternative architecture, which combines batch and stream processing in one processing path.
The key idea is to use a single stream processing engine.

Lambda architecture:

Show Image

Kappa architecture:

Show Image

Our high-level design uses Kappa architecture as reprocessing of historical data also goes through the aggregation service.

Whenever we have to recalculate aggregated data due to eg a major bug in aggregation logic, we can recalculate the aggregation from the raw data we store.

- Recalculation service retrieves data from raw storage. This is a batch job.
- Retrieved data is sent to a dedicated aggregation service, so that the real-time processing aggregation service is not impacted.
- Aggregated results are sent to the second message queue, after which we update the results in the aggregation database.

  Show Image

Time

We need a timestamp to perform aggregation. It can be generated in two places:

- event time - when ad click occurs
- Processing time - system time when the server processes the event

Due to the usage of async processing (message queues) and network delays, there can be significant difference between event time and processing time.

- If we use processing time, aggregation results can be inaccurate
- If we use event time, we have to deal with delayed events

There is no perfect solution, we need to consider trade-offs:

	Pros	Cons
Event time	Aggregation results are more accurate	Clients might have the wrong time or timestamp might be generated by malicious users
Processing time	Server timestamp is more reliable	The timestamp is not accurate if event is late

Since data accuracy is important, we'll use the event time for aggregation.

To mitigate the issue of delayed events, a technique called "watermark" can be leveraged.

In the example below, event 2 misses the window where it needs to be aggregated:

Show Image

However, if we purposefully extend the aggregation window, we can reduce the likelihood of missed events. The extended part of a window is called a "watermark":

Show Image

- Short watermark increases likelihood of missed events, but reduces latency
- Longer watermark reduces likelihood of missed events, but increases latency

There is always likelihood of missed events, regardless of the watermark's size. But there is no use in optimizing for such low-probability events.

We can instead resolve such inconsistencies by doing end-of-day reconciliation.

Aggregation window

There are four types of window functions:

- Tumbling (fixed) window
- Hopping window
- Sliding window
- Session window

In our design, we leverage a tumbling window for ad click aggregations:

Show Image

As well as a sliding window for the top N clicked ads in M minutes aggregation:

Show Image

Delivery guarantees

Since the data we're aggregating is going to be used for billing, data accuracy is a priority.

Hence, we need to discuss:

- How to avoid processing duplicate events
- How to ensure all events are processed

There are three delivery guarantees we can use - at-most-once, at-least-once and exactly once.

In most circumstances, at-least-once is sufficient when a small amount of duplicates is acceptable.
This is not the case for our system, though, as a difference in small percent can result in millions of dollars of discrepancy.
Hence, we'll need to use exactly-once delivery semantics.

Data deduplication

One of the most common data quality issues is duplicated data.

It can come from a wide range of sources:

- Client-side - a client might resend the same event multiple times. Duplicated events sent with malicious intent are best handled by a risk engine.
- Server outage - An aggregation service node goes down in the middle of aggregation and the upstream service hasn't received an acknowledgment so event is resent.

Here's an example of data duplication occurring due to failure to acknowledge an event on the last hop:

Show Image

In this example, offset 100 will be processed and sent downstream multiple times.

One option to try and mitigate this is to store the last seen offset in HDFS/S3, but this risks the result never reaching downstream:

Show Image

Finally, we can store the offset while interacting with downstream atomically. To achieve this, we need to implement a distributed transaction:

Show Image

Personal side-note: Alternatively, if the downstream system handles the aggregation result idempotently, there is no need for a distributed transaction.

Scale the system

Let's discuss how we scale the system as it grows.

We have three independent components - message queue, aggregation service and database. Since they are decoupled, we can scale them independently.

How do we scale the message queue:

- We don't put a limit on producers, so they can be scaled easily
- Consumers can be scaled by assigning them to consumer groups and increasing the number of consumers.
- For this to work, we also need to ensure there are enough partitions created preemptively
- Also, consumer rebalancing can take a while when there are thousands of consumers so it is recommended to do it off peak hours
- We could also consider partitioning the topic by geography, eg `topic_na`, `topic_eu`, etc.

Show Image

How do we scale the aggregation service:

Show Image

- The map-reduce nodes can easily be scaled by adding more nodes
- The throughput of the aggregation service can be scaled by by utilising multi-threading
- Alternatively, we can leverage resource providers such as Apache YARN to utilize multi-processing
- Option 1 is easier, but option 2 is more widely used in practice as it's more scalable
- Here's the multi-threading example:

  Show Image

How do we scale the database:

- If we use Cassandra, it natively supports horizontal scaling utilizing consistent hashing
- If a new node is added to the cluster, data automatically gets rebalanced across all (virtual) nodes
- With this approach, no manual (re)sharding is required

  Show Image

Another scalability issue to consider is the hotspot issue - what if an ad is more popular and gets more attention than others?

Show Image

- In the above example, aggregation service nodes can apply for extra resources via the resource manager
- The resource manager allocates more resources, so the original node isn't overloaded
- The original node splits the events into 3 groups and each of the aggregation nodes handles 100 events
- Result is written back to the original aggregation node

Alternative, more sophisticated ways to handle the hotspot problem:

- Global-Local Aggregation
- Split Distinct Aggregation

Fault Tolerance

Within the aggregation nodes, we are processing data in-memory. If a node goes down, the processed data is lost.

We can leverage consumer offsets in kafka to continue from where we left off once another node picks up the slack.
However, there is additional intermediary state we need to maintain, as we're aggregating the top N ads in M minutes.

We can make snapshots at a particular minute for the on-going aggregation:

Show Image

If a node goes down, the new node can read the latest committed consumer offset, as well as the latest snapshot to continue the job:

Show Image

Data monitoring and correctness

As the data we're aggregating is critical as it's used for billing, it is very important to have rigorous monitoring in place in order to ensure correctness.

Some metrics we might want to monitor:

- Latency - Timestamps of different events can be tracked in order to understand the e2e latency of the system
- Message queue size - If there is a sudden increase in queue size, we need to add more aggregation nodes. As Kafka is implemented via a distributed commit log, we need to keep track of records-lag metrics instead.
- System resources on aggregation nodes - CPU, disk, JVM, etc.

We also need to implement a reconciliation flow which is a batch job, running at the end of the day. It calculates the aggregated results from the raw data and compares them against the actual data stored in the aggregation database:

Show Image

Alternative design

In a generalist system design interview, you are not expected to know the internals of specialized software used in big data processing.

Explaining the thought process and discussing trade-offs is more important than knowing specific tools, which is why the chapter covers a generic solution.

An alternative design, which leverages off-the-shelf tooling, is to store ad click data in Hive with an ElasticSearch layer on top built for faster queries.

Aggregation is typically done in OLAP databases such as ClickHouse or Druid.

Show Image

Step 4 - Wrap up

Things we covered:

- Data model and API Design

- Using MapReduce to aggregate ad click events
- Scaling the message queue, aggregation service and database
- Mitigating the hotspot issue
- Monitoring the system continuously
- Using reconciliation to ensure correctness
- Fault tolerance

The ad click event aggregation is a typical big data processing system.

It would be easier to understand and design it if you have prior knowledge of related technologies:

- Apache Kafka
- Apache Spark
- Apache Flink

Chapter 23

Hotel Reservation System

In this chapter, we're designing a hotel reservation system, similar to Marriott International.

Applicable to other types of systems as well - Airbnb, flight reservation, movie ticket booking.

Step 1 - Understand the Problem and Establish Design Scope

Before diving into designing the system, we should ask the interviewer questions to clarify the scope:

- C: What is the scale of the system?
- I: We're building a website for a hotel chain \w 5000 hotels and 1mil rooms
- C: Do customers pay when they make a reservation or when they arrive at the hotel?
- I: They pay in full when making reservations.
- C: Do customers book hotel rooms through the website only? Do we have to support other reservation options such as phone calls?
- I: They make bookings through the website or app only.
- C: Can customers cancel reservations?
- I: Yes
- C: Other things to consider?
- I: Yes, we allow overbooking by 10%. Hotel will sell more rooms than there actually are. Hotels do this in anticipation that clients will cancel bookings.

- C: Since not much time, we'll focus on - show hotel-related page, hotel-room details page, reserve a room, admin panel, support overbooking.
- I: Sounds good.
- I: One more thing - hotel prices change all the time. Assume a hotel room's price changes every day.
- C: OK.

Non-functional requirements

- Support high concurrency - there might be a lot of customers trying to book the same hotel during peak season.
- Moderate latency - it's ideal to have low latency when a user makes a reservation, but it's acceptable if the system takes a few seconds to process it.

Back-of-the-envelope estimation

- 5000 hotels and 1mil rooms in total
- Assume 70% of rooms are occupied and average stay duration is 3 days
- Estimated daily reservations - 1mil * 0.7 / 3 = ~240k reservations per day
- Reservations per second - 240k / 10^5 seconds in a day = ~3. Average reservation TPS is low.

Let's estimate the QPS. If we assume that there are three steps to reach the reservation page and there is a 10% conversion rate per page, we can estimate that if there are 3 reservations, then there must be 30 views of reservation page and 300 views of hotel room detail page.

Show Image

Step 2 - Propose High-Level Design and Get Buy-In

We'll explore - API Design, Data model, high-level design.

API Design

This API Design focuses on the core endpoints (using RESTful practices), we'll need in order to support a hotel reservation system.

A fully-fledged system would require a more extensive API with support for searching for rooms based on lots of criteria, but we won't be focusing on that in this section.
Reason is that they aren't technically challenging, so they're out of scope.

Hotel-related API

- `GET /v1/hotels/{id}` - get detailed info about a hotel
- `POST /v1/hotels` - add a new hotel. Only available to ops
- `PUT /v1/hotels/{id}` - update hotel info. Only available to ops
- `DELETE /v1/hotels/{id}` - delete a hotel. API is only available to ops

Room-related API

- `GET /v1/hotels/{id}/rooms/{id}` - get detailed information about a room
- `POST /v1/hotels/{id}/rooms` - Add a room. Only available to ops
- `PUT /v1/hotels/{id}/rooms/{id}` - Update room info. Only available to ops
- `DELETE /v1/hotels/{id}/rooms/{id}` - Delete a room. Only available to ops

Reservation-related API

- `GET /v1/reservations` - get reservation history of current user
- `GET /v1/reservations/{id}` - get detailed info about a reservation
- `POST /v1/reservations` - make a new reservation
- `DELETE /v1/reservations/{id}` - cancel a reservation

Here's an example request to make a reservation:

```
{
  "startDate":"2021-04-28",
  "endDate":"2021-04-30",
  "hotelID":"245",
  "roomID":"U12354673389",
  "reservationID":"13422445"
}
```

Note that the `reservationID` is an idempotency key to avoid double booking. Details explained in concurrency section

Data model

Before we choose what database to use, let's consider our access patterns.

We need to support the following queries:

- View detailed info about a hotel
- Find available types of rooms given a date range
- Record a reservation
- Look up a reservation or past history of reservations

From our estimations, we know the scale of the system is not large, but we need to prepare for traffic surges.

Given this knowledge, we'll choose a relational database because:

- Relational DBs work well with read-heavy and less write-heavy systems.
- NoSQL databases are normally optimized for writes, but we know we won't have many as only a fraction of users who visit the site make a reservation.
- Relational DBs provide ACID guarantees. These are important for such a system as without them, we won't be able to prevent problems such as negative balance, double charge, etc.
- Relational DBs can easily model the data as the structure is very clear.

Here is our schema design:

Show Image

Most fields are self-explanatory. Only field worth mentioning is the `status` field which represents the state machine of a given room:

Show Image

This data model works well for a system like Airbnb, but not for hotels where users don't reserve a particular room but a room type.
They reserve a type of room and a room number is chosen at the point of reservation.

This shortcoming will be addressed in the Improved Data Model section.

High-level Design

We've chosen a microservice architecture for this design. It has gained great popularity in recent years:

Show Image

- Users book a hotel room on their phone or computer
- Admin perform administrative functions such as refunding/cancelling a payment, etc

- CDN caches static resources such as JS bundles, images, videos, etc
- Public API Gateway - fully-managed service which supports rate limiting, authentication, etc.
- Internal APIs - only visible to authorized personnel. Usually protected by a VPN.
- Hotel service - provides detailed information about hotels and rooms. Hotel and room data is static, so it can be cached aggressively.
- Rate service - provides room rates for different future dates. An interesting note about this domain is that prices depend on how full a hotel is at a given day.
- Reservation service - receives reservation requests and reserves hotel rooms. Also tracks room inventory as reservations are made/cancelled.
- Payment service - processes payments and updates reservation statuses on success.
- Hotel management service - available to authorized personnel only. Allows certain administrative functions for managing and viewing reservations, hotels, etc.

Inter-service communication can be facilitated via a RPC framework, such as gRPC.

Step 3 - Design Deep Dive

Let's dive deeper into:

- Improved data model
- Concurrency issues
- Scalability
- Resolving data inconsistency in microservices

Improved data model

As mentioned in a previous section, we need to amend our API and schema to enable reserving a type of room vs. a particular one.

For the reservation API, we no longer reserve a `roomID`, but we reserve a `roomTypeID`:

```
POST /v1/reservations
{
  "startDate":"2021-04-28",
  "endDate":"2021-04-30",
  "hotelID":"245",
  "roomTypeID":"12354673389",
  "roomCount":"3",
  "reservationID":"13422445"
}
```

Here's the updated schema:

Show Image

- room - contains information about a room
- room_type_rate - contains information about prices for a given room type
- reservation - records guest reservation data
- room_type_inventory - stores inventory data about hotel rooms.

Let's take a look at the `room_type_inventory` columns as that table is more interesting:

- hotel_id - id of hotel
- room_type_id - id of a room type
- date - a single date
- total_inventory - total number of rooms minus those that are temporarily taken off the inventory.
- total_reserved - total number of rooms booked for given (hotel_id, room_type_id, date)

There are alternative ways to design this table, but having one room per (hotel_id, room_type_id, date) enables easy
reservation management and easier queries.

The rows in the table are pre-populated using a daily CRON job.

Sample data:

hotel_id	room_type_id	date	total_inventory	total_reserved
211	1001	2021-06-01	100	80
211	1001	2021-06-02	100	82
211	1001	2021-06-03	100	86
211	1001	...	...	
211	1001	2023-05-31	100	0
211	1002	2021-06-01	200	16
2210	101	2021-06-01	30	23
2210	101	2021-06-02	30	25

Sample SQL query to check the availability of a type of room:

```
SELECT date, total_inventory, total_reserved
FROM room_type_inventory
WHERE room_type_id = ${roomTypeId} AND hotel_id = ${hotelId}
AND date between ${startDate} and ${endDate}
```

How to check availability for a specified number of rooms using that data (note that we support overbooking):

```
if (total_reserved + ${numberOfRoomsToReserve}) <= 110% * total_inventory
```

Now let's do some estimation about the storage volume.

- We have 5000 hotels.
- Each hotel has 20 types of rooms.
- 5000 * 20 * 2 (years) * 365 (days) = 73mil rows

73 million rows is not a lot of data and a single database server can handle it.
It makes sense, however, to setup read replication (potentially across different zones) to enable high availability.

Follow-up question - if reservation data is too large for a single database, what would you do?

- Store only current and future reservation data. Reservation history can be moved to cold storage.
- Database sharding - we can shard our data by `hash(hotel_id) % servers_cnt` as we always select the `hotel_id` in our queries.

Concurrency issues

Another important problem to address is double booking.

There are two issues to address:

- Same user clicks on "book" twice
- Multiple users try to book a room at the same time

Here's a visualization of the first problem:

Show Image

There are two approaches to solving this problem:

- Client-side handling - front-end can disable the book button once clicked. If a user disabled javascript, however, they won't see the button becoming grayed out.
- Idemptent API - Add an idempotency key to the API, which enables a user to execute an action once, regardless of how many times the endpoint is invoked:

  Show Image

Here's how this flow works:

- A reservation order is generated once you're in the process of filling in your details and making a booking. The reservation order is generated using a globally unique identifier.
- Submit reservation 1 using the `reservation_id` generated in the previous step.
- If "complete booking" is clicked a second time, the same `reservation_id` is sent and the backend detects that this is a duplicate reservation.
- The duplication is avoided by making the `reservation_id` column have a unique constraint, preventing multiple records with that id being stored in the DB.

  Show Image

What if there are multiple users making the same reservation?

Show Image

- Let's assume the transaction isolation level is not serializable
- User 1 and 2 attempt to book the same room at the same time.
- Transaction 1 checks if there are enough rooms - there are
- Transaction 2 check if there are enough rooms - there are
- Transaction 2 reserves the room and updates the inventory
- Transaction 1 also reserves the room as it still sees there are 99 `total_reserved` rooms out of 100.

- Both transactions successfully commit the changes

This problem can be solved using some form of locking mechanism:

- Pessimistic locking
- Optimistic locking
- Database constraints

Here's the SQL we use to reserve a room:

```sql
# step 1: check room inventory
SELECT date, total_inventory, total_reserved
FROM room_type_inventory
WHERE room_type_id = ${roomTypeId} AND hotel_id = ${hotelId}
AND date between ${startDate} and ${endDate}

# For every entry returned from step 1
if((total_reserved + ${numberOfRoomsToReserve}) > 110% * total_inventory) {
  Rollback
}

# step 2: reserve rooms
UPDATE room_type_inventory
SET total_reserved = total_reserved + ${numberOfRoomsToReserve}
WHERE room_type_id = ${roomTypeId}
AND date between ${startDate} and ${endDate}

Commit
```

Option 1: Pessimistic locking

Pessimistic locking prevents simultaneous updates by putting a lock on a record while it's being updated.

This can be done in MySQL by using the `SELECT... FOR UPDATE` query, which locks the rows selected by the query until the transaction is committed.

Show Image

Pros:

- Prevents applications from updating data that is being changed

- Easy to implement and avoids conflict by serializing updates. Useful when there is heavy data contention.

Cons:

- Deadlocks may occur when multiple resources are locked.
- This approach is not scalable - if transaction is locked for too long, this has impact on all other transactions trying to access the resource.
- The impact is severe when the query selects a lot of resources and the transaction is long-lived.

The author doesn't recommend this approach due to its scalability issues.

Option 2: Optimistic locking

Optimistic locking allows multiple users to attempt to update a record at the same time.

There are two common ways to implement it - version numbers and timestamps. Version numbers are recommended as server clocks can be inaccurate.

Show Image

- A new `version` column is added to the database table
- Before a user modifies a database row, the version number is read
- When the user updates the row, the version number is increased by 1 and written back to the database
- Database validation prevents the insert if the new version number doesn't exceed the previous one

Optimistic locking is usually faster than pessimistic locking as we're not locking the database. Its performance tends to degrade when concurrency is high, however, as that leads to a lot of rollbacks.

Pros:

- It prevents applications from editing stale data
- We don't need to acquire a lock in the database
- Preferred option when data contention is low, ie rarely are there update conflicts

Cons:

- Performance is poor when data contention is high

Optimistic locking is a good option for our system as reservation QPS is not extremely high.

Option 3: Database constraints

This approach is very similar to optimistic locking, but the guardrails are implemented using a database constraint:

```
CONSTRAINT `check_room_count` CHECK((`total_inventory - total_reserved` >= 0))
```

Show Image

Pros:

- Easy to implement
- Works well when data contention is small

Cons:

- Similar to optimistic locking, performs poorly when data contention is high
- Database constraints cannot be easily version-controlled like application code
- Not all databases support constraints

This is another good option for a hotel reservation system due to its ease of implementation.

Scalability

Usually, the load of a hotel reservation system is not high.

However, the interviewer might ask you how you'd handle a situation where the system gets adopted for a larger, popular travel site such as booking.com
In that case, QPS can be 1000 times larger.

When there is such a situation, it is important to understand where our bottlenecks are. All the services are stateless, so they can be easily scaled via replication.

The database, however, is stateful and it's not as obvious how it can get scaled.

One way to scale it is by implementing database sharding - we can split the data across multiple databases, where each of them contain a portion of the data.

We can shard based on `hotel_id` as all queries filter based on it. Assuming, QPS is 30,000, after sharding the database in 16 shards, each shard handles 1875 QPS, which is within a single MySQL cluster's load capacity.

Show Image

We can also utilize caching for room inventory and reservations via Redis. We can set TTL so that old data can expire for days which are past.

Show Image

The way we store an inventory is based on the `hotel_id`, `room_type_id` and `date`:

```
key: hotelID_roomTypeID_{date}
value: the number of available rooms for the given hotel ID, room type ID and
date.
```

Data consistency happens async and is managed by using a CDC streaming mechanism - database changes are read and applied to a separate system.
Debezium is a popular option for synchronizing database changes with Redis.

Using such a mechanism, there is a possibility that the cache and database are inconsistent for some time.
This is fine in our case because the database will prevent us from making an invalid reservation.

This will cause some issue on the UI as a user would have to refresh the page to see that "there are no more rooms left",
but that is something which can happen regardless of this issue if eg a person hesitates a lot before making a reservation.

Caching pros:

- Reduced database load
- High performance, as Redis manages data in-memory

Caching cons:

- Maintaining data consistency between cache and DB is hard. We need to consider how the inconsistency impacts user experience.

Data consistency among services

A monolithic application enables us to use a shared relational database for ensuring data

consistency.

In our microservice design, we chose a hybrid approach where some services are separate, but the reservation and inventory APIs are handled by the same servicefor the reservation and inventory APIs.

This is done because we want to leverage the relational database's ACID guarantees to ensure consistency.

However, the interviewer might challenge this approach as it's not a pure microservice architecture, where each service has a dedicated database:

Show Image

This can lead to consistency issues. In a monolithic server, we can leverage a relational DBs transaction capabilities to implement atomic operations:

Show Image

It's more challenging, however, to guarantee this atomicity when the operation spans across multiple services:

Show Image

There are some well-known techniques to handle these data inconsistencies:

- Two-phase commit - a database protocol which guarantees atomic transaction commit across multiple nodes. It's not performant, though, since a single node lag leads to all nodes blocking the operation.
- Saga - a sequence of local transactions, where compensating transactions are triggered if any of the steps in a workflow fail. This is an eventually consistent approach.

It's worth noting that addressing data inconsistencies across microservices is a challenging problem, which raise the system complexity.
It is good to consider whether the cost is worth it, given our more pragmatic approach of encapsulating dependent operations within the same relational database.

Step 4 - Wrap Up

We presented a design for a hotel reservation system.

These are the steps we went through:

- Gathering requirements and doing back-of-the-envelope calculations to understand the system's scale
- We presented the API Design, Data Model and system architecture in the high-level design
- In the deep dive, we explored alternative database schema designs as requirements changed
- We discussed race conditions and proposed solutions - pessimistic/optimistic locking, database constraints
- Ways to scale the system via database sharding and caching
- Finally we addressed how to handle data consistency issues across multiple microservices

---

Chapter 24

Distributed Email Service

We'll design a distributed email service, similar to gmail in this chapter.

In 2020, gmail had 1.8bil active users, while Outlook had 400mil users worldwide.

Step 1 - Understand the Problem and Establish Design Scope

- C: How many users use the system?
- I: 1bil users
- C: I think following features are important - auth, send/receive email, fetch email, filter emails, search email, anti-spam protection.
- I: Good list. Don't worry about auth for now.
- C: How do users connect \w email servers?
- I: Typically, email clients connect via SMTP, POP, IMAP, but we'll use HTTP for this problem.
- C: Can emails have attachments?
- I: Yes

Non-functional requirements

- Reliability - we shouldn't lose data
- Availability - We should use replication to prevent single points of failure. We should also tolerate partial system failures.

- Scalability - As userbase grows, our system should be able to handle them.
- Flexibility and extensibility - system should be flexible and easy to extend with new features. One of the reasons we chose HTTP over SMTP/other mail protocols.

Back-of-the-envelope estimation

- 1bil users
- Assuming one person sends 10 emails per day -> 100k emails per second.
- Assuming one person receives 40 emails per day and each email on average has 50kb metadata -> 730pb storage per year.
- Assuming 20% of emails have storage attachments and average size is 500kb -> 1,460pb per year.

Step 2 - Propose High-Level Design and Get Buy-In

Email knowledge 101

There are various protocols used for sending and receiving emails:

- SMTP - standard protocol for sending emails from one server to another.
- POP - standard protocol for receiving and downloading emails from a remote mail server to a local client. Once retrieved, emails are deleted from remote server.
- IMAP - similar to POP, it is used for receiving and downloading emails from a remote server, but it keeps the emails on the server-side.
- HTTPS - not technically an email protocol, but it can be used for web-based email clients.

Apart from the mailing protocol, there are some DNS records we need to configure for our email server - the MX records:

Show Image

Email attachments are sent base64-encoded and there is usually a size limit of 25mb on most mail services.
This is configurable and varies from individual to corporate accounts.

Traditional mail servers

Traditional mail servers work well when there are a limited number of users, connected to a single server.

Show Image

- Alice logs into her Outlook email and presses "send". Email is sent to Outlook mail server. Communication is via SMTP.
- Outlook server queries DNS to find MX record for gmail.com and transfers the email to their servers. Communication is via SMTP.
- Bob fetches emails from his gmail server via IMAP/POP.

In traditional mail servers, emails were stored on the local file system. Every email was a separate file.

Show Image

As the scale grew, disk I/O became a bottleneck. Also, it doesn't satisfy our high availability and reliability requirements.
Disks can be damaged and server can go down.

Distributed mail servers

Distributed mail servers are designed to support modern use-cases and solve modern scalability issues.

These servers can still support IMAP/POP for native email clients and SMTP for mail exchange across servers.

But for rich web-based mail clients, a RESTful API over HTTP is typically used.

Example APIs:

- `POST /v1/messages` - sends a message to recipients in To, Cc, Bcc headers.
- `GET /v1/folders` - returns all folders of an email account Example response:

```
[{id: string        Unique folder identifier.
  name: string      Name of the folder.
                    According to RFC6154 [9], the default folders can be one of
                    the following: All, Archive, Drafts, Flagged, Junk, Sent,
                    and Trash.
  user_id: string   Reference to the account owner
}]
```

- `GET /v1/folders/{:folder_id}/messages` - returns all messages under a folder \w pagination
- `GET /v1/messages/{:message_id}` - get all information about a particular message
  Example response:

```
{
  user_id: string                       // Reference to the account owner.
  from: {name: string, email: string}   // <name, email> pair of the sender.
  to: [{name: string, email: string}]   // A list of <name, email> paris
  subject: string                       // Subject of an email
  body: string                          //  Message body
  is_read: boolean                      //  Indicate if a message is read or not.
}
```

Here's the high-level design of the distributed mail server:

Show Image

- Webmail - users use web browsers to send/receive emails
- Web servers - public-facing request/response services used to manage login, signup, user profile, etc.
- Real-time servers - Used for pushing new email updates to clients in real-time. We use websockets for real-time communication but fallback to long-polling for older browsers that don't support them.
- Metadata db - stores email metadata such as subject, body, from, to, etc.
- Attachment store - Object store (eg Amazon S3), suitable for storing large files.
- Distributed cache - We can cache recent emails in Redis to improve UX.
- Search store - distributed document store, used for supporting full-text searches.

Here's what the email sending flow looks like:

Show Image

- User writes an email and presses "send". Email is sent to load balancer.
- Load balancer rate limits excessive mail sends and routes to one of the web servers.
- Web servers do basic email validation (eg email size) and short-circuits outbound flow if domain is same as sender. But does spam check first.
- If basic validation passes, email is sent to message queue (attachment is referenced from object store)
- If basic validation fails, email is sent to error queue
- SMTP outgoing workers pull messages from outgoing queue, do spam/virus checks and route to destination mail server.
- Email is stored in the "Sent Emails" folder

We need to also monitor size of outgoing message queue. Growing too large might indicate a problem:

- Recipient's mail server is unavailable. We can retry sending the email at a later time using exponential backoff.
- Not enough consumers to handle the load, we might have to scale the consumers.

Here's the email receiving flow:

Show Image

- Incoming emails arrive at the SMTP load balancer. Mails are distributed to SMTP servers, where mail acceptance policy is done (eg invalid emails are directly discarded).
- If attachment of email is too large, we can put it in object store (s3).
- Mail processing workers do preliminary checks, after which mails are forwarded to storage, cache, object store and real-time servers.
- Offline users get their new emails once they come back online via HTTP API.

Step 3 - Design Deep Dive

Let's now go deeper into some of the components.

Metadata database

Here are some of the characteristics of email metadata:

- headers are usually small and frequently accessed
- Body size ranges from small to big, but is typically read once
- Most mail operations are isolated to a single user - eg fetching email, marking as read, searching.
- Data recency impacts data usage. Users typically read only recent emails
- Data has high-reliability requirements. Data loss is unacceptable.

At gmail/outlook scale, the database is typically custom made to reduce input/output operations per second (IOPS).

Let's consider what database options we have:

- Relational database - we can build indexes for headers and body, but these DBs are typically optimized for small chunks of data.
- Distributed object store - this can be a good option for backup storage, but can't efficiently support searching/marking as read/etc.
- NoSQL - Google BigTable is used by gmail, but it's not open-sourced.

Based on above analysis, very few existing solutions seems to fit our needs perfectly.
In an interview setting, it's infeasible to design a new distributed database solution, but important to mention characteristics:

- Single column can be a single-digit MB
- Strong data consistency
- Designed to reduce disk I/O
- Highly available and fault tolerant
- Should be easy to create incremental backups

In order to partition the data, we can use the `user_id` as a partition key, so that one user's data is stored on a single shard. This prohibits us from sharing an email with multiple users, but this is not a requirement for this interview.

Let's define the tables:

- Primary key consists of partition key (data distribution) and clustering key (sorting data)
- Queries we need to support - get all folders for a user, display all emails for a folder, create/get/delete an email, fetch read/unread email, get conversation threads (bonus)

Legend for tables to follow:

Show Image

Here is the folders table:

Show Image

emails table:

Show Image

- email_id is timeuuid which allows sorting based on timestamp when email was created

Attachments are stored in a separate table, identified by filename:

Show Image

Supporting fetchin read/unread emails is easy in a traditional relational database, but not in Cassandra, since filtering on non-partition/clustering key is prohibited.
One workaround to fetch all emails in a folder and filter in-memory, but that doesn't work well for a big-enough application.

What we can do is denormalize the emails table into read/unread emails tables:

Show Image

In order to support conversation threads, we can include some headers, which mail clients interpret and use to reconstruct a conversation thread:

```
{
  "headers" {
     "Message-Id": "<7BA04B2A-430C-4D12-8B57-862103C34501@gmail.com>",
     "In-Reply-To": "<CAEWTXuPfN=LzECjDJtgY9Vu03kgFvJnJUSHTt6TW@gmail.com>",
     "References": ["<7BA04B2A-430C-4D12-8B57-862103C34501@gmail.com>"]
  }
}
```

Finally, we'll trade availability for consistency for our distributed database, since it is a hard requirement for this problem.

Hence, in the event of a failover or network parititon, sync/update actions will be briefly unavailable to impacted users.

Email deliverability

It is easy to setup a server to send emails, but getting the email to a receiver's inbox is hard, due to spam-protection algorithms.

If we just setup a new mail server and start sending mails through it, our emails will probably end up in the spam folder.

Here's what we can do to prevent that:

- Dedicated IPs - use dedicated IPs for sending emails, otherwise, recipient servers will not trust you.
- Classify emails - avoid sending marketing emails from the same servers to prevent more important email to be classified as spam
- Warm up your IP address slowly to build a good reputation with big email providers. It takes 2 to 6 weeks to warm up a new IP
- Ban spammers quickly to not deteriorate your reputation
- Feedback processing - setup a feedback loop with ISPs to keep track of complaint rate and ban spam accounts quickly.
- Email authentication - use common techniques to combat phishing such as Sender Policy Framework, DomainKeys Identified Mail, etc.

You don't need to remember all of this. Just know that building a good mail server requires a lot of domain knowledge.

Search

Searching includes doing a full-text search based on email contents or more advanced queries based on from, to, subject, unread, etc filters.

One characteristic of email search is that it is local to the user and it has more writes than reads, because we need to re-index it on each operation, but users rarely use the search tab.

Let's compare google search with email search:

	Scope	Sorting	Accuracy
Google search	The whole internet	Sort by relevance	Indexing takes some time, so not instant results.
Email search	User's own email box	Sort by attributes eg time, date, etc	Indexing should be quick and results accurate.

To achieve this search functionality, one option is to use an Elasticsearch cluster. We can use `user_id` as the partition key to group data under the same node:

Show Image

Mutating operations are async via Kafka in order to decouple services from the reindexing flow. Actually searching for data happens synchronously.

Elasticsearch is one of the most popular search-engine databases and supports full-text search for emails very well.

Alternatively, we can attempt to develop our own custom search solution to meet our specific requirements.

Designing such a system is out of scope. One of the core challenges when building it is to optimize it for write-heavy workloads.

To achieve that, we can use Log-Structured Merge-Trees (LSM) to structure the index data on disk. Write path is optimized for sequential writes only.
This technique is used in Cassandra, BigTable and RocksDB.

Its core idea is to store data in-memory until a predefined threshold is reached, after which it is merged in the next layer (disk):

Show Image

Main trade-offs between the two approaches:

- Elasticsearch scales to some extent, whereas a custom search engine can be fine-tuned for the email use-case, allowing it to scale further.

- Elasticsearch is a separate service we need to maintain, alongside the metadata store. A custom solution can be the datastore itself.
- Elasticsearch is an off-the-shelf solution, whereas the custom search engine would require significant engineering effort to build.

Scalability and availability

Since individual user operations don't collide with other users, most components can be independently scaled.

To ensure high availability, we can also use a multi-DC setup with leader-folower failover in case of failures:

Show Image

Step 4 - Wrap up

Additional talking points:

- Fault tolerance - Many parts of the system could fail. It is worthwhile how we'd handle node failures.
- Compliance - PII needs to be stored in a reasonable way, given Europe's GDPR laws.
- Security - email encryption, phishing protection, safe browsing, etc.
- Optimizations - eg preventing duplication of the same attachments, sent multiple times by different users.

Chapter 25

S3-like Object Storage

In this chapter, we'll be designing an object storage service, similar to Amazon S3.

Storage systems fall into three broad categories:

- Block storage
- File storage
- Object storage

Block storage are devices, which came out in 1960s. HDDs and SSDs are such examples.
These devices are typically physically attached to a server, although they can also be network-

attached via high-speed network protocols.
Servers can format the raw blocks and use them as a file system or it can hand control of them to servers directly.

File storage is built on top of block storage. It provides a higher level of abstraction, making it easier to manage folders and files.

Object storage sacrifices performance for high durability, vast scale and low cost. It targets "cold" data and is mainly used for archival and backup. There is no hierarchical directory structure, all data is stored as objects in a flat structure. It is relatively slow compared to other storage types. Most cloud providers have an object storage offering - Amazon S3, Google GCS, etc.

Show Image

	Block Storage	**File Storage**	**Object Storage**
Mutable Content	Y	Y	N (has object versioning)
Cost	High	Medium to high	Low
Performance	Medium to high, very high	Medium to high	Low to medium
Consistency	Strong consistency	Strong consistency	Strong consistency [5]
Data access	SAS/iSCSI/FC	Standard file access, CIFS/SMB, and NFS	RESTful API
Scalability	Medium scalability	High scalability	Vast scalability
Good for	Virtual machines (VM), databases	General-purpose file system access	Binary data, unstructured data

Some terminology, related to object storage:

- Bucket - logical container for objects. Name is globally unique.
- Object - An individual piece of data, stored in a bucket. Contains object data and metadata.
- Versioning - A feature keeping multiple variants of an object in the same bucket.
- Uniform Resource Identifier (URI) - each resource is uniquely identified by a URI.
- Service-level Agreement (SLA) - contract between service provider and client.

Amazon S3 Standard-Infrequent Access storage class SLAs:

- Durability of 99.999999999% across multiple Availability Zones
- Data is resilient in the event of entire Availability Zone being destroyed
- Designed for 99.9% availability

Step 1 - Understand the Problem and Establish Design Scope

- C: Which features should be included?
- I: Bucket creation, Object upload/download, versioning, Listing objects in a bucket
- C: What is the typical data size?
- I: We need to store both massive objects and small objects efficiently
- C: How much data do we store in a year?
- I: 100 petabytes
- C: Can we assume 6 nines of data durbility (99.9999%) and service availability of 4 nines (99.99%)?
- I: Yes, sounds reasonable

Non-functional requirements

- 100 PB of data
- 6 nines of data durability
- 4 nines of service availability
- Storage efficiency. Reduce storage cost while maintaining high reliability and performance

Back-of-the-envelope estimation

Object storage is likely to have bottlenecks in disk capacity or IO per second (IOPS).

Assumptions:

- we have 20% small (less than 1mb), 60% mid-size (1-64mb) and 20% large objects (greater than 64mb),
- One hard disk (SATA, 7200rpm) is capable of doing 100-150 random seeks per second (100-150 IOPS)

Given the assumptions, we can estimate the total number of objects the system can persist.

- Let's use median size per object type to simplify calculation - 0.5mb for small, 32mb for medium, 200mb for large.
- Given 100PB of storage (10^11 MB) and 40% of storage usage results in 0.68bil objects
- If we assume metadata is 1kb, then we need 0.68tb space to store metadata info

Step 2 - Propose High-Level Design and Get Buy-In

Let's explore some interesting properties of object storage before diving into the design:

- Object immutability - objects in object storage are immutable (not the case in other storage systems). We may delete them or replace them, but no update.
- Key-value store - an object URI is its key and we can get its contents by making an HTTP call
- Write once, read many times - data access pattern is writing once and reading many times. According to some Linkedin research, 95% of operations are reads
- Support both small and large objects

Design philosophy of object storage is similar to UNIX - when we save a file, it creates the filename in a data structure, called inode and file data is stored in different disk locations. The inode contains a list of file block pointers, which point to different locations on disk.

When accessing a file, we first fetch its metadata from the inode, prior to fetching the file contents.

Object storage works similarly - metadata store is used for file information, but contents are stored on disk:

Show Image

By separating metadata from file contents, we can scale the different stores independently:

Show Image

High-level design

Show Image

- Load balancer - distributes API requests across service replicas
- API service - Stateless server, orchestrating calls to metadata and object store, as well as IAM service.

- Identity and access management (IAM) - central place for auth, authz, access control.
- Data store - stores and retrieves actual data. Operations are based on object ID (UUID).
- Metadata store - stores object metadata

Uploading an object

Show Image

- Create a bucket named "bucket-to-share" via HTTP PUT request
- API service calls IAM to ensure user is authorized and has write permissions
- API service calls metadata store to create a bucket entry. Once created, success response is returned.
- After bucket is created, HTTP PUT is sent to create an object named "script.txt"
- API service verifies user identity and ensures user has write permissions
- Once validation passes, object payload is sent via HTTP PUT to the data store. Data store persists it and returns a UUID.
- API service calls metadata store to create a new entry with object_id, bucket_id and bucket_name, among other metadata.

Example object upload request:

```
PUT /bucket-to-share/script.txt HTTP/1.1
Host: foo.s3example.org
Date: Sun, 12 Sept 2021 17:51:00 GMT
Authorization: authorization string
Content-Type: text/plain
Content-Length: 4567
x-amz-meta-author: Alex

[4567 bytes of object data]
```

Downloading an object

Buckets have no directory hierarchy, buy we can create a logical hierarchy by concatenating bucket name and object name to simulate a folder structure.

Example GET request for fetching an object:

```
GET /bucket-to-share/script.txt HTTP/1.1
Host: foo.s3example.org
Date: Sun, 12 Sept 2021 18:30:01 GMT
Authorization: authorization string
```

Show Image

- Client sends an HTTP GET request to the load balancer, ie `GET /bucket-to-share/script.txt`
- API service queries IAM to verify the user has correct permissions to read the bucket
- Once validated, UUID of object is retrieved from metadata store
- Object payload is retrieved from data store based on UUID and returned to the client

// sprint 1

Step 3 - Design Deep Dive

Data store

Here's how the API service interacts with the data store:

Show Image

The data store's main components:

Show Image

The data routing service provides a RESTful or gRPC API to access the data node cluster. It is a stateless service, which scales by adding more servers.

It's main responsibilities are:

- querying the placement service to get the best data node to store data
- reading data from data nodes and returning it to the API service

- Writing data to data nodes

The placement service determines which data nodes should store an object. It maintains a virtual cluster map, which determines the physical topology of a cluster.

Show Image

The service also sends heartbeats to all data nodes to determine if they should be removed from the virtual cluster.

Since this is a critical service, it is recommended to maintain a cluster of 5 or 7 replicas, synchronized via Paxos or Raft consensus algorithms.
Eg a 7 node cluster can tolerate 3 nodes failing.

Data nodes store the actual object data.
Reliability and durability is ensured by replicating data to multiple data nodes.

Each data node has a daemon running, which sends heartbeats to the placement service.

The heartbeat includes:

- How many disk drives (HDD or SSD) does the data node manage?
- How much data is stored on each drive?

Data persistence flow

Show Image

- API service forwards the object data to data store
- Data routing service sends the data to the primary data node
- Primary data node saves the data locally and replicates it to two secondary data nodes. Response is sent after successful replication.
- The UUID of the object is returned to the API service.

Caveats:

- Given an object UUID, it's replication group is deterministically chosen by using consistent hashing

- In step 4, the primary data node replicates the object data before returning a response. This favors strong consistency over higher latency.

  Show Image

How data is organized

One simple approach to managing data is to store each object in a separate file.

This works, but is not performant with many small files in a file system:

- Data blocks on HDD are wasted, because every file uses the whole block size. Typical block size is 4kb.
- Many files means many inodes. Operating systems don't deal well with too many inodes and there is also a max inode limit.

These issues can be addressed by merging many small files into bigger ones via a write-ahead log (WAL). Once the file reaches its capacity (typically a few GB), a new file is created:

Show Image

The downside of this approach is that write access to the file needs to be serialized. Multiple cores accessing the same file must wait for each other.
To fix this, we can confine files to specific cores to avoid lock contention.

Object lookup

To support storing multiple objects in the same file, we need to maintain a table, which tells the data node:

- `object_id`
- `filename` where object is stored
- `file_offset` where object starts
- `object_size`

We can deploy this table in a file-based db like RocksDB or a traditional relational database.
Since the access pattern is low write+high read, a relational database works better.

How should we deploy it?
We could deploy the db and scale it separately in a cluster, accessed by all data nodes.

Downsides:

- we'd need to aggressively scale the cluster to serve all requests
- there's additional network latency between data node and db cluster

An alternative is to take advantage of the fact that data nodes are only interested to data related to them,
so we can deploy the relational db within the data node itself.

SQLite is a good option as it's a lightweight file-based relational database.

Updated data persistence flow

Show Image

- API Service sends a request to save a new object
- Data node service appends the new object at the end of a file, named "/data/c"
- A new record for the object is inserted into the object mapping table

Durability

Data durability is an important requirement in our design. In order to achieve 6 nines of durability, every failure case needs to be properly examined.

First problem to address is hardware failures. We can achieve that by replicating data nodes to minimize probability of failure. But in addition to that, we also ought to replicate across different failure domains (cross-rack, cross-dc, separate networks, etc). A critical event can cause multiple hardware failures within the same domain:

Show Image

Assuming annual failure rate of a typical HDD is 0.81%, making three copies gives us 6 nines of durability.

Replicating the data nodes like that grants us the durability we want, but we could also leverage erasure coding to reduce storage costs.

Erasure coding enables us to use parity bits, which allow us to reconstruct lost bits in the event of a failure:

Show Image

Imagine those bits are data nodes. If two of them go down, they can be recovered using the remaining four ones.

There are different erasure coding schemes. In our case, we could use 8+4 erasure coding, split across different failure domains to maximize reliability:

Show Image

Erasure coding enables us to achieve a much lower storage cost (50% improvement) at the expense of access speed due to the data routing service having to collect data from multiple locations:

Show Image

Other caveats:

- Replication requires 200% storage overhead (in case of 3 replicas) vs. 50% via erasure coding
- Erasure coding gives us 11 nines of durability vs 6 nines via replication
- Erasure coding requires more computation to calculate and store parities

In sum, replication is more useful for latency-sensitive applications, whereas erasure coding is attractive for storage cost efficiency and durability.
Erasure coding is also much harder to implement.

Correctness verification

If a disk fails entirely, then the failure is easy to detect. This is less straightforward in the event part of the disk memory gets corrupted.

To detect this, we can use checksums - a hash of the file contents, which can be used to verify the file's integrity.

In our case, we'll store checksums for each file and each object:

Show Image

In the case of erasure coding (8+4), we'll need to fetch each of the 8 pieces of data separately and verify each of their checksums.

// sprint 2

Metadata data model

Table schemas:

Show Image

Queries we need to support:

- Find an object ID by name
- Insert/delete object based on name
- List objects in a bucket sharing the same prefix

There is usually a limit on the number of buckets a user can create, hence, the size of the buckets table is small and can fit into a single db server.
But we still need to scale the server for read throughput.

The object table will probably not fit into a single database server, though. Hence, we can scale the table via sharding:

- Sharding by bucket_id will lead to hotspot issues as a bucket can have billions of objects
- Sharding by bucket_id makes the load more evenly distributed, but our queries will be slow
- We choose sharding by `hash(bucket_name, object_name)` since most queries are based on the object/bucket name.

Even with this sharding scheme, though, listing objects in a bucket will be slow.

Listing objects in a bucket

In a single database, listing an object based on its prefix (looks like a directory) works like this:

```
SELECT * FROM object WHERE bucket_id = "123" AND object_name LIKE `abc/%`
```

This is challenging to fulfill when the database is sharded. To achieve it, we can run the query on every shard and aggregate the results in-memory.
This makes pagination challenging though, since different shards contain a different result size and we need to maintain separate limit/offset for each.

We can leverage the fact that typically object stores are not optimized for listing objects, so we can sacrifice listing performance.
We can also create a denormalized table for listing objects, sharded by bucket ID.
That would make our listing query sufficiently fast as it's isolated to a single database instance.

Object versioning

Versioning works by having another `object_version` column which is of type TIMEUUID, enabling us to sort records based on it.

Each new version produces a new `object_id`:

Show Image

Deleting an object creates a new version with a special `object_id` indicating that the object was deleted. Queries for it return 404:

Show Image

Optimizing uploads of large files

Uploading large files can be optimized by using multipart uploads - splitting a big file into several chunks, uploaded independently:

Show Image

- Client calls service to initiate a multipart upload
- Data store returns an upload ID which uniquely identifies the upload
- Client splits the large file into several chunks, uploaded independently using the upload id
- When a chunk is uploaded, the data store returns an etag, which is a md5 checksum, identifying that upload chunk

- After all parts are uploaded, client sends a complete multipart upload request, which includes upload_id, part numbers and all etags
- Data store reassembles the object from its parts. The process can take a few minutes. After that, success response is returned to the client.

Old parts, which are no longer useful can be removed at this point. We can introduce a garbage collector to deal with it.

Garbage collection

Garbage collection is the process of reclaiming storage space, which is no longer used. There are a few ways data becomes garbage:

- lazy object deletion - object is marked as deleted without actually getting deleted
- orphan data - eg an upload failed mid-flight and old parts need to be deleted
- corrupted data - data which failed checksum verification

The garbage collector is also responsible for reclaiming unused space in replicas.
With replication, data is deleted from both primaries and replicas. With erasure coding (8+4), data is deleted from all 12 nodes.

To facilitate the deletion, we'll use a process called compaction:

- Garbage collector copies objects which are not deleted from "data/b" to "data/d"
- `object_mapping` table is updated once copying is complete using a database transaction
- To avoid making too many small files, compaction is done on files which grow beyond a certain threshold

Show Image

Step 4 - Wrap Up

Things we covered:

- Designing an S3-like object storage
- Comparing differences between object, block and file storages
- Covered uploading, downloading, listing, versioning of objects in a bucket
- Deep dived in the design - data store and metadata store, replication and erasure coding, multipart uploads, sharding

Chapter 26

Real-time Gaming Leaderboard

We are going to design a leaderboard for an online mobile game:

Show Image

Step 1 - Understand the Problem and Establish Design Scope

- C: How is the score calculated for the leaderboard?
- I: User gets a point whenever they win a match.
- C: Are all players included in the leaderboard?
- I: Yes
- C: Is there a time segment, associated with the leaderboard?
- I: Each month, a new tournament starts which starts a new leaderboard.
- C: Can we assume we only care about top 10 users?
- I: We want to display top 10 users, along with position of specific user. If time permits, we can discuss showing users around particular user in the leaderboard.
- C: How many users are in a tournament?
- I: 5mil DAU and 25mil MAU
- C: How many matches are played on average during a tournament?
- I: Each player plays 10 matches per day on average
- C: How do we determine the rank if two players have the same score?
- I: Their rank is the same in that case. If time permits, we can discuss breaking ties.
- C: Does the leaderboard need to be real-time?
- I: Yes, we want to present real-time results or as close as possible to real-time. It is not okay to present batched result history.

Functional requirements

- Display top 10 players on leaderboard
- Show a user's specific rank
- Display users which are four places above and below given user (bonus)

Non-functional requirements

- Real-time updates on scores
- Score update is reflected on the leaderboard in real-time
- General scalability, availability, reliability

Back-of-the-envelope estimation

With 50mil DAU, if the game has an even distribution of players during a 24h period, we'd have an average of 50 users per second.
However, since distribution is typically uneven, we can estimate that the peak online users would be 250 users per second.

QPS for users scoring a point - given 10 games per day on average, 50 users/s * 10 = 500 QPS. Peak QPS = 2500.

QPS for fetching the top 10 leaderboard - assuming users open that once a day on average, QPS is 50.

Step 2 - Propose High-Level Design and Get Buy-In

API Design

The first API we need is one to update a user's score:

```
POST /v1/scores
```

This API takes two params - `user_id` and `points` scored for winning a game.

This API should only be accessible to game servers, not end clients.

Next one is for getting the top 10 players of the leaderboard:

```
GET /v1/scores
```

Example response:

```
{
  "data": [
    {
      "user_id": "user_id1",
      "user_name": "alice",
      "rank": 1,
      "score": 12543
    },
    {
      "user_id": "user_id2",
      "user_name": "bob",
      "rank": 2,
      "score": 11500
    }
  ],
  ...
  "total": 10
}
```

You can also get the score of a particular user:

```
GET /v1/scores/{:user_id}
```

Example response:

```
{
    "user_info": {
        "user_id": "user5",
        "score": 1000,
        "rank": 6,
    }
}
```

High-level architecture

Show Image

- When a player wins a game, client sends a request to the game service
- Game service validates if win is valid and calls the leaderboard service to update the player's score

- Leaderboard service updates the user's score in the leaderboard store
- Player makes a call to leaderboard service to fetch leaderboard data, eg top 10 players and given player's rank

An alternative design which was considered is the client updating their score directly within the leaderboard service:

Show Image

This option is not secure as it's susceptible to man-in-the-middle attacks. Players can put a proxy and change their score as they please.

One additional caveat is that for games, where the game logic is managed by the server, cliets don't need to call the server explicitly to record their win.
Servers do it automatically for them based on the game logic.

One additional consideration is whether we should put a message queue between the game server and the leaderboard service. This would be useful if other services are interested in game results, but that is not an explicit requirement in the interview so far, hence it's not included in the design:

Show Image

Data models

Let's discuss the options we have for storing leaderboard data - relational DBs, Redis, NoSQL.

The NoSQL solution is discussed in the deep dive section.

Relational database solution

If the scale doesn't matter and we don't have that many users, a relational DB serves our quite well.

We can start from a simple leaderboard table, one for each month (personal note - this doesn't make sense. You can just add a `month` column and avoid the headache of maintaining new tables each month):

Show Image

There is additional data to include in there, but that is irrelevant to the queries we'd run, so it's omitted.

What happens when a user wins a point?

Show Image

If a user doesn't exist in the table yet, we need to insert them first:

```
INSERT INTO leaderboard (user_id, score) VALUES ('mary1934', 1);
```

On subsequent calls, we'd just update their score:

```
UPDATE leaderboard set score=score + 1 where user_id='mary1934';
```

How do we find the top players of a leaderboard?

Show Image

We can run the following query:

```
SELECT (@rownum := @rownum + 1) AS rank, user_id, score
FROM leaderboard
ORDER BY score DESC;
```

This is not performant though as it makes a table scan to order all records in the database table.

We can optimize it by adding an index on `score` and using the `LIMIT` operation to avoid scanning everything:

```
SELECT (@rownum := @rownum + 1) AS rank, user_id, score
FROM leaderboard
ORDER BY score DESC
LIMIT 10;
```

This approach, however, doesn't scale well if the user is not at the top of the leaderboard and you'd want to locate their rank.

Redis solution

We want to find a solution, which works well even for millions of players without having to fallback on complex database queries.

Redis is an in-memory data store, which is fast as it works in-memory and has a suitable data structure to serve our needs - sorted set.

A sorted set is a data structure similar to sets in programming languages, which allows you to keep a data structure sorted by a given criteria. Internally, it is implemented using a hash-map to maintain mapping between key (user_id) and value (score) and a skip list which maps scores to users in sorted order:

Show Image

How does a skip list work?

- It is a linked list which allows for fast search
- It consists of a sorted linked list and multi-level indexes

  Show Image

This structure enables us to quickly search for specific values when the data set is large enough. In the example below (64 nodes), it requires traversing 62 nodes in a base linked list to find the given value and 11 nodes in the skip-list case:

Show Image

Sorted sets are more performant than relational databases as the data is kept sorted at all times at the price of O(logN) add and find operation.

In contract, here's an example nested query we need to run to find the rank of a given user in a relational DB:

```
SELECT *,(SELECT COUNT(*) FROM leaderboard lb2
WHERE lb2.score >= lb1.score) RANK
FROM leaderboard lb1
WHERE lb1.user_id = {:user_id};
```

What operations do we need to operate our leaderboard in Redis?

- `ZADD` - insert the user into the set if they don't exist. Otherwise, update the score. O(logN) time complexity.
- `ZINCRBY` - increment the score of a user by given amount. If user doesn't exist, score starts at zero. O(logN) time complexity.
- `ZRANGE/ZREVRANGE` - fetch a range of users, sorted by their score. We can specify order (ASC/DESC), offset and result size. O(logN+M) time complexity where M is result size.
- `ZRANK/ZREVRANK` - Fetch the position (rank) of given user in ASC/DESC order. O(logN) time complexity.

What happens when a user scores a point?

```
ZINCRBY leaderboard_feb_2021 1 'mary1934'
```

There's a new leaderboard created every month while old ones are moved to historical storage.

What happens when a user fetches top 10 players?

```
ZREVRANGE leaderboard_feb_2021 0 9 WITHSCORES
```

Example result:

```
[(user2,score2),(user1,score1),(user5,score5)...]
```

What about user fetching their leaderboard position?

Show Image

This can be easily achieved by the following query, given that we know a user's leaderboard position:

```
ZREVRANGE leaderboard_feb_2021 357 365
```

A user's position can be fetched using `ZREVRANK <user-id>`.

Let's explore what our storage requirements are:

- Assuming worst-case scenario of all 25mil MAU participating in the game for a given month
- ID is 24-character string and score is 16-bit integer, we need 26 bytes * 25mil = ~650MB of storage
- Even if we double the storage cost due to the overhead of the skip list, this would still easily fit in a modern redis cluster

Another non-functional requirement to consider is supporting 2500 updates per second. This is well within a single Redis server's capabilities.

Additional caveats:

- We can spin up a Redis replica to avoid losing data when a redis server crashes
- We can still leverage Redis persistence to not lose data in the event of a crash
- We'll need two supporting tables in MySQL to fetch user details such as username, display name, etc as well as store when eg a user won a game
- The second table in MySQL can be used to reconstruct leaderboard when there is an infrastructure failure
- As a small performance optimization, we could cache the user details of top 10 players as they'd be frequently accessed

Step 3 - Design Deep Dive

To use a cloud provider or not

We can either choose to deploy and manage our own services or use a cloud provider to manage them for us.

If we choose to manage the services our selves, we'll use redis for leaderboard data, mysql for user profile and potentially a cache for user profile if we want to scale the database:

Show Image

Alternatively, we could use cloud offerings to manage a lot of the services for us. For example, we can use AWS API Gateway to route API calls to AWS Lambda functions:

Show Image

AWS Lambda enables us to run code without managing or provisioning servers ourselves. It runs only when needed and scales automatically.

Exmaple user scoring a point:

Show Image

Example user retrieving leaderboard:

Show Image

Lambdas are an implementation of a serverless architecture. We don't need to manage scaling and environment setup.

Author recommends going with this approach if we build the game from the ground up.

Scaling Redis

With 5mil DAU, we can get away with a single Redis instance from both a storage and QPS perspective.

However, if we imagine userbase grows 10x to 500mil DAU, then we'd need 65gb for storage and QPS goes to 250k.

Such scale would require sharding.

One way to achieve it is by range-partitioning the data:

Show Image

In this example, we'll shard based on user's score. We'll maintain the mapping between user_id and shard in application code.
We can do that either via MySQL or another cache for the mapping itself.

To fetch the top 10 players, we'd query the shard with the highest scores (`[900-1000]`).

To fetch a user's rank, we'll need to calculate the rank within the user's shard and add up all users with higher scores in other shards.
The latter is a O(1) operation as total records per shard can quickly be accessed via the info keyspace command.

Alternatively, we can use hash partitioning via Redis Cluster. It is a proxy which distributes data across redis nodes based on partitioning similar to consistent hashing, but not exactly the same:

Show Image

Calculating the top 10 players is challenging with this setup. We'll need to get the top 10 players of each shard and merge the results in the application:

Show Image

There are some limitations with the hash partitioning:

- If we need to fetch top K users, where K is high, latency can increase as we'll need to fetch a lot of data from all the shards
- Latency increases as the number of partitions grows
- There is no straightforward approach to determine a user's rank

Due to all this, the author leans towards using fixed partitions for this problem.

Other caveats:

- A best practice is to allocate twice as much memory as required for write-heavy redis nodes to accommodate snapshots if required
- We can use a tool called Redis-benchmark to track the performance of a redis setup and make data-driven decisions

Alternative solution: NoSQL

An alternative solution to consider is using an appropriate NoSQL database optimized for:

- heavy writes
- effectively sorting items within the same partition by score

DynamoDB, Cassandra or MongoDB are all good fits.

In this chapter, the author has decided to use DynamoDB. It is a fully-managed NoSQL database, which offers reliable performance and great scalability. It also enables usage of global secondary indexes when we need to query fields not part of the primary key.

Show Image

Let's start from a table for storing a leaderboard for a chess game:

Show Image

This works well, but doesn't scale well if we need to query anything by score. Hence, we can put the score as a sort key:

Show Image

Another problem with this design is that we're partitioning by month. This leads to a hotspot partition as the latest month will be unevenly accessed compared to the others.

We could use a technique called write sharding, where we append a partition number for each key, calculated via `user_id % num_partitions`:

Show Image

An important trade-off to consider is how many partitions we should use:

- The more partitions there are, the higher the write scalability
- However, read scalability suffers as we need to query more partitions to collect aggregate results

Using this approach requires that we use the "scatter-gather" technique we saw earlier, which grows in time complexity as we add more partitions:

Show Image

To make a good evaluation on the number of partitions, we'd need to do some benchmarking.

This NoSQL approach still has one major downside - it is hard to calculate the specific rank of a user.

If we have sufficient scale to require us to shard, we could then perhaps tell users what "percentile" of scores they're in.

A cron job can periodically run to analyze score distributions, based on which a user's percentile is determined, eg:

```
10th percentile = score < 100
20th percentile = score < 500
...
90th percentile = score < 6500
```

Step 4 - Wrap Up

Other things to discuss if time permits:

- Faster retrieval - We can cache the user object via a Redis hash with mapping `user_id -> user object`. This enables faster retrieval vs. querying the database.
- Breaking ties - When two players have the same score, we can break the tie by sorting them based on last played game.
- System failure recovery - In the event of a large-scale Redis outage, we can recreate the leaderboard by going through the MySQL WAL entries and recreate it via an ad-hoc script

---

Chapter 27

Payment System

We'll design a payment system in this chapter, which underpins all of modern e-commerce.

A payment system is used to settle financial transactions, transferring monetary value.

Step 1 - Understand the Problem and Establish Design Scope

- C: What kind of payment system are we building?
- I: A payment backend for an e-commerce system, similar to Amazon.com. It handles everything related to money movement.
- C: What payment options are supported - Credit cards, PayPal, bank cards, etc?
- I: The system should support all these options in real life. For the purposes of the interview, we can use credit card payments.
- C: Do we handle credit card processing ourselves?
- I: No, we use a third-party provider like Stripe, Braintree, Square, etc.
- C: Do we store credit card data in our system?
- I: Due to compliance reasons, we do not store credit card data directly in our systems. We rely on third-party payment processors.
- C: Is the application global? Do we need to support different currencies and international payments?
- I: The application is global, but we assume only one currency is used for the purposes of the interview.
- C: How many payment transactions per day do we support?
- I: 1mil transactions per day.
- C: Do we need to support the payout flow to eg payout to payers each month?
- I: Yes, we need to support that
- C: Is there anything else I should pay attention to?
- I: We need to support reconciliations to fix any inconsistencies in communicating with internal and external systems.

Functional requirements

- Pay-in flow - payment system receives money from customers on behalf of merchants
- Pay-out flow - payment system sends money to sellers around the world

Non-functional requirements

- Reliability and fault-tolerance. Failed payments need to be carefully handled
- A reconciliation between internal and external systems needs to be setup.

Back-of-the-envelope estimation

The system needs to process 1mil transactions per day, which is 10 transactions per second.

This is not a high throughput for any database system, so it's not the focus of this interview.

Step 2 - Propose High-Level Design and Get Buy-In

At a high-level, we have three actors, participating in money movement:

Show Image

Pay-in flow

Here's the high-level overview of the pay-in flow:

Show Image

- Payment service - accepts payment events and coordinates the payment process. It typically also does a risk check using a third-party provider for AML violations or criminal activity.
- Payment executor - executes a single payment order via the Payment Service Provider (PSP). Payment events may contain several payment orders.
- Payment service provider (PSP) - moves money from one account to another, eg from buyer's credit card account to e-commerce site's bank account.
- Card schemes - organizations that process credit card operations, eg Visa MasterCard, etc.
- Ledger - keeps financial record of all payment transactions.
- Wallet - keeps the account balance for all merchants.

Here's an example pay-in flow:

- user clicks "place order" and a payment event is sent to the payment service
- payment service stores the event in its database
- payment service calls the payment executor for all payment orders, part of that payment event
- payment executor stores the payment order in its database
- payment executor calls external PSP to process the credit card payment
- After the payment executor processes the payment, the payment service updates the wallet to record how much money the seller has
- wallet service stores updated balance information in its database
- payment service calls the ledger to record all money movements

APIs for payment service

```
POST /v1/payments
{
  "buyer_info": {...},
  "checkout_id": "some_id",
  "credit_card_info": {...},
  "payment_orders": [{...}, {...}, {...}]
}
```

Example `payment_order`:

```
{
  "seller_account": "SELLER_IBAN",
  "amount": "3.15",
  "currency": "USD",
  "payment_order_id": "globally_unique_payment_id"
}
```

Caveats:

- The `payment_order_id` is forwarded to the PSP to deduplicate payments, ie it is the idempotency key.
- The amount field is `string` as `double` is not appropriate for representing monetary values.

```
GET /v1/payments/{:id}
```

This endpoint returns the execution status of a single payment, based on the `payment_order_id`.

Payment service data model

We need to maintain two tables - `payment_events` and `payment_orders`.

For payments, performance is typically not an important factor. Strong consistency, however, is.

Other considerations for choosing the database:

- Strong market of DBAs to hire to administer the databaseS
- Proven track-record where the database has been used by other big financial institutions
- Richness of supporting tools
- Traditional SQL over NoSQL/NewSQL for its ACID guarantees

Here's what the `payment_events` table contains:

- `checkout_id` - string, primary key
- `buyer_info` - string (personal note - prob a foreign key to another table is more appropriate)
- `seller_info` - string (personal note - same remark as above)
- `credit_card_info` - depends on card provider
- `is_payment_done` - boolean

Here's what the `payment_orders` table contains:

- `payment_order_id` - string, primary key
- `buyer_account` - string
- `amount` - string
- `currency` - string
- `checkout_id` - string, foreign key
- `payment_order_status` - enum (`NOT_STARTED`, `EXECUTING`, `SUCCESS`, `FAILED`)
- `ledger_updated` - boolean
- `wallet_updated` - boolean

Caveats:

- there are many payment orders, linked to a given payment event
- we don't need the `seller_info` for the pay-in flow. That's required on pay-out only
- `ledger_updated` and `wallet_updated` are updated when the respective service is called to record the result of a payment
- payment transitions are managed by a background job, which checks updates of in-flight payments and triggers an alert if a payment is not processed in a reasonable timeframe

Double-entry ledger system

The double-entry accounting mechanism is key to any payment system. It is a mechanism of tracking money movements by always applying money operations to two accounts, where one's account balance increases (credit) and the other decreases (debit):

Account	Debit	Credit
buyer	$1	
seller		$1

Sum of all transaction entries is always zero. This mechanism provides end-to-end traceability of all money movements within the system.

Hosted payment page

To avoid storing credit card information and having to comply with various heavy regulations, most companies prefer utilizing a widget, provided by PSPs, which store and handle credit card payments for you:

Show Image

Pay-out flow

The components of the pay-out flow are very similar to the pay-in flow.

Main differences:

- money is moved from e-commerce site's bank account to merchant's bank account
- we can utilize a third-party account payable provider such as Tipalti
- There's a lot of bookkeeping and regulatory requirements to handle with regards to pay-outs as well

Step 3 - Design Deep Dive

This section focuses on making the system faster, more robust and secure.

PSP Integration

If our system can directly connect to banks or card schemes, payment can be made without a PSP.
These kinds of connections are very rare and uncommon, typically done at large companies which can justify the investment.

If we go down the traditional route, a PSP can be integrated in one of two ways:

- Through API, if our payment system can collect payment information
- Through a hosted payment page to avoid dealing with payment information regulations

Here's how the hosted payment page workflow works:

Show Image

- User clicks "checkout" button in the browser
- Client calls the payment service with the payment order information
- After receiving payment order information, the payment service sends a payment registration request to the PSP.
- The PSP receives payment info such as currency, amount, expiration, etc, as well as a UUID for idempotency purposes. Typically the UUID of the payment order.
- The PSP returns a token back which uniquely identifies the payment registration. The token is stored in the payment service database.
- Once token is stored, the user is served with a PSP-hosted payment page. It is initialized using the token as well as a redirect URL for success/failure.
- User fills in payment details on the PSP page, PSP processes payment and returns the payment status
- User is now redirected back to the redirectURL. Example redirect url - `https://your-company.com/?tokenID=JIOUIQ123NSF&payResult=X324FSa`
- Asynchronously, the PSP calls our payment service via a webhook to inform our backend of the payment result
- Payment service records the payment result based on the webhook received

Reconciliation

The previous section explains the happy path of a payment. Unhappy paths are detected and reconciled using a background reconciliation process.

Every night, the PSP sends a settlement file which our system uses to compare the external system's state against our internal system's state.

Show Image

This process can also be used to detect internal inconsistencies between eg the ledger and the wallet services.

Mismatches are handled manually by the finance team. Mismatches are handled as:

- classifiable, hence, it is a known mismatch which can be adjusted using a standard procedure
- classifiable, but can't be automated. Manually adjusted by the finance team
- unclassifiable. Manually investigated and adjusted by the finance team

Handling payment processing delays

There are cases, where a payment can take hours to complete, although it typically takes seconds.

This can happen due to:

- a payment being flagged as high-risk and someone has to manually review it
- credit card requires extra protection, eg 3D Secure Authentication, which requires extra details from card holder to complete

These situations are handled by:

- waiting for the PSP to send us a webhook when a payment is complete or polling its API if the PSP doesn't provide webhooks
- showing a "pending" status to the user and giving them a page, where they can check-in for payment updates. We could also send them an email once their payment is complete

Communication among internal services

There are two types of communication patterns services use to communicate with one another - synchronous and asynchronous.

Synchronous communication (ie HTTP) works well for small-scale systems, but suffers as scale increases:

- low performance - request-response cycle is long as more services get involved in the call chain
- poor failure isolation - if PSPs or any other service fails, user will not receive a response
- tight coupling - sender needs to know the receiver
- hard to scale - not easy to support sudden increase in traffic due to not having a buffer

Asynchronous communication can be divided into two categories.

Single receiver - multiple receivers subscribe to the same topic and messages are processed only once:

Show Image

Multiple receivers - multiple receivers subscribe to the same topic, but messages are forwarded to all of them:

Show Image

Latter model works well for our payment system as a payment can trigger multiple side effects, handled by different services.

In a nutshell, synchronous communication is simpler but doesn't allow services to be autonomous.
Async communication trades simplicity and consistency for scalability and resilience.

Handling failed payments

Every payment system needs to address failed payments. Here are some of the mechanism we'll use to achieve that:

- Tracking payment state - whenever a payment fails, we can determine whether to retry/refund based on the payment state.
- Retry queue - payments which we'll retry are published to a retry queue
- Dead-letter queue - payments which have terminally failed are pushed to a dead-letter queue, where the failed payment can be debugged and inspected.

  Show Image

Exactly-once delivery

We need to ensure a payment gets processed exactly-once to avoid double-charging a customer.

An operation is executed exactly-once if it is executed at-least-once and at-most-once at the same time.

To achieve the at-least-once guarantee, we'll use a retry mechanism:

Show Image

Here are some common strategies on deciding the retry intervals:

- immediate retry - client immediately sends another request after failure
- fixed intervals - wait a fixed amount of time before retrying a payment
- incremental intervals - incrementally increase retry interval between each retry
- exponential back-off - double retry interval between subsequent retries
- cancel - client cancels the request. This happens when the error is terminal or retry threshold is reached

As a rule of thumb, default to an exponential back-off retry strategy. A good practice is for the server to specify a retry interval using a `Retry-After` header.

An issue with retries is that the server can potentially process a payment twice:

- client clicks the "pay button" twice, hence, they are charged twice
- payment is successfully processed by PSP, but not by downstream services (ledger, wallet). Retry causes the payment to be processed by the PSP again

To address the double payment problem, we need to use an idempotency mechanism - a property that an operation applied multiple times is processed only once.

From an API perspective, clients can make multiple calls which produce the same result. Idempotency is managed by a special header in the request (eg `idempotency-key`), which is typically a UUID.

Show Image

Idempotency can be achieved using the database's mechanism of adding unique key constraints:

- server attempts to insert a new row in the database
- the insertion fails due to a unique key constraint violation
- server detects that error and instead returns the existing object back to the client

Idempotency is also applied at the PSP side, using the nonce, which was previously discussed. PSPs will take care to not process payments with the same nonce twice.

Consistency

There are several stateful services called throughout a payment's lifecycle - PSP, ledger, wallet, payment service.

Communication between any two services can fail.
We can ensure eventual data consistency between all services by implementing exactly-once processing and reconciliation.

If we use replication, we'll have to deal with replication lag, which can lead to users observing inconsistent data between primary and replica databases.

To mitigate that, we can serve all reads and writes from the primary database and only utilize replicas for redundancy and fail-over.
Alternatively, we can ensure replicas are always in-sync by utilizing a consensus algorithm such as Paxos or Raft.
We could also use a consensus-based distributed database such as YugabyteDB or CockroachDB.

Payment security

Here are some mechanisms we can use to ensure payment security:

- Request/response eavesdropping - we can use HTTPS to secure all communication
- Data tampering - enforce encryption and integrity monitoring
- Man-in-the-middle attacks - use SSL \w certificate pinning
- Data loss - replicate data across multiple regions and take data snapshots
- DDoS attack - implement rate limiting and firewall
- Card theft - use tokens instead of storing real card information in our system
- PCI compliance - a security standard for organizations which handle branded credit cards
- Fraud - address verification, card verification value (CVV), user behavior analysis, etc

Step 4 - Wrap Up

Other talking points:

- Monitoring and alerting
- Debugging tools - we need tools which make it easy to understand why a payment has failed
- Currency exchange - important when designing a payment system for international use
- Geography - different regions might have different payment methods
- Cash payment - very common in places like India and Brazil
- Google/Apple Pay integration

Chapter 28

Digital Wallet

Payment platforms usually have a wallet service, where they allow clients to store funds within the application, which they can withdraw later.

You can also use it to pay for goods & services or transfer money to other users, who use the digital wallet service. That can be faster and cheaper than doing it via normal payment rails.

Show Image

Step 1 - Understand the Problem and Establish Design Scope

- C: Should we only focus on transfers between digital wallets? Should we support any other operations?
- I: Let's focus on transfers between digital wallets for now.
- C: How many transactions per second does the system need to support?
- I: Let's assume 1mil TPS
- C: A digital wallet has strict correctness requirements. Can we assume transactional guarantees are sufficient?
- I: Sounds good
- C: Do we need to prove correctness?
- I: We can do that via reconciliation, but that only detects discrepancies vs. showing us the root cause for them. Instead, we want to be able to replay data from the beginning to reconstruct the history.
- C: Can we assume availability requirement is 99.99%?
- I: Yes
- C: Do we need to take foreign exchange into consideration?
- I: No, it's out of scope

Here's what we have to support in summary:

- Support balance transfers between two accounts
- Support 1mil TPS
- Reliability is 99.99%
- Support transactions
- Support reproducibility

Back-of-the-envelope estimation

A traditional relational database, provisioned in the cloud can support ~1000 TPS.

In order to reach 1mil TPS, we'd need 1000 database nodes. But if each transfer has two legs, then we actually need to support 2mil TPS.

One of our design goals would be to increase the TPS a single node can handle so that we can have less database nodes.

Per-node TPS	Node Number
100	20,000
1,000	2,000
10,000	200

Step 2 - Propose High-Level Design and Get Buy-In

API Design

We only need to support one endpoint for this interview:

```
POST /v1/wallet/balance_transfer - transfers balance from one wallet to another
```

Request parameters - from_account, to_account, amount (string to not lose precision), currency, transaction_id (idempotency key).

Sample response:

```
{
    "status": "success"
    "transaction_id": "01589980-2664-11ec-9621-0242ac130002"
}
```

In-memory sharding solution

Our wallet application maintains account balances for every user account.

One good data structure to represent this is a `map<user_id, balance>`, which can be implemented using an in-memory Redis store.

Since one redis node cannot withstand 1mil TPS, we need to partition our redis cluster into multiple nodes.

Example partitioning algorithm:

```
String accountID = "A";
Int partitionNumber = 7;
Int myPartition = accountID.hashCode() % partitionNumber;
```

Zookeeper can be used to store the number of partitions and addresses of redis nodes as it's a highly-available configuration storage.

Finally, a wallet service is a stateless service responsible for carrying out transfer operations. It can easily scale horizontally:

Show Image

Although this solution addresses scalability concerns, it doesn't allow us to execute balance transfers atomically.

Distributed transactions

One approach for handling transactions is to use the two-phase commit protocol on top of standard, sharded relational databases:

Show Image

Here's how the two-phase commit (2PC) protocol works:

Show Image

- Coordinator (wallet service) performs read and write operations on multiple databases as normal
- When application is ready to commit the transaction, coordinator asks all databases to prepare it
- If all databases replied with a "yes", then the coordinator asks the databases to commit the transaction.
- Otherwise, all databases are asked to abort the transaction

Downsides to the 2PC approach:

- Not performant due to lock contention
- The coordinator is a single point of failure

Distributed transaction using Try-Confirm/Cancel (TC/C)

TC/C is a variation of the 2PC protocol, which works with compensating transactions:

- Coordinator asks all databases to reserve resources for the transaction
- Coordinator collects replies from DBs - if yes, DBs are asked to try-confirm. If no, DBs are asked to try-cancel.

One important difference between TC/C and 2PC is that 2PC performs a single transaction, whereas in TC/C, there are two independent transactions.

Here's how TC/C works in phases:

Phase	Operation	A	C
1	Try	Balance change: -$1	Do nothing
2	Confirm	Do nothing	Balance change: +$1
	Cancel	Balance change: +$1	Do Nothing

Phase 1 - try:

Show Image

- coordinator starts local transaction in A's DB to reduce A's balance by 1$
- C's DB is given a NOP instruction, which does nothing

Phase 2a - confirm:

Show Image

- if both DBs replied with "yes", confirm phase starts.
- A's DB receives NOP, whereas C's DB is instructed to increase C's balance by 1$ (local transaction)

Phase 2b - cancel:

Show Image

- If any of the operations in phase 1 fails, the cancel phase starts.
- A's DB is instructed to increase A's balance by 1$, C's DB receives NOP

Here's a comparison between 2PC and TC/C:

	First Phase	**Second Phase: success**	**Second Phase: fail**
2PC	transactions are not done yet	Commit/Cancel all transactions	Cancel all transactions
TC/C	All transactions are completed - committed or canceled	Execute new transactions if needed	Reverse the already committed transaction

TC/C is also referred to as a distributed transaction by compensation. High-level operation is handled in the business logic.

Other properties of TC/C:

- database agnostic, as long as database supports transactions
- Details and complexity of distributed transactions need to be handled in the business logic

TC/C Failure modes

If the coordinator dies mid-flight, it needs to recover its intermediary state. That can be done by maintaining phase status tables, atomically updated within the database shards:

Show Image

What does that table contain:

- ID and content of distributed transaction
- status of try phase - not sent, has been sent, response received
- second phase name - confirm or cancel
- status of second phase

- out-of-order flag (explained later)

One caveat when using TC/C is that there is a brief moment where the account states are inconsistent with each other while a distributed transaction is in-flight:

Show Image

This is fine as long as we always recover from this state and that users cannot use the intermediary state to eg spend it.
This is guaranteed by always executing deductions prior to additions.

Try phase choices	Account A	Account C
Choice 1	-$1	NOP
Choice 2 (invalid)	NOP	+$1
Choice 3 (invalid)	-$1	+$1

Note that choice 3 from table above is invalid because we cannot guarantee atomic execution of transactions across different databases without relying on 2PC.

One edge-case to address is out of order execution:

Show Image

It is possible that a database receives a cancel operation, before receiving a try. This edge case can be handled by adding an out of order flag in our phase status table.
When we receive a try operation, we first check if the out of order flag is set and if so, a failure is returned.

Distributed transaction using Saga

Another popular approach is using Sagas - a standard for implementing distributed transactions with microservice architectures.

Here's how it works:

- all operations are ordered in a sequence. All operations are independent in their own databases.

- operations are executed from first to last
- when an operation fails, the entire process starts to roll back until the beginning with compensating operations

Show Image

How do we coordinate the workflow? There are two approaches we can take:

- Choreography - all services involved in a saga subscribe to the related events and do their part in the saga
- Orchestration - a single coordinator instructs all services to do their jobs in the correct order

The challenge of using choreography is that business logic is split across multiple service, which communicate asynchronously.
The orchestration approach handles complexity well, so it is typically the preferred approach in a digital wallet system.

Here's a comparison between TC/C and Saga:

	TC/C	**Saga**
Compensating action	In Cancel phase	In rollback phase
Central coordination	Yes	Yes (orchestration mode)
Operation execution order	any	linear
Parallel execution possibility	Yes	No (linear execution)
Could see the partial inconsistent status	Yes	Yes
Application or database logic	Application	Application

The main difference is that TC/C is parallelizable, so our decision is based on the latency requirement - if we need to achieve low latency, we should go for the TC/C approach.

Regardless of the approach we take, we still need to support auditing and replaying history to recover from failed states.

Event sourcing

In real-life, a digital wallet application might be audited and we have to answer certain questions:

- Do we know the account balance at any given time?

- How do we know the historical and current balances are correct?
- How do we prove the system logic is correct after a code change?

Event sourcing is a technique which helps us answer these questions.

It consists of four concepts:

- command - intended action from the real world, eg transfer 1$ from account A to B. Need to have a global order, due to which they're put into a FIFO queue.
  - commands, unlike events, can fail and have some randomness due to eg IO or invalid state.
  - commands can produce zero or more events
  - event generation can contain randomness such as external IO. This will be revisited later
- event - historical facts about events which occured in the system, eg "transferred 1$ from A to B".
  - unlike commands, events are facts that have happened within our system
  - similar to commands, they need to be ordered, hence, they're enqueued in a FIFO queue
- state - what has changed as a result of an event. Eg a key-value store between account and their balances.
- state machine - drives the event sourcing process. It mainly validates commands and applies events to update the system state.
  - the state machine should be deterministic, hence, it shouldn't read external IO or rely on randomness.

Show Image

Here's a dynamic view of event sourcing:

Show Image

For our wallet service, the commands are balance transfer requests. We can put them in a FIFO queue, such as Kafka:

Show Image

Here's the full picture:

Show Image

- state machine reads commands from the command queue
- balance state is read from the database
- command is validated. If valid, two events for each of the accounts is generated
- next event is read and applied by updating the balance (state) in the database

The main advantage of using event sourcing is its reproducibility. In this design, all state update operations are saved as immutable history of all balance changes.

Historical balances can always be reconstructed by replaying events from the beginning. Because the event list is immutable and the state machine is deterministic, we are guaranteed to succeed in replaying any of the intermediary states.

Show Image

All audit-related questions asked in the beginning of the section can be addressed by relying on event sourcing:

- Do we know the account balance at any given time? - events can be replayed from the start until the point which we are interested in
- How do we know the historical and current balances are correct? - correctness can be verified by recalculating all events from the start
- How do we prove the system logic is correct after a code change? - we can run different versions of the code against the events and verify their results are identical

Answering client queries about their balance can be addressed using the CQRS architecture - there can be multiple read-only state machines which are responsible for querying the historical state, based on the immutable events list:

Show Image

Step 3 - Design Deep Dive

In this section we'll explore some performance optimizations as we're still required to scale to 1mil TPS.

High-performance event sourcing

The first optimization we'll explore is to save commands and events into local disk store instead of an external store such as Kafka.

This avoids the network latency and also, since we're only doing appends, that operation is generally fast for HDDs.

The next optimization is to cache recent commands and events in-memory in order to save the time of loading them back from disk.

At a low-level, we can achieve the aforementioned optimizations by leveraging a command called mmap, which stores data in local disk as well as cache it in-memory:

Show Image

The next optimization we can do is also store state in the local file system using SQLite - a file-based local relational database. RocksDB is also another good option.

For our purposes, we'll choose RocksDB because it uses a log-structured merge-tree (LSM), which is optimized for write operations. Read performance is optimized via caching.

Show Image

To optimize the reproducibility, we can periodically save snapshots to disk so that we don't have to reproduce a given state from the very beginning every time. We could store snapshots as large binary files in distributed file storage, eg HDFS:

Show Image

Reliable high-performance event sourcing

All the optimizations done so far are great, but they make our service stateful. We need to introduce some form of replication for reliability purposes.

Before we do that, we should analyze what kind of data needs high reliability in our system:

- state and snapshot can always be regenerated by reproducing them from the events list. Hence, we only need to guarantee the event list reliability.
- one might think we can always regenerate the events list from the command list, but that is not true, since commands are non-deterministic.
- conclusion is that we need to ensure high reliability for the events list only

In order to achieve high reliability for events, we need to replicate the list across multiple nodes. We need to guarantee:

- that there is no data loss
- the relative order of data within a log file remains the same across replicas

To achieve this, we can employ a consensus algorithm, such as Raft.

With Raft, there is a leader who is active and there are followers who are passive. If a leader dies, one of the followers picks up. As long as more than half of the nodes are up, the system continues running.

Show Image

With this approach, all nodes update the state, based on the events list. Raft ensures leader and followers have the same events list.

Distributed event sourcing

So far, we've managed to design a system which has high single-node performance and is reliable.

Some limitations we have to tackle:

- The capacity of a single raft group is limited. At some point, we need to shard the data and implement distributed transactions
- In the CQRS architecture, the request/response flow is slow. A client would need to periodically poll the system to learn when their wallet has been updated

Polling is not real-time, hence, it can take a while for a user to learn about an update in their balance. Also, it can overload the query services if the polling frequency is too high:

Show Image

To mitigate the system load, we can introduce a reverse proxy, which sends commands on behalf of the user and polls for response on their behalf:

Show Image

This alleviates the system load as we could fetch data for multiple users using a single request, but it still doesn't solve the real-time receipt requirement.

One final change we could do is make the read-only state machines push responses back to the reverse proxy once it's available. This can give the user the sense that updates happen real-time:

Show Image

Finally, to scale the system even further, we can shard the system into multiple raft groups, where we implement distributed transactions on top of them using an orchestrator either via TC/C or Sagas:

Show Image

Here's an example lifecycle of a balance transfer request in our final system:

- User A sends a distributed transaction to the Saga coordinator with two operations - `A-1` and `C+1`.

- Saga coordinator creates a record in the phase status table to trace the status of the transaction
- Coordinator determines which partitions it needs to send commands to.
- Partition 1's raft leader receives the `A-1` command, validates it, converts it to an event and replicates it across other nodes in the raft group
- Event result is synchronized to the read state machine, which pushes a response back to the coordinator
- Coordinator creates a record indicating that the operation was successful and proceeds with the next operation - `C+1`
- Next operation is executed similarly to the first one - partition is determined, command is sent, executed, read state machine pushes back a response
- Coordinator creates a record indicating operation 2 was also successful and finally informs the client of the result

Step 4 - Wrap Up

Here's the evolution of our design:

- We started from a solution using an in-memory Redis. The problem with this approach is that it is not durable storage.
- We moved on to using relational databases, on top of which we execute distributed transactions using 2PC, TC/C or distributed saga.
- Next, we introduced event sourcing in order to make all the operations auditable
- We started by storing the data into external storage using external database and queue, but that's not performant
- We proceeded to store data in local file storage, leveraging the performance of append-only operations. We also used caching to optimize the read path
- The previous approach, although performant, wasn't durable. Hence, we introduced Raft consensus with replication to avoid single points of failure
- We also adopted CQRS with a reverse proxy to manage a transaction's lifecycle on behalf of our users
- Finally, we partitioned our data across multiple raft groups, which are orchestrated using a distributed transaction mechanism - TC/C or distributed saga

---

Chapter 29

Stock Exchange

We'll design an electronic stock exchange in this chapter.

Its basic function is to efficiently match buyers and sellers.

Major stock exchanges are NYSE, NASDAQ, among others.

Show Image

Step 1 - Understand the Problem and Establish Design scope

- C: Which securities are we going to trade? Stocks, options or futures?
- I: Only stocks for simplicity
- C: Which order types are supported - place, cancel, replace? What about limit, market, conditional orders?
- I: We need to support placing and canceling an order. We need to only consider limit orders for the order type.
- C: Does the system need to support after hours trading?
- I: No, just normal trading hours
- C: Could you describe the exchange's basic functions?
- I: Clients can place or cancel limit orders and receive matched trades in real-time. They should be able to see the order book in real time.
- C: What's the scale of the exchange?
- I: Tens of thousands of users trading at the same time and ~100 symbols. Billions of orders per day. We need to also support risk checks for compliance.
- C: What kind of risk checks?
- I: Let's do simple risk checks - eg limiting a user to trade only 1mil apple stocks in a day
- C: How about user wallet engagement?
- I: We need to ensure clients have sufficient funds before placing orders. Funds meant for pending orders need to be withheld until order is finalized.

Non-functional requirements

The scale mentioned by the interviewer hints that we are to design a small to medium scale exchange.
We need to also ensure flexibility to support more symbols and users in the future.

Other non-functional requirements:

- Availability - At least 99.99%. Downtime can harm reputation

- Fault tolerance - fault tolerance and a fast recovery mechanism are needed to limit the impact of a production incident
- Latency - Round-trip latency should be in the ms level with focus on 99th percentile. Persistently high 99p latency causes bad experience for a handful or users.
- Security - We should have an account management system. For legal compliance, we need to support KYC to verify user identity. We should also protect against DDoS for public resources.

Back-of-the-envelope estimation

- 100 symbols, 1bil orders per day
- Normal trading hours are from 09:30 to 16:00 (6.5h)
- QPS = 1bil / 6.5 / 3600 = 43000
- Peak QPS = 5*QPS = 215000
- Trading volume is significantly higher when the market opens

Step 2 - Propose High-Level Design and Get Buy-In

Business Knowledge 101

Let's discuss some basic concepts, related to an exchange.

A broker mediates interactions between an exchange and end users - Robinhood, Fidelity, etc.

Institutional clients trade in large quantities using specialized trading software. They need specialized treatment.
Eg order splitting when trading in large volumes to avoid impacting the market.

Types of orders:

- Limit - buy or sell at a fixed price. It might not find a match immediately or it might be partially matched.
- Market - doesn't specify a price. Executed at the current market price immediately.

Prices:

- Bid - highest price a buyer is willing to buy a stock
- Ask - lowest price a seller is willing to sell a stock

The US market has three tiers of price quotes - L1, L2, L3.

L1 market data contains best bid/ask prices and quantities:

Show Image

L2 includes more price levels:

Show Image

L3 shows levels and queued quantity at each level:

Show Image

A candlestick shows the market open and close price, as well as the highest and lowest prices in the given interval:

Show Image

FIX is a protocol for exchanging securities transaction information, used by most vendors. Example securities transaction:

```
8=FIX.4.2 | 9=176 | 35=8 | 49=PHLX | 56=PERS | 52=20071123-05:30:00.000 |
11=ATOMNOCCC9990900 | 20=3 | 150=E | 39=E | 55=MSFT | 167=CS | 54=1 | 38=15 | 40=2
| 44=15 | 58=PHLX EQUITY TESTING | 59=0 | 47=C | 32=0 | 31=0 | 151=15 | 14=0 | 6=0
| 10=128 |
```

High-level design

Show Image

Trade flow:

- Client places order via trading interface
- Broker sends the order to the exchange
- Order enters exchange through client gateway, which validates, rate limits, authenticates, etc. Order is forwarded to order manager.
- Order manager performs risk checks based on rules set by the risk manager
- After passing risk checks, order manager verifies there are sufficient funds in the wallet for the order
- Order is sent to matching engine. When match is found, matching engine emits two executions (called fills) for buy and sell. Both orders are sequenced so that they're deterministic.
- Executions are returned to the client.

Market data flow (M1-M3):

- matching engine generates a stream of executions, sent to the market data publisher
- Market data publisher constructs the candlestick charts and sends them to the data service
- Market data is stored in specialized storage for real-time analytics. Brokers connect to the data service for timely market data.

Reporter flow (R1-R2):

- reporter collects all necessary reporting fields from orders and executions and writes them to DB
- reporting fields - client_id, price, quantity, order_type, filled_quantity, remaining_quantity

Trading flow is on the critical path, whereas the rest of the flows are not, hence, latency requirements differ between them.

Trading flow

The trading flow is on the critical path, hence, it should be highly optimized for low latency.

The matching engine is at its heart, also called the cross engine. Primary responsibilities:

- Maintain the order book for each symbol - a list of buy/sell orders for a symbol.
- Match buy and sell orders - a match results in two executions (fills), with one each for the buy and sell sides. This function must be fast and accurate
- Distribute the execution stream as market data
- Matches must be produced in a deterministic order. Foundational for high availability

Next is the sequencer - it is the key component making the matching engine deterministic by stamping each inbound order and outbound fill with a sequence ID.

Show Image

We stamp inbound orders and outbound fills for several reasons:

- timeliness and fairness
- fast recovery/replay
- exactly-once guarantee

Conceptually, we could use Kafka as our sequencer since it's effectively an inbound and outbound message queue. However, we're going to implement it ourselves in order to achieve lower latency.

The order manager manages the orders state. It also interacts with the matching engine - sending orders and receiving fills.

The order manager's responsibilities:

- Sends orders for risk checks - eg verifying user's trade volume is less than 1mil
- Checks the order against the user wallet and verifies there are sufficient funds to execute it
- It sends the order to the sequencer and on to the matching engine. To reduce bandwidth, only necessary order information is passed to the matching engine
- Executions (fills) are received back from the sequencer, where they are then send to the brokers via the client gateway

The main challenge with implementing the order manager is the state transition management. Event sourcing is one viable solution (discussed in deep dive).

Finally, the client gateway receives orders from users and sends them to the order manager. Its responsibilities:

Show Image

Since the client gateway is on the critical path, it should stay lightweight.

There can be multiple client gateways for different clients. Eg a colo engine is a trading engine server, rented by the broker in the exchange's data center:

Show Image

Market data flow

The market data publisher receives executions from the matching engine and builds the order book/candlestick charts from the execution stream.

That data is sent to the data service, which is responsible for showing the aggregated data to subscribers:

Show Image

Reporting flow

The reporter is not on the critical path, but it is an important component nevertheless.

Show Image

It is responsible for trading history, tax reporting, compliance reporting, settlements, etc. Latency is not a critical requirement for the reporting flow. Accuracy and compliance are more important.

API Design

Clients interact with the stock exchange via the brokers to place orders, view executions, market data, download historical data for analysis, etc.

We use a RESTful API for communication between the client gateway and the brokers.

For institutional clients, a proprietary protocol is used to satisfy their low-latency requirements.

Create order:

```
POST /v1/order
```

Parameters:

- symbol - the stock symbol. String

- side - buy or sell. String
- price - the price of the limit order. Long
- orderType - limit or market (we only support limit orders in our design). String
- quantity - the quantity of the order. Long

Response:

- id - the ID of the order. Long
- creationTime - the system creation time of the order. Long
- filledQuantity - the quantity that has been successfully executed. Long
- remainingQuantity - the quantity still to be executed. Long
- status - new/canceled/filled. String
- rest of the attributes are the same as the input parameters

Get execution:

```
GET /execution?symbol={:symbol}&orderId={:orderId}&startTime={:startTime}&endTime=
{:endTime}
```

Parameters:

- symbol - the stock symbol. String
- orderId - the ID of the order. Optional. String
- startTime - query start time in epoch [11]. Long
- endTime - query end time in epoch. Long

Response:

- executions - array with each execution in scope (see attributes below). Array
- id - the ID of the execution. Long
- orderId - the ID of the order. Long
- symbol - the stock symbol. String
- side - buy or sell. String
- price - the price of the execution. Long
- orderType - limit or market. String
- quantity - the filled quantity. Long

Get order book:

```
GET /marketdata/orderBook/L2?symbol={:symbol}&depth={:depth}
```

Parameters:

- symbol - the stock symbol. String
- depth - order book depth per side. Int

Response:

- bids - array with price and size. Array
- asks - array with price and size. Array

get candlesticks:

```
GET /marketdata/candles?symbol={:symbol}&resolution={:resolution}&startTime=
{:startTime}&endTime={:endTime}
```

Parameters:

- symbol - the stock symbol. String
- resolution - window length of the candlestick chart in seconds. Long
- startTime - start time of the window in epoch. Long
- endTime - end time of the window in epoch. Long

Response:

- candles - array with each candlestick data (attributes listed below). Array
- open - open price of each candlestick. Double
- close - close price of each candlestick. Double
- high - high price of each candlestick. Double
- low - low price of each candlestick. Double

Data models

There are three main types of data in our exchange:

- Product, order, execution
- order book
- candlestick chart

Product, order, execution

Products describe the attributes of a traded symbol - product type, trading symbol, UI display symbol, etc.

This data doesn't change frequently, it is primarily used for rendering in a UI.

An order represents an instruction for a buy/sell order. Executions are outbound matched result.

Here's the data model:

Show Image

We encounter orders and executions in all of our three flows:

- in the critical path, they are processed in-memory for high performance. They are stored and recovered from the sequencer.
- The reporter writes orders and executions to the database for reporting use-cases
- Executions are forwarded to market data to reconstruct the order book and candlestick chart

Order book

The order book is a list of buy/sell orders for an instrument, organized by price level.

An efficient data structure for this model, needs to satisfy:

- constant lookup time - getting volume at price level or between price levels
- fast add/execute/cancel operations
- query best bid/ask price
- iterate through price levels

Example order book execution:

Show Image

After fulfilling this large order, the price increases as the bid/ask spread widens.

Example order book implementation in pseudo code:

```
class PriceLevel{
    private Price limitPrice;
    private long totalVolume;
    private List<Order> orders;
}

class Book<Side> {
    private Side side;
    private Map<Price, PriceLevel> limitMap;
}

class OrderBook {
    private Book<Buy> buyBook;
    private Book<Sell> sellBook;
    private PriceLevel bestBid;
    private PriceLevel bestOffer;
    private Map<OrderID, Order> orderMap;
}
```

For a more efficient implementation, we can use a doubly-linked list instead of a standard list:

- Placing a new order is O(1), because we're adding an order to the tail of the list.
- Matching an order is O(1), because we are deleting an order from the head
- Canceling an order means deleting an order from the order book. We utilize `orderMap` for O(1) lookup and O(1) delete (due to the `Order` having a reference to the previous element in the list).

  Show Image

This data structure is also used in the market data services to reconstruct the order book.

Candlestick chart

The candlestick data is calcualated within the market data services based on processing orders in a time interval:

```
class Candlestick {
    private long openPrice;
    private long closePrice;
    private long highPrice;
    private long lowPrice;
    private long volume;
    private long timestamp;
    private int interval;
}

class CandlestickChart {
    private LinkedList<Candlestick> sticks;
}
```

Some optimizations to avoid consuming too much memory:

- Use pre-allocated ring buffers to hold sticks to reduce the allocation number
- Limit the number of sticks in memory and persist the rest to disk

We'll use an in-memory columnar database (eg KDB) for real-time analytics. After market close, data is persisted in historical database.

Step 3 - Design Deep Dive

One interesting thing to be aware of about modern exchanges is that unlike most other software, they typically run everything on one gigantic server.

Let's explore the details.

Performance

For an exchange, it is very important to have good overall latency for all percentiles.

How can we reduce latency?

- Reduce the number of tasks on the critical path
- Shorten the time spent on each task by reducing network/disk usage and/or reducing task execution time

To achieve the first goal, we're stripped the critical path from all extraneous responsibility, even logging is removed to achieve optimal latency.

If we follow the original design, there are several bottlenecks - network latency between services and disk usage of the sequencer.

With such a design we can achieve tens of milliseconds end to end latency. We want to achieve tens of microseconds instead.

Hence, we'll put everything on one server and processes are going to communicate via mmap as an event store:

Show Image

Another optimization is using an application loop (while loop executing mission-critical tasks), pinned to the same CPU to avoid context switching:

Show Image

Another side effect of using an application loop is that there is no lock contention - multiple threads fighting for the same resource.

Let's now explore how mmap works - it is a UNIX syscall, which maps a file on disk to an application's memory.

One trick we can use is creating the file in `/dev/shm`, which stands for "shared memory". Hence, we have no disk access at all.

Event sourcing

Event sourcing is discussed in-depth in the digital wallet chapter. Reference it for all the details.

In a nutshell, instead of storing current states, we store immutable state transitions:

Show Image

- On the left - traditional schema
- On the right - event source schema

Here's how our design looks like thus far:

Show Image

- external domain interacts with our client gateway using the FIX protocol
- Order manager receives the new order event, validates it and adds it to its internal state. Order is then sent to matching core
- If order is matched, the `OrderFilledEvent` is generated and sent over mmap
- Other components subscribe to the event store and do their part of the processing

One additional optimizations - all components hold a copy of the order manager, which is packaged as a library to avoid extra calls for managing orders

The sequencer in this design, changes to not be an event store, but be a single writer, sequencing events before forwarding them to the event store:

Show Image

High availability

We aim for 99.99% availability - only 8.64s of downtime per day.

To achieve that, we have to identify single-point-of-failures in the exchange architecture:

- setup backup instances of critical services (eg matching engine) which are on stand-by
- aggressively automate failure detection and failover to the backup instance

Stateless services such as the client gateway can easily be horizontally scaled by adding more servers.

For stateful components, we can process inbound events, but not publish outbound events if we're not the leader:

Show Image

To detect the primary replica being down, we can send heartbeats to detect that its non-functional.

This mechanism only works within the boundary of a single server.
If we want to extend it, we can setup an entire server as hot/warm replica and failover in case of failure.

To replicate the event store across the replicas, we can use reliable UDP for faster communication.

Fault tolerance

What if even the warm instances go down? It is a low probability event but we should be ready for it.

Large tech companies tackle this problem by replicating core data to data centers in multiple cities to mitigate eg natural disasters.

Questions to consider:

- If the primary instance is down, how and when do we failover to the backup instance?
- How do we choose the leader among the backup instances?
- What is the recovery time needed (RTO - recovery time objective)?
- What functionalities need to be recovered? Can our system operate under degraded conditions?

How to address these:

- System can be down due to a bug (affecting primary and replicas), we can use chaos engineering to surface edge-cases and disastrous outcomes like these
- Initially though, we could perform failovers manually until we gather sufficient knowledge about the system's failure modes
- leader-election can be used (eg Raft) to determine which replica becomes the leader in the event of the primary going down

Example of how replication works across different servers:

Show Image

Example leader-election terms:

Show Image

For details on how Raft works, check this out

Finally, we need to also consider loss tolerance - how much data can we lose before things get critical?
This will determine how often we backup our data.

For a stock exchange, data loss is unacceptable, so we have to backup data often and rely on raft's replication to reduce probability of data loss.

Matching algorithms

Slight detour on how matching works via pseudo code:

```
Context handleOrder(OrderBook orderBook, OrderEvent orderEvent) {
    if (orderEvent.getSequenceId() != nextSequence) {
        return Error(OUT_OF_ORDER, nextSequence);
    }

    if (!validateOrder(symbol, price, quantity)) {
        return ERROR(INVALID_ORDER, orderEvent);
    }

    Order order = createOrderFromEvent(orderEvent);
    switch (msgType):
        case NEW:
            return handleNew(orderBook, order);
        case CANCEL:
            return handleCancel(orderBook, order);
        default:
            return ERROR(INVALID_MSG_TYPE, msgType);

}

Context handleNew(OrderBook orderBook, Order order) {
    if (BUY.equals(order.side)) {
        return match(orderBook.sellBook, order);
    } else {
        return match(orderBook.buyBook, order);
    }
}

Context handleCancel(OrderBook orderBook, Order order) {
    if (!orderBook.orderMap.contains(order.orderId)) {
        return ERROR(CANNOT_CANCEL_ALREADY_MATCHED, order);
    }

    removeOrder(order);
    setOrderStatus(order, CANCELED);
    return SUCCESS(CANCEL_SUCCESS, order);
}

Context match(OrderBook book, Order order) {
    Quantity leavesQuantity = order.quantity - order.matchedQuantity;
    Iterator<Order> limitIter = book.limitMap.get(order.price).orders;
    while (limitIter.hasNext() && leavesQuantity > 0) {
        Quantity matched = min(limitIter.next.quantity, order.quantity);
        order.matchedQuantity += matched;
        leavesQuantity = order.quantity - order.matchedQuantity;
        remove(limitIter.next);
```

```
        generateMatchedFill();
    }
    return SUCCESS(MATCH_SUCCESS, order);
}
```

This matching algorithm uses the FIFO algorithm for determining which orders at a price level to match.

Determinism

Functional determinism is guaranteed via the sequencer technique we used.

The actual time when the event happens doesn't matter:

Show Image

Latency determinism is something we have to track. We can calculate it based on monitoring 99 or 99.99 percentile latency.

Things which can cause latency spikes are garbage collector events in eg Java.

Market data publisher optimizations

The market data publisher receives matched results from the matching engine and rebuilds the order book and candlestick charts based on them.

We only keep part of the candlesticks as we don't have infinite memory. Clients can choose how much granular info they want. More granular info might require a higher price:

Show Image

A ring buffer (aka circular buffer) is a fixed-size queue with the head connected to the tail. The space is preallocated to avoid allocations. The data structure is also lock-free.

Another technique to optimize the ring buffer is padding, which ensures the sequence number is never in a cache line with anything else.

Distribution fairness of market data and multicast

We need to ensure subscribers receive the data at the same time since if one receives data before another, that gives them crucial market insight, which they can use to manipulate the market.

To achieve this, we can use multicast using reliable UDP when publishing data to subscribers.

Data can be transported via the internet in three ways:

- Unicast - one source, one destination
- Broadcast - one source to entire subnetwork
- Multicast - one source to a set of hosts on different subnetworks

In theory, by using multicast, all subscribers should receive the data at the same time.

UDP, however, is unreliable and the data might not reach everyone. It can be enhanced with retransmissions, however.

Colocation

Exchanges offer brokers the ability to colocate their servers in the same data center as the exchange.

This reduces the latency drastically and can be considered a VIP service.

Network Security

DDoS is a challenge for exchanges as there are some internet-facing services. Here's our options:

- Isolate public services and data from private services, so DDoS attacks don't impact the most important clients
- Use a caching layer to store data which is infrequently updated
- Harden URLs against DDoS, eg prefer `https://my.website.com/data/recent` vs. `https://my.website.com/data?from=123&to=456`, because the former is more cacheable
- Effective allowlist/blocklist mechanism is needed.
- Rate limiting can be used to mitigate DDoS

Step 4 - Wrap Up

Other interesting notes:

- not all exchanges rely on putting everything on one big server, but some still do
- modern exchanges rely more on cloud infrastructure and also on automatic market makers (AMM) to avoid maintaining an order book

---